zPSC Calibration and Test Manual

Michael Capotosto

May 2026

Document Revision: Rev. 1
Source Version: 931217e
Commit: 931217e-dirty

Contents

Chapter 1

Introduction

Chapter 2

Cable Assemblies and Harnesses

2.1 Overview

This chapter provides a brief overview of the makeup of various cable assemblies used in the sections to follow. Some cables are off-the-shelf cables with gender changers installed on one end. Others are made up of commercial cable assemblies with fly leads that must connect to breakout boards.

Some off the shelf fiber and network patch cables will be required.

2.1.1 2-Channel DCCT Cable

This cable consists of a Molex Micro-D15 cable, 72" long, fly lead, connecting to a Winford HD26 breakout board.

- 1x Molex 834229018 72" Micro-D15 Fly Lead Cable
- 1x Winford BRKEGD26HDF-T HD26F Breakout Board

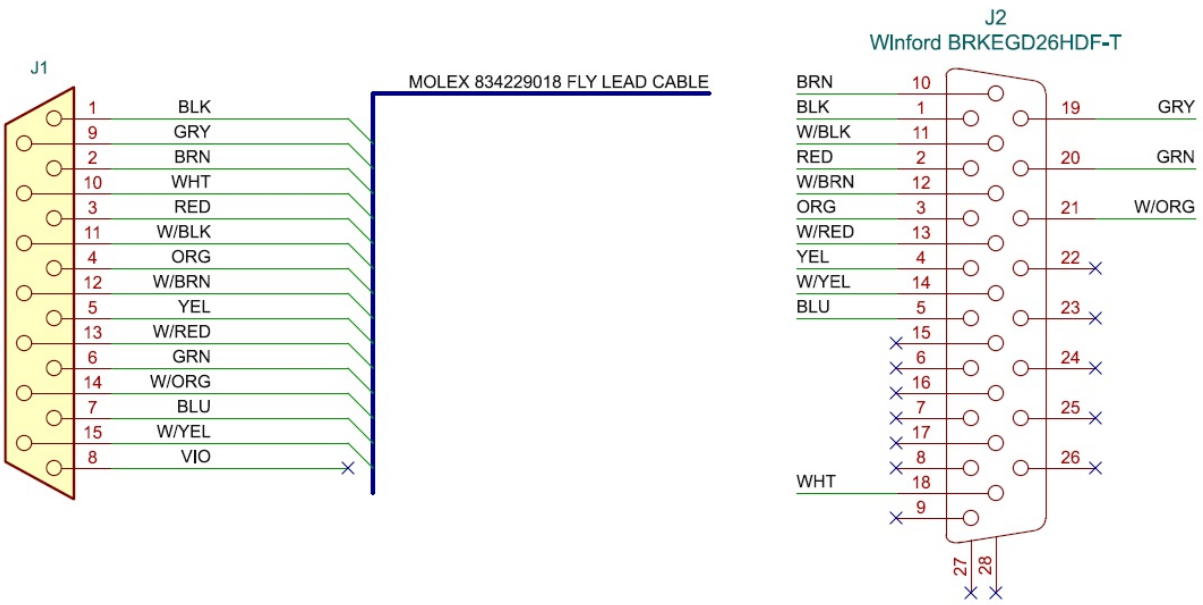


Figure 2.1: Pinout of 2-Channel microD15 to HD26 Cable

2.1.2 4-Channel DCCT Cable

This cable assembly consists of two parts:

- 1x Amphenol CS-DSDHD26MF0-005 1.52m HD26 M/F Cable
- 1x NorComp Inc. GCHDLP26F26F HD26 F/F Gender Changer

2.1.3 Power Supply Channels 1 through 4 ATE Interface Cables

This cable assembly consists of two parts:

- 4x Molex 83424-9062 Micro-D25 to DB25
- 4x L-com DGB25F DB25F/F Gender Changer

2.1.4 SFP Transceivers

Quantities depend on unit type and test workstation configuration. One SFP port is used for timing/event code tests on all units.

If testing FOFB, two more SFPs are required - the NSLS-II test stand uses one active copper and one fiber; though the simplest test setup would use two active copper SFPs.

- Fiber SFP: ATGBICS AFBR-57D7APZ-C or equivalent (8.5G+ SFP+, 850nm MM)
- Active Copper SFP: Finisar FCLF8522P2BTL or equivalent

2.1.5 Other Miscellaneous Cables

Other miscelaneous cables needed include:

- CAT6 Network cables
- OM3 LC/LC Duplex Fiber Patch Cables (e.g. FS.com SKU 40233)
- IEC C13 to NEMA 5-15P Power Cords
- Assorted discrete wires for DC power, signal, 16-18AWG

Chapter 3

ATE Connections

3.1 Overview

On initial setup, there are a few fixed connections that need to be made to the ATE:

- Current Shunt: Connect this to a resistance standard for calibrations, or to a wire loop if not being used for current measurement
- $\pm 15V$ Supply: Connect this to a split supply, draws less than 400mA under full load
- Network: Connect to the local test network
- SD Card must be configured once, but does not require any regular changes
- Jumpers: Install jumpers in J8-9, J12-13, J19-20, JJ23-24, J30-31, J34-35, J41-42, and J45-J46 for normal operations.

For every device under test:

- Channel 1, 2, 3, and 4 Power Supply Cables: These cables connect to each of the micro-D Power Supply I/O connectors on the rear of the PSC
- Tuning Boards: Tuning boards must be installed to match the loads being tested.
- DCCT Cable must be connected for every PSC. There are two cables - one for 4-Channel units, and another for 2-Channel units.

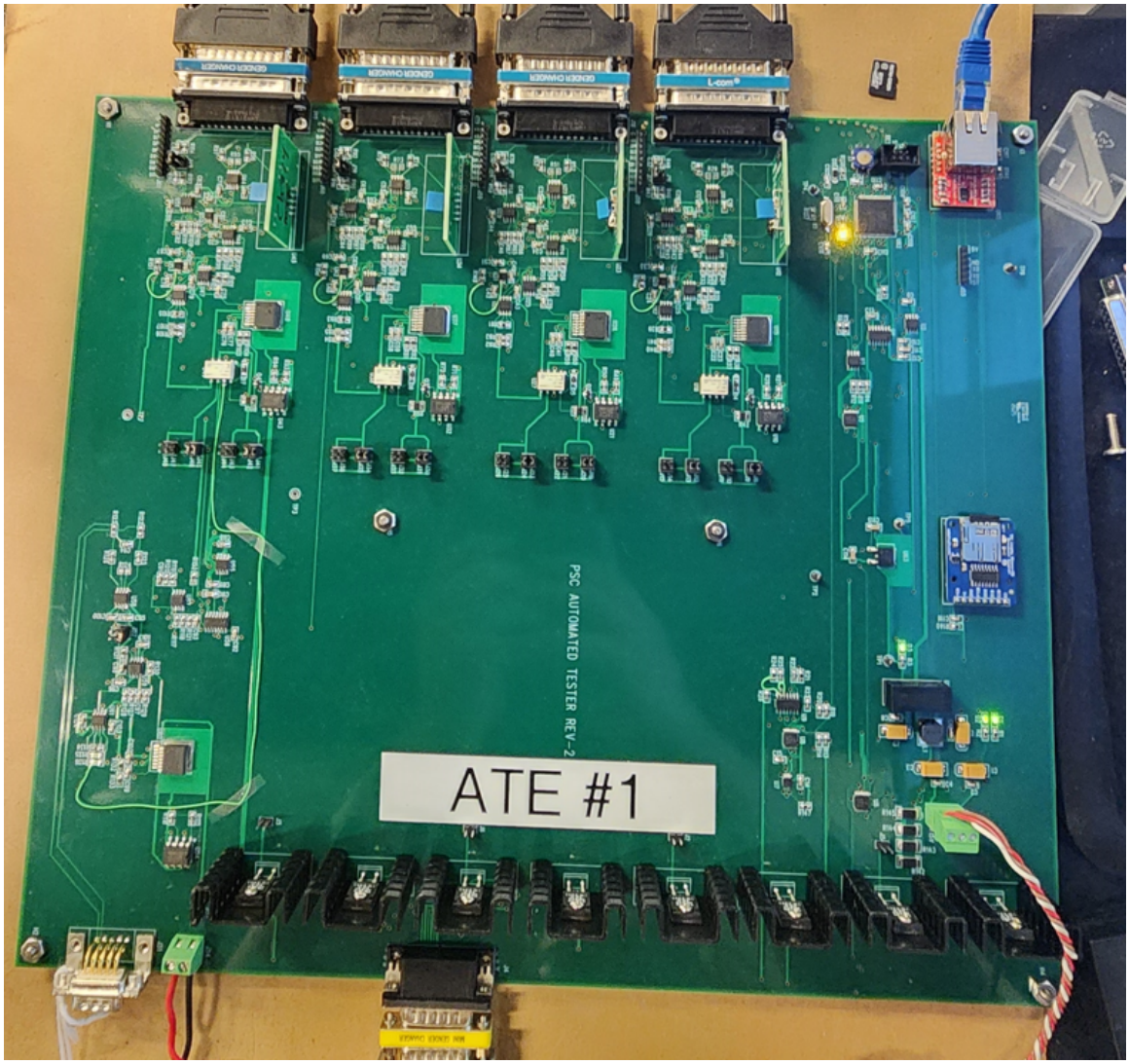
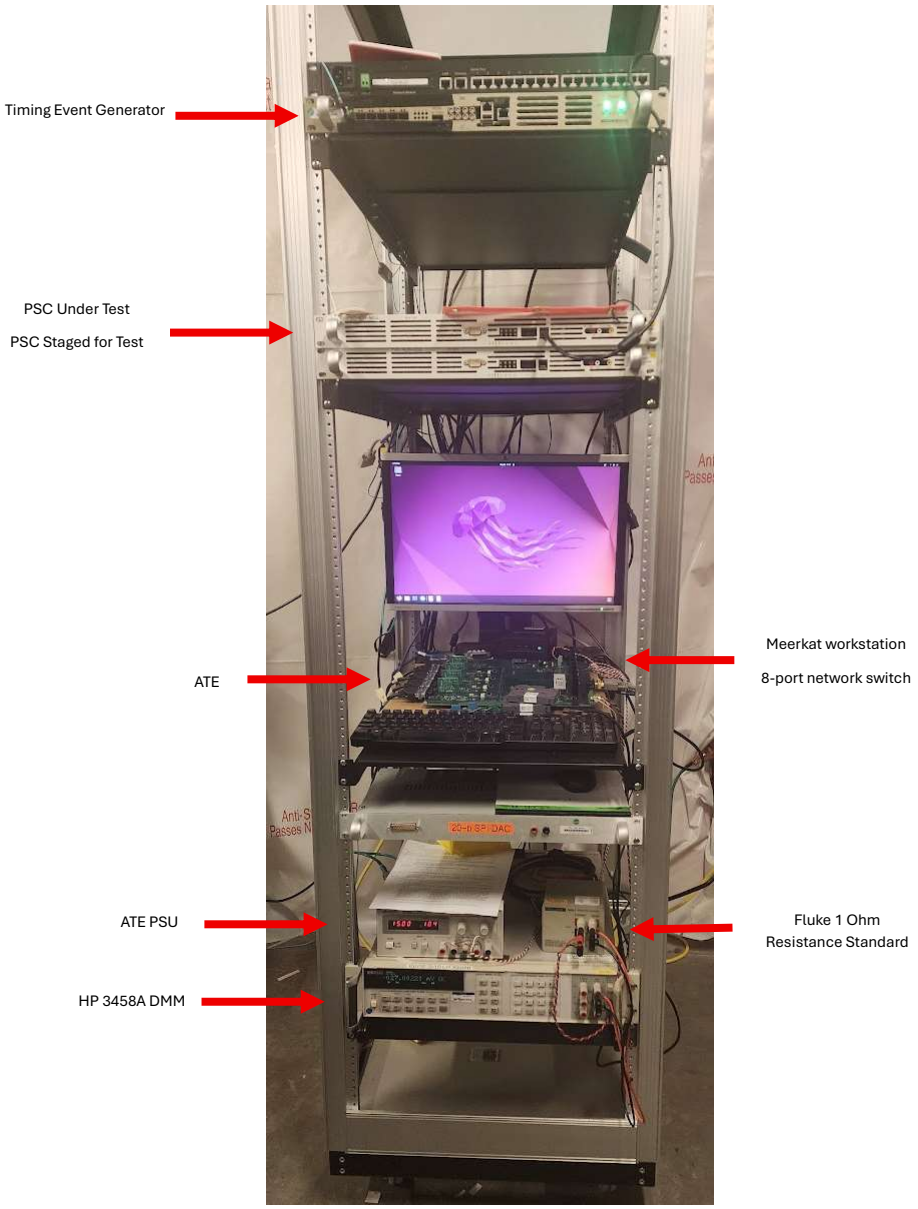


Figure 3.1: ATE #1

3.1.1 ATE Test Rack at NSLS-II



3.2 Connections

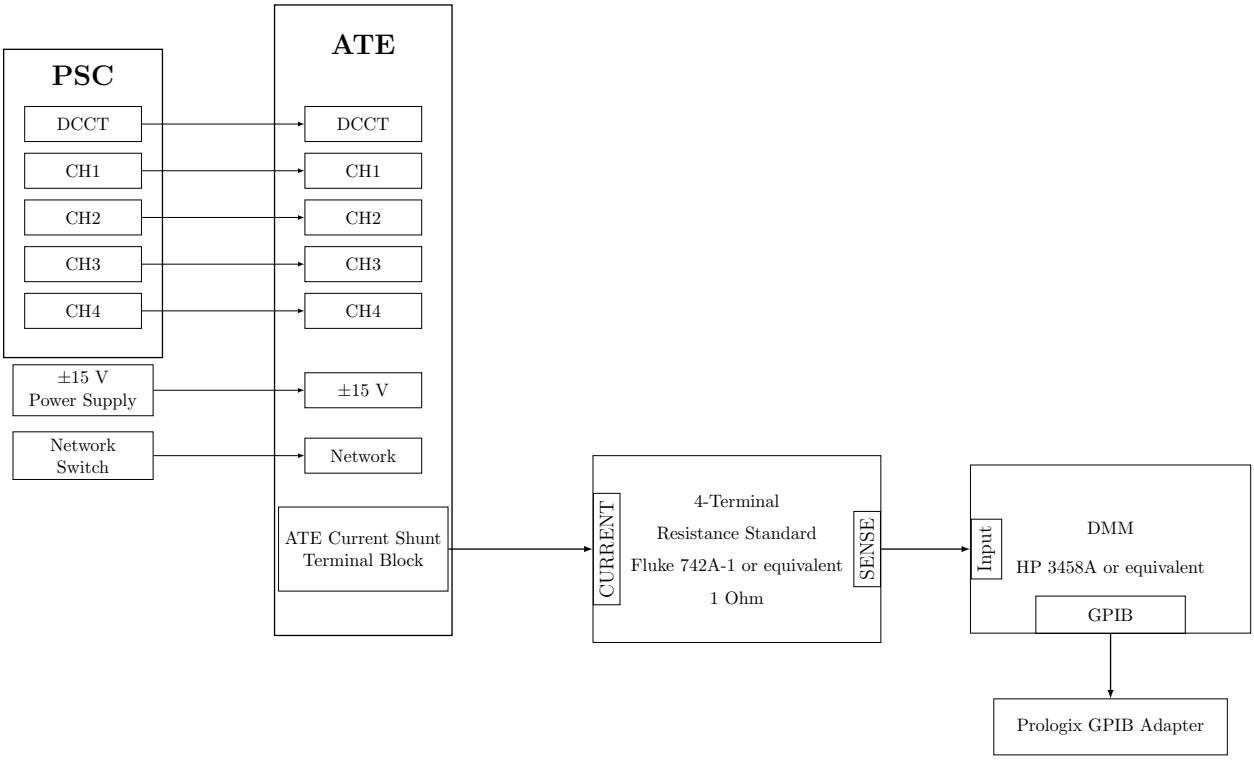


Figure 3.2: ATE Connection Block Diagram

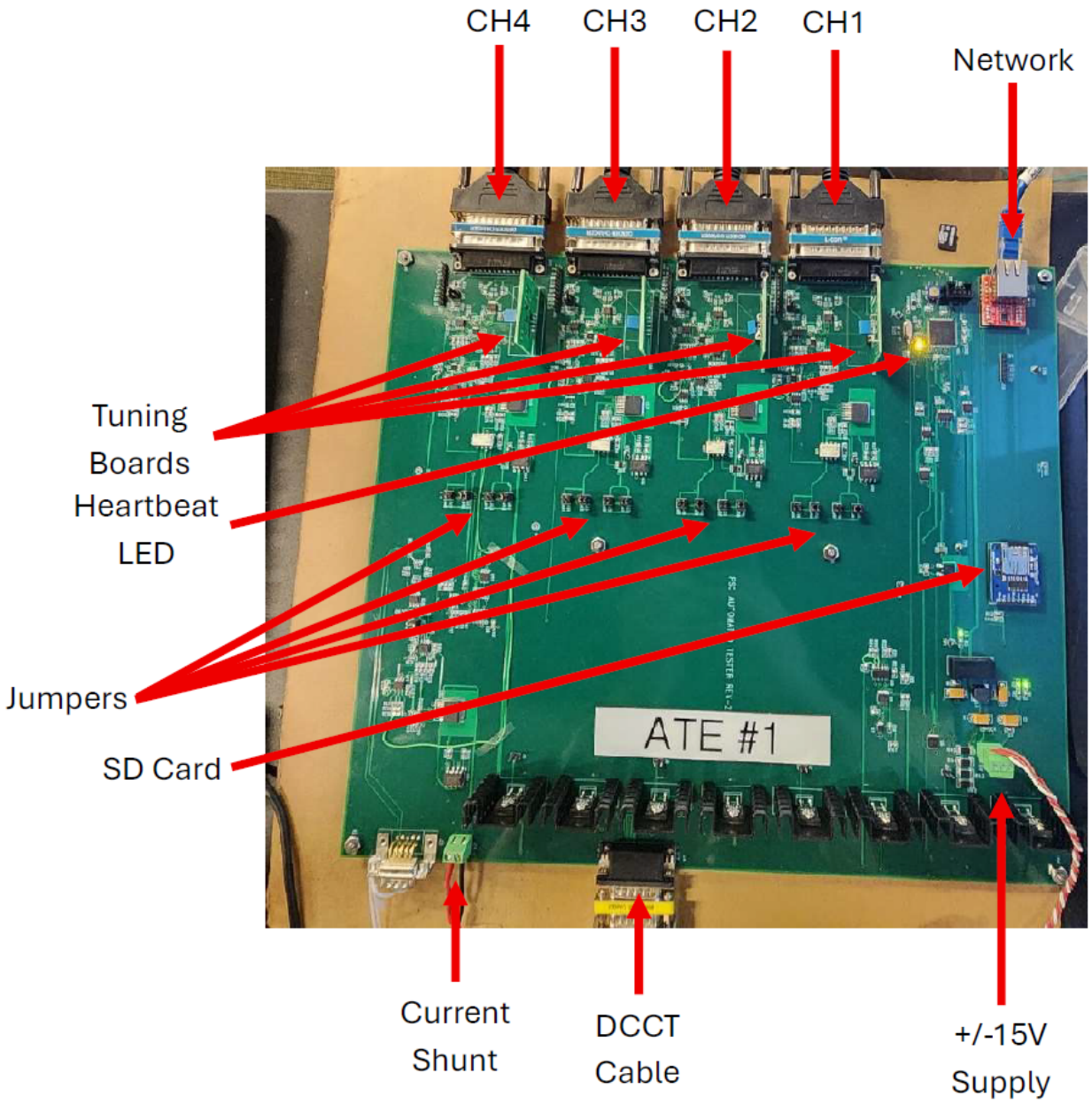


Figure 3.3: Annotated ATE Board

3.2.1 Current Measurement Loop/DMM Connection

The DMM is connected to the workstation via a GPIB adapter from Prologix. These are available in both USB and Ethernet forms.

If the DMM/Resistance standard is not used, a jumper must be connected across this terminal block on the ATE if CALDAC is to be used!

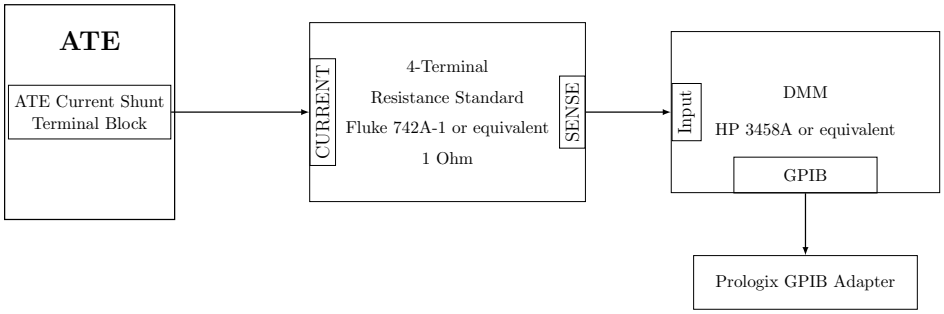


Figure 3.4: ATE → DMM Current Loop

3.2.2 Tuning Boards

The ATE contains four Tuning Board sockets, one per channel. Tuning boards must be installed for each channel under test.

The tuning boards must be matched to the PSC configuration. Each channel's tuning board must be installed as a counterpart to the burdens and comps installed in the PSC, as it simulates the response of the magnet load.

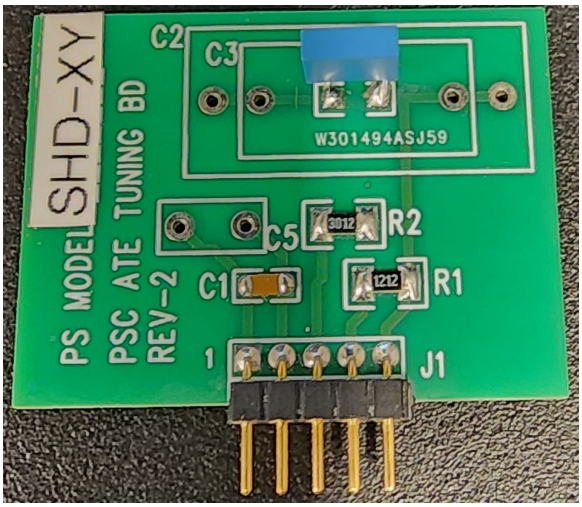


Figure 3.5: ATE Tuning Board for the ALSu SHD-XY Magnet

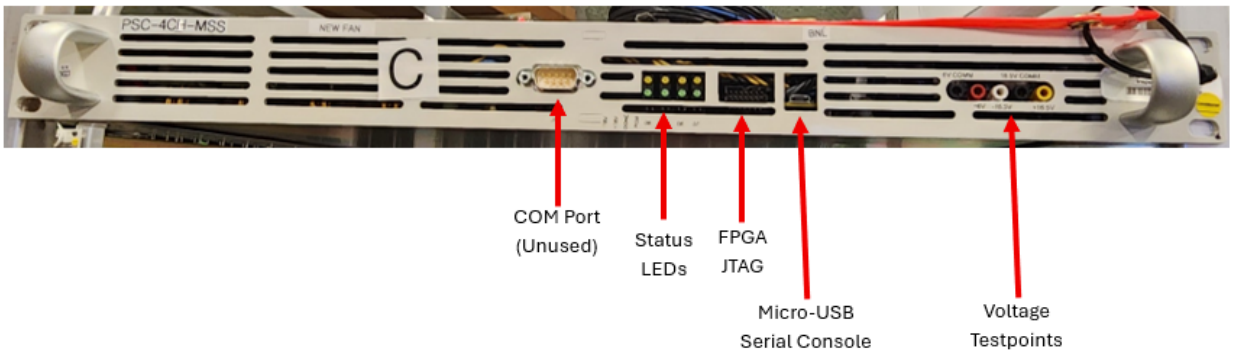
Chapter 4

zPSC Chassis Overview

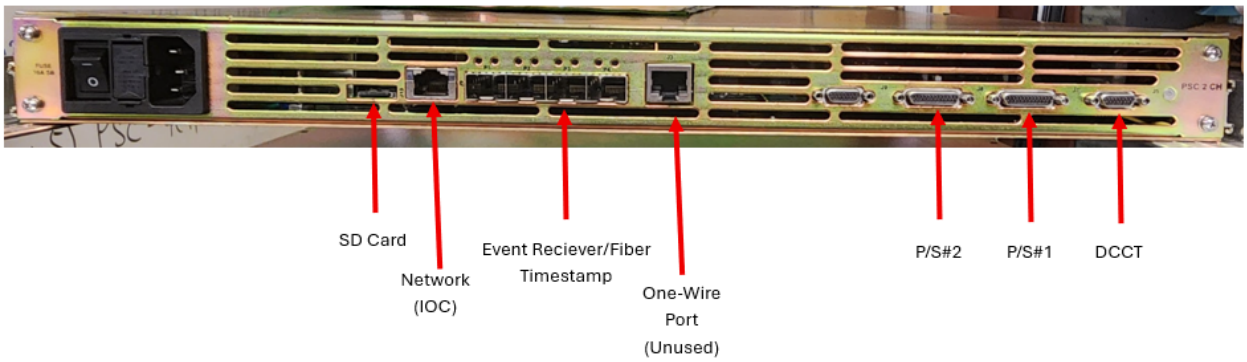
4.1 Introduction

4.2 Front Panels (2- and 4-Channel PSCs)

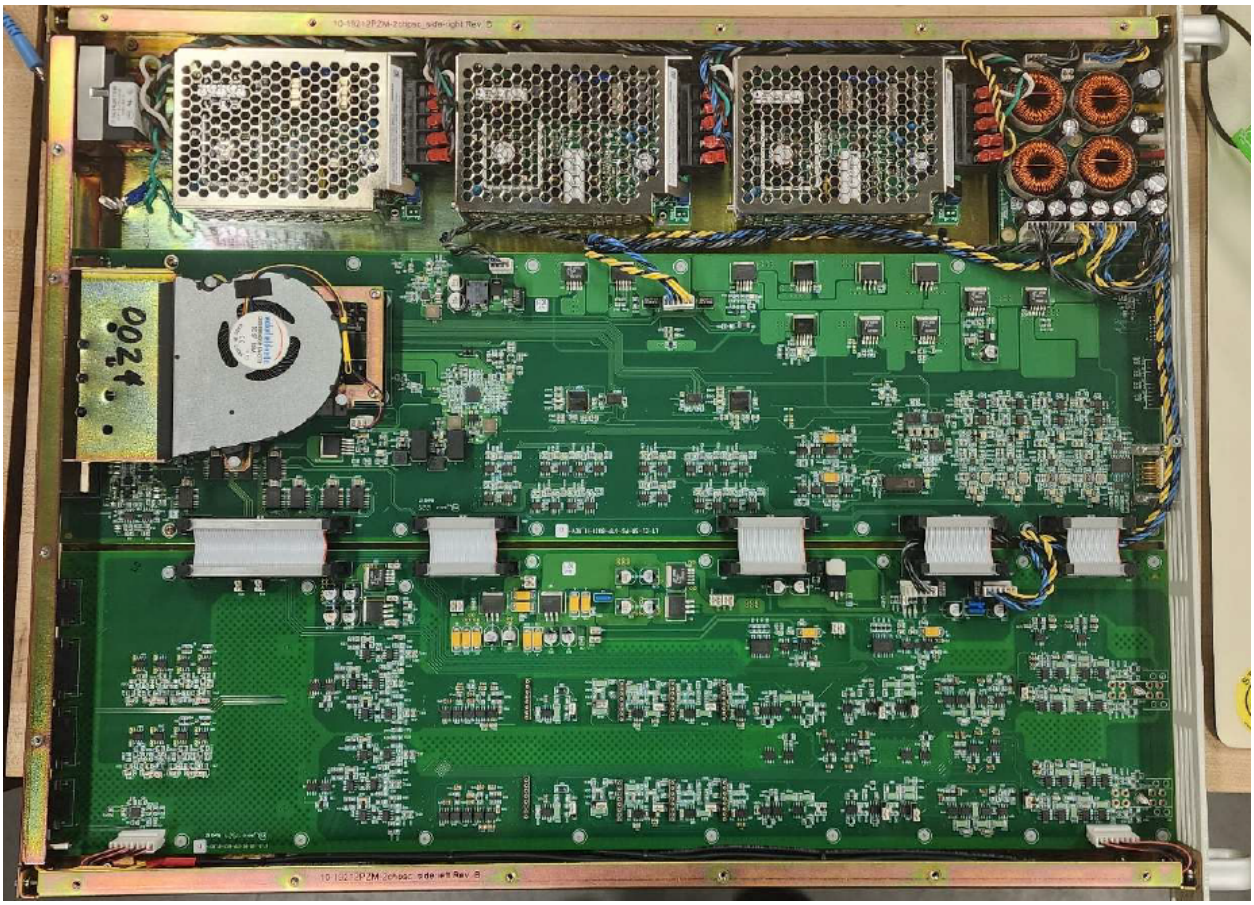
Both 2- and 4-Channel units share identical front panels:



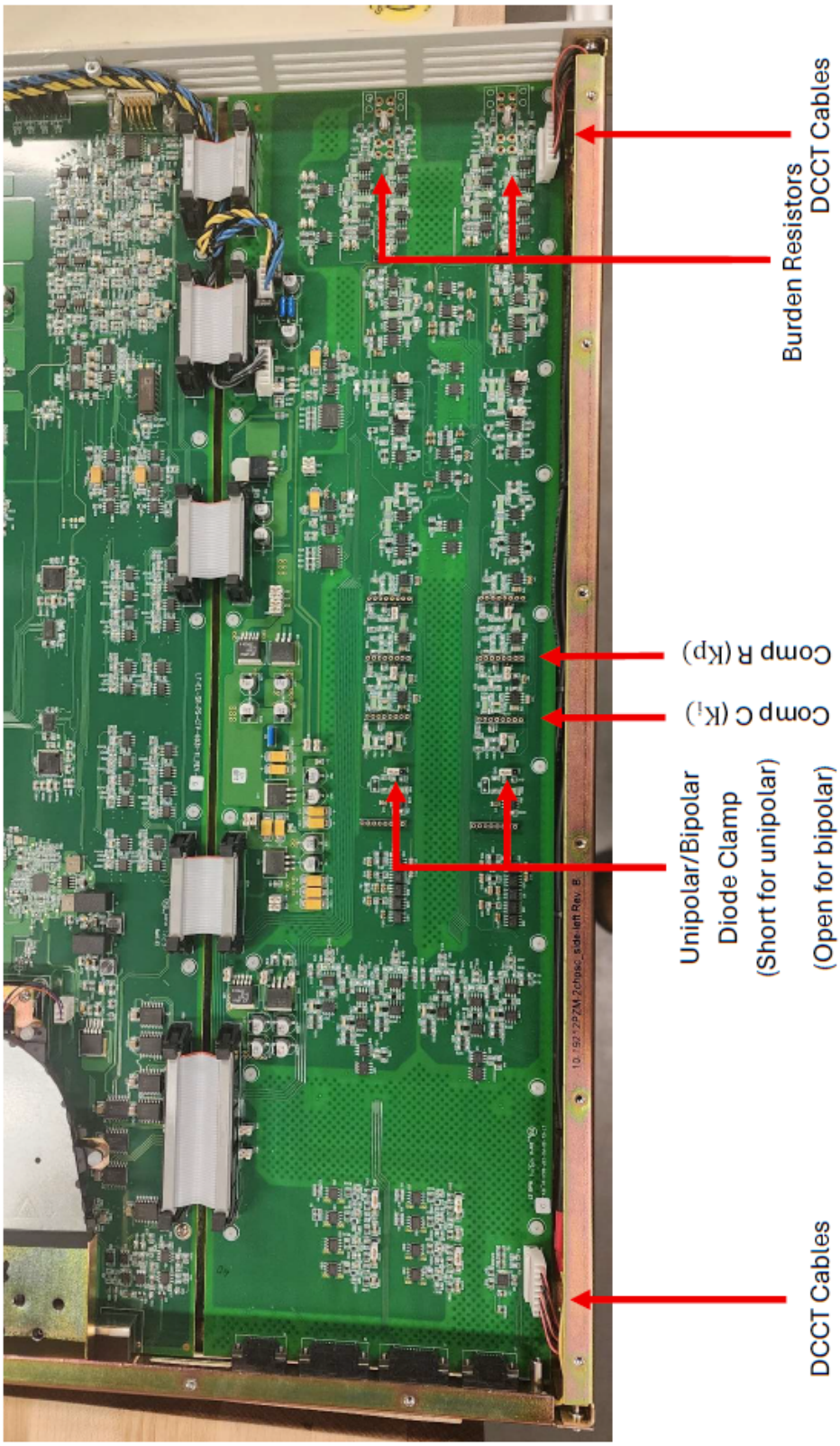
4.3 Rear Panel - 2 Channel



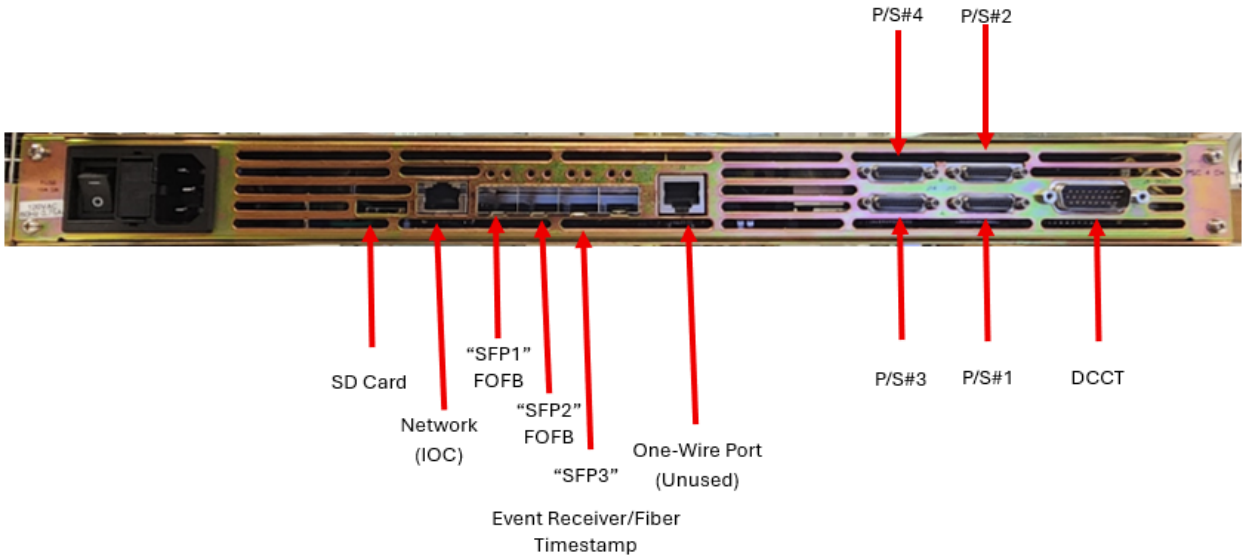
4.4 Internal Component Layout - 2 Channel



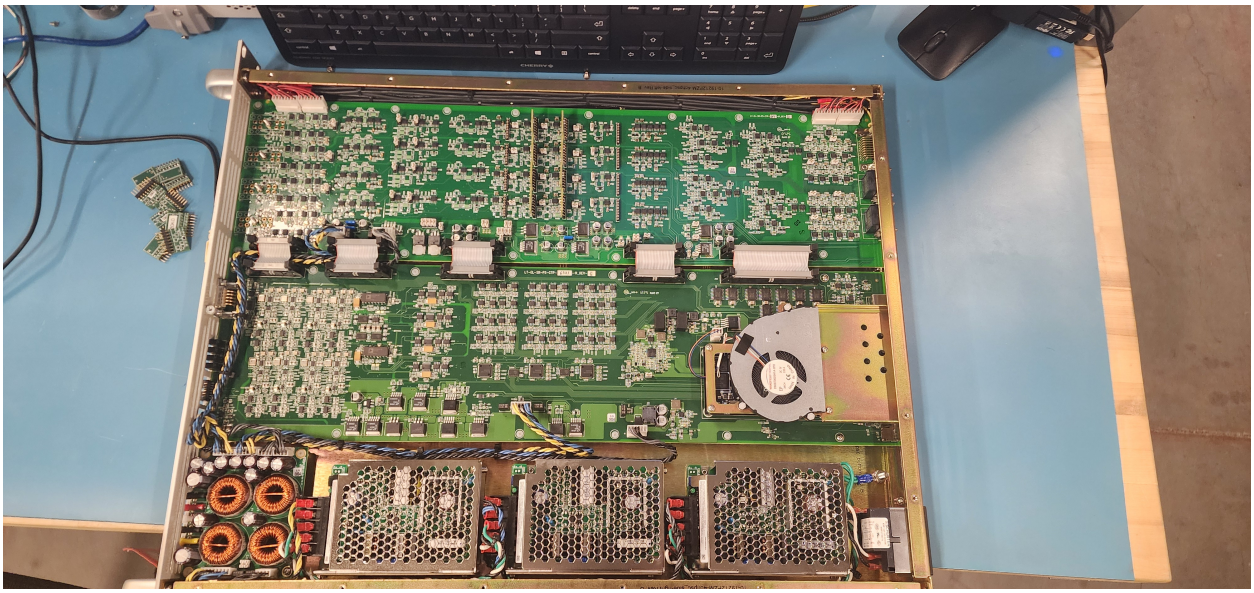
4.4.1 Analog Board Components



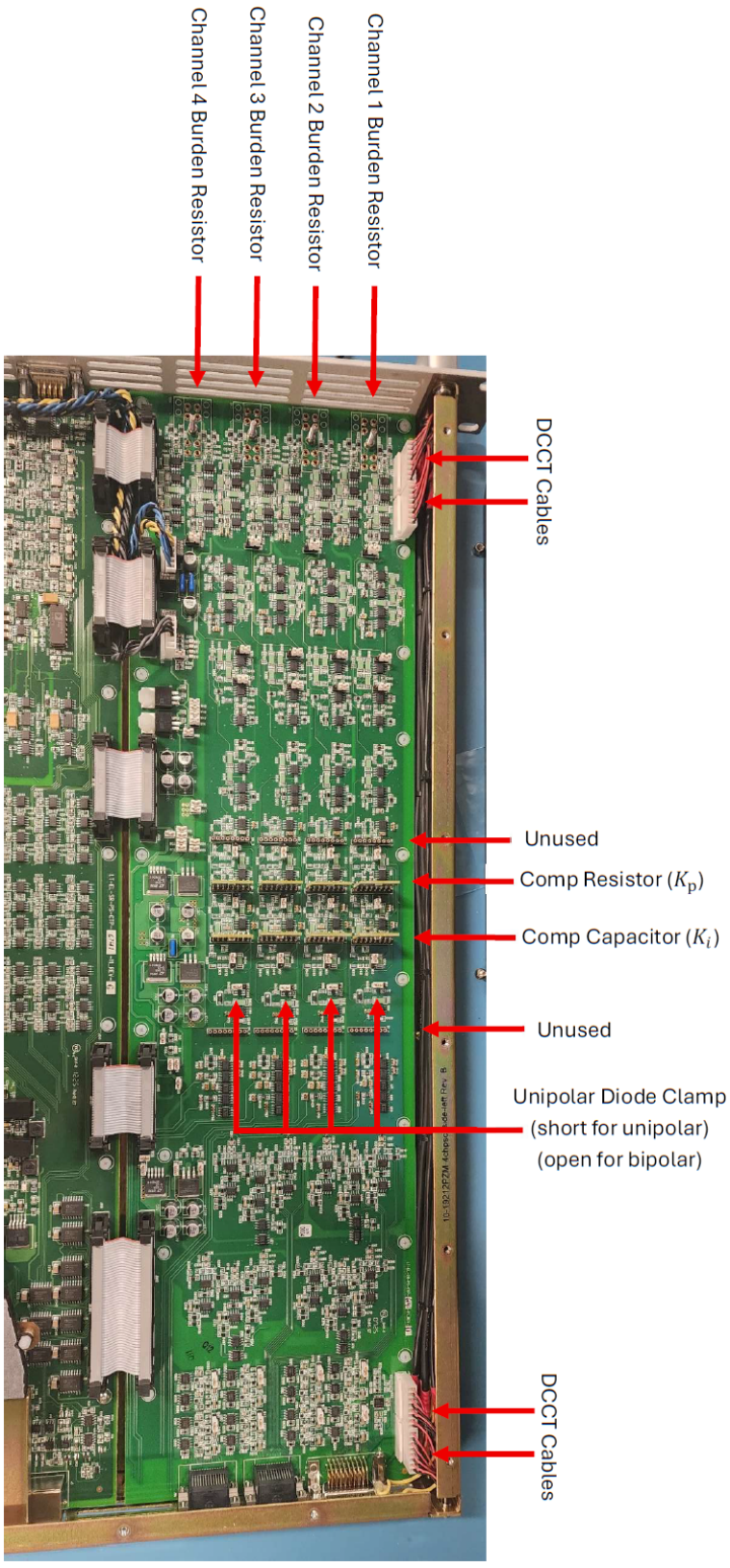
4.5 Rear Panel - 4 Channel



4.6 Internal Component Layout - 4 Channel



4.6.1 Analog Board Components



Chapter 5

EPICS Base and IOCs

5.1 Introduction

There are two IOCs associated with the calibration and test of the zPSC: The ATE IOC, and the zPSC IOC.

5.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- `EPICS_BASE=/epics/base` or `/epics/base_[version]` or similar
- `SUPPORT=/epics/support` – or sometimes MODULES instead of SUPPORT is used.

  Modules required for the ATE and PSC are installed here:
  - seq: SNL-Sequencer
  - pscdrv: Portable Streaming Controller Driver
  - asyn: EPICS module for driver and device support
- `epics/iocs` - each IOC is cloned into this repository
- `epics/systemd-softioc` - This is an NSLS-II utility for managing IOCs as a system service

5.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:
   `sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev autoconf automake libtool pkg-config`

2. Create the EPICS user group, and the empty EPICS directory:
```
sudo groupadd epics
sudo usermod -aG epics $USER
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into `/epics/base` via:
```
git clone --recursive -b 7.0
   https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:
```
make clean
make
```

5. Create the remaining directory structure:
```
mkdir -p /epics/iocs /epics/modules
```

6. Clone asyn:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/epics-modules/asyn.git asyn`
   (c) `cd /epics/modules/asyn`
   (d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`
   (e) Build with `make clean`, and `make`

7. Clone Sequencer:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/epics-modules/sequencer.git seq`
   (c) `cd /epics/modules/seq`
   (d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`
   (e) Build with `make clean`, and `make`

8. Clone PSC Driver:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/mdavidsaver/pscdrv.git pscdrv`

(c) `cd /epics/modules/pscdrv`

(d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`

(e) Build with `make clean`, and `make`

9. Clone Manage IOCs Utility:

(a) `cd /epics`

(b) `git clone https://github.com/NSLS2/systemd-softioc.git`

(c) Navigate to the repo directory `cd systemd-softioc`

(d) Make the install script executable and run:

```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

5.4 Installing the ATE IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
   `git clone https://github.com/capotostobnl/ALSu-PSC_ATE-ioc.git PSCtest`
3. `cd PSCtest`
4. `git switch Released,-UPC/BPC-Split-IOC-Ver-2`
5. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
6. `echo -e "EPICS_BASE=/epics/base\nASYN=/epics/modules/asyn" > /epics/iocs/PSCtest/configure/RELEASE.local`
7. Build the IOC: `make clean`, `make`
8. Update the st.cmd file: `cd iocBoot/iocPSCtest`

(a) Edit the st.cmd file: `nano st.cmd`

(b) Update the EPICS_BASE variable if needed.

(c) Update the IPPORT to match the ATE. Repository has 10.69.26.4:5000 as default. Changing this later from the Phoebus/CSS panels may not work!

9. Find the next available port: `manage-iocs nextport`
10. Configure this in the config file: `nano config`

11. Set the `NAME=PSCtest`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
12. Update the directory path in the root-level st.cmd to point to the right binaries
13. Enroll in manage-iocs as a service: `sudo manage-iocs install PSCtest`
14. Enable auto-start on boot: `sudo manage-iocs enable PSCtest`
15. Check the status, it should be running! `manage-iocs status`
16. And if you want other information on the ioc: `manage-iocs report`

5.5 Installing the PSC IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
   `git clone https://github.com/jamead/zpsc-ioc.git zpsc_ioc`
3. `cd zpsc_ioc`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. ```
   echo -e "EPICS_BASE = /epics/base\n
       PSCDRV = /epics/modules/pscdrv\n
       SEQ = /epics/modules/seq" >
        /epics/iocs/zpsc_ioc/configure/RELEASE.local
   ```
6. Build the IOC: `make clean`, `make`
7. Update the st.cmd file as needed: `cd iocBoot/ioczpsc`
8. Create the st.cmd file used by systemd-softioc:

   ```
   echo -e '#!/bin/bash\n\n
     cd "/epics/iocs/zpsc_ioc/iocBoot/ioczpsc"\n./st.cmd' > st.cmd
   ```

9. Make the new st.cmd executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the config file: `nano config`
12. Set the `NAME=zpsc_ioc`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level st.cmd to the `iocBoot/ioczpsc/st.cmd`
14. Enroll in manage-iocs as a service: `sudo manage-iocs install zpsc_ioc`
15. Enable auto-start on boot: `sudo manage-iocs enable zpsc_ioc`
   ```

16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`

Chapter 6

CS-Studio - Phoebus

6.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directories, `/epics/iocs/PSCtest/gui` and `/epics/iocs/zpsc_ioc/gui`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'PSC Launcher' panel using the Home button, instead of having to manually navigate to the file when needed.

6.2 Installing Phoebus

1. If not installed, install Java, e.g.:
   ```
   sudo apt update && sudo apt install openjdk-17-jdk
   ```
2. Create the directory: `sudo mkdir /opt/phoebus`
   ```
   sudo chown $USER:epics /opt/phoebus
   chmod 2775 /opt/phoebus
   ```
3. Download Phoebus and extract to the `/opt/phoebus` directory
   https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz

   ```
   cd /opt/phoebus
   wget https://github.com/ControlSystemStudio/phoebus/releases/download/
   v4.7.4/phoebus-4.7.4-linux.tar.gz
   ```

```
tar -xvzf phoebus-4.7.4-linux.tar.gz -C . --strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`
5. Copy in necessary configuration changes from the template:
   ```
   git clone
   https://github.com/capotostobnl/Phoebus-ALSU.git /opt/phoebus/config
   ```
6. overwrite the defaults with the new template: `mv config/* .`
7. Update paths as necessary in the `settings.ini`
8. Alias Phoebus to run via command `run-phoebus`:
   ```
   echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' >> ~/.bashrc
   source ~/.bashrc
   ```
9. Now test! Try run-phoebus from a terminal, and then click the 'Home' button!

Chapter 7

Networking

7.1 Overview

The default configuration for these devices is to use static IP addresses on the 10.69.26.0/23 subnet. Deviating from this will require updates be made to the IOCs for the PSC and ATE, as well as changes to some parts of the calibration scripts, and, if applicable, the FOFB Test module of the functional test script set.

The network topology is generally as follows:

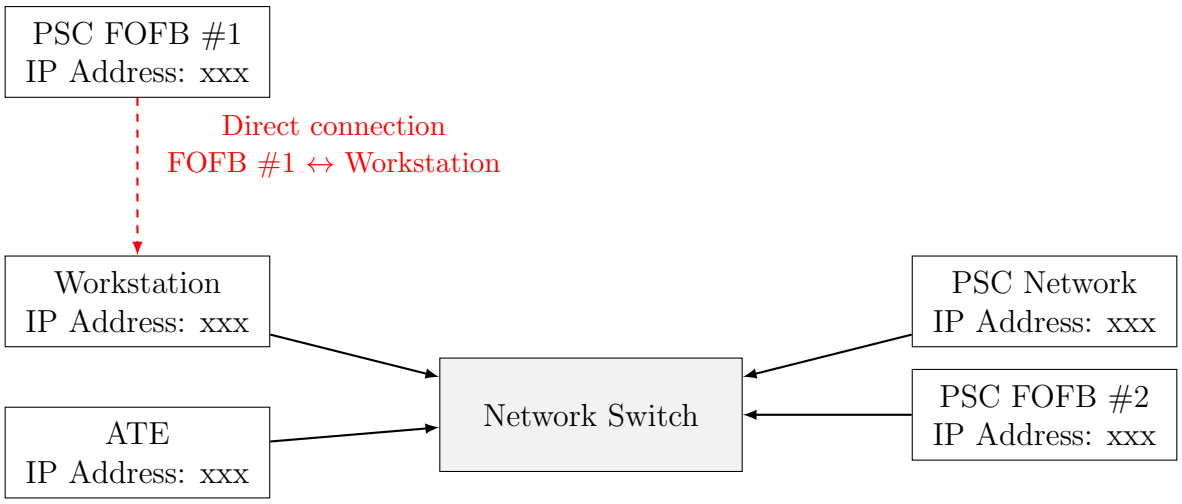


Figure 7.1: Network Topology with Clean FOFB Direct Link

Chapter 8

Installing and Configuring Minicom

8.1 Overview

Minicom is the serial terminal emulator of choice for NSLS-II, but many other options exist such as Teraterm for Windows, or PuTTY for cross platform.

8.2 Installation in a Linux environment

User Permissions

Access to the serial port by default requires privilege escalation. This issue can be sidestepped by adding the user accounts which need to access the serial ports to the dialout (or uucp for RHEL) user group.

This is also required later for use of the calibration script, if a USB-based GPIB adapter is used, otherwise those scripts must run as root, which can cause permissions issues with reports.

To add a user to the dialout group: `sudo usermod -aG dialout $USER`
Then to test: `newgrp dialout`, then `groups`. You should see dialout in your groups.

You may need to log out and log back in to ensure this persists in new terminal windows.

8.2.1 Installing Minicom

Installation of Minicom is simple, in a Debian-based distribution, use apt:
`sudo apt-get update && sudo apt-get install -y minicom`

8.3 Configuring Minicom

Create a default configuration if one does not exist for Minicom:

Open the a text file as root: `sudo nano /etc/minicom/minirc.dfl` and enter the following (One line per command, whitespace/tabs inside the line isn't critical)

```
pu baudrate         115200
pu bits             8
pu parity           N
pu stopbits         1
pu rtscts           No
pu xonxoff          No
```

This disables hardware and software flow control, and sets baud rate, parity, and stop bits.

8.4 Running Minicom

You are now ready to run Minicom. Find the PSC's serial port. With no PSC connected, you run: `ls /dev/`

Then plug in the PSC, and re-run the command; it should appear as a USB device, e.g. ttyUSB2. Search the list for which one new device will suddenly appear in the list.

Then: `minicom -D /dev/ttyUSB2`, replacing ttyUSB2 with whatever your PSC's port is.
A serial terminal should open; press return, and the PSC's menu should now print on the screen.

Chapter 9

Python Installation and Configuration

9.1 Overview

Python3 is required for the calibration and test scripts to be run for the PSCs. Below sections will describe the process for installing Python3, and handling package management and virtual environment management, if desired.

9.2 Installing and Configuring Python, and the Python Environment

9.2.1 Installing Python

Many downstream Debian-based distributions come bundled with Python3. If yours does not, install Python3 via: `sudo apt update && sudo apt install python3`

9.2.2 Python Package Management

Installing Pip

These scripts were written using Python3. It is assumed that Python3 is already installed on your workstation.

A Pip requirements file is included in the repository for ease of package management, but pip may not come bundled with the OS; to install Pip:
`sudo apt install python3-pip`

Installing Packages

Packages can be simply installed via pip now.

If within a virtual environment, use: `pip install -r requirements.txt`
If not using environments: `python3 -m pip install -r requirements.txt`
If you run into permissions issues: `python3 -m pip install --user -r requirements.txt`

Python Virtual Environments

It may be desireable to use a virtual environment, e.g., venv or Conda, especially if other scripts are used on this machine that may have overlapping or conflicting packages installed. My personal preference is for venv, however Conda is used by NSLS-II at the organizational level. venv is a simpler and light weight tool, if it is needed only for running Python projects.

venv can be installed with:
`sudo apt install python3-venv`

The environment can then be activated with `python3 -m venv my_venv_name`. You can then activate the environment with `source my_venv/bin/activate`.

Chapter 10

Calibration and Test Script Installation

10.1 Overview

The repository for the calibration and test scripts can be found at: `https://github.com/capotostobnl/NSLS2_zPSC_Calibration_and_Test`
Details regarding the use of the script will come in the following chapters on Calibration and on Functional Tests, though all scripts are launched from this one repository's 'launcher.py' script.

10.2 Cloning the Script Repository

1. Choose a location for the repo, and go there: `cd /`
2. Clone the repository: `git clone https://github.com/capotostobnl/NSLS2_zPSC_Calibration_and_Test`

10.3 Creating the Virtual Environment

If you choose to use a virtual environment, create it and then install the dependencies. If you choose to not use venv, you can skip to installing dependencies in the next section.

1. Navigate to the repository and then: `python3 -m venv .venv`
2. Activate the environment: `source .venv/bin/activate`

10.4 Installing Dependencies

Required Python modules can be installed via the requirements.txt from the repository via: `python3 -m pip install -r requirements.txt`

10.5 Compiling the FOFB Packet Generator

The Fast Orbit Feedback (FOFB) Test script relies on a packet generator program written in C, and executed by a shell script. The script will run these as subprocesses during the FOFB test, so if they are not available FOFB testing will fail.

The program must be compiled on your system, and the shell script marked executable before the FOFB script can run properly.

1. Navigate to the functional tests subdirectory: From the repository root-level directory, `cd testing/functional_tests`
2. If needed, change the target IP address set in `main()`. The default is 10.69.26.55. You may also choose to change the setpoints here as well. The default is to use a setpoint of 11.50000, 11.50001, 11.50002, and 11.50003 for channels 1 through 4 (0 through 3).
3. `gcc caen_fast_genpacket.c -o caen_fast_genpacket`
4. `chmod +x caen_fast_genpacket_loop_inf.sh`
5. Attempt to run `caen_fast_genpacket_loop_inf.sh`. It should repeatedly print the message `UDP packet sent successfully.`: `./caen_fast_genpacket_loop_inf.sh`

10.6 Configuring Network Interfaces in the `fofb_test.py` Module

There are a few items that must be configured in the `fofb_test.py` module which will vary from workstation to workstation, network to network.

- UDP Packet Capture Interface:
  You must set the interface that captures UDP packets, and the port it uses, if not using the existing values. This is passed as an argument for the function call in Line 278, `status, out, err, returncode, cmd = capture_udp_packets(iface="enp115s0", port=12345, timeout_s=10)`
- ARP table entries:
  ARP table entries must be updated, so the packets can be routed to the capture interface. This is done at Line 169, `cmd = "sudo arp -s 10.69.26.55 00:11:22:33:44:55"`. If using the default IP addresses, this does not need to be changed.
- FOFB IP Address PV Value:
  The FOFB IP Address PV is written with a type-converted hex value. The hex value is a 32-bit int with one byte per octet, so 10.69.26.55 becomes 0x0A451A37. 10.0.142.100 would become 0x0A008E64. The hex value needs to be set in the function call, Line 222, `safe_caput(fofb_ip_pv, int(0x0A451A37))`

Chapter 11

Configuring the Test and Calibration Scripts

11.1 Overview

There are many places where changes may need to be made when bringing a new workstation up for the first time.

After initial configuration, no further changes should be required, with the exception of the `psc_models.py` class, which will be modified any time a new PSC is added, or a particular PSC's configuration requirements are changed.

11.2 The `common` Files Directory

This directory contains a number of modules which may need reconfiguration as outlined below.

Files in this directory may be used by both the Calibration side and the Test side of the script package.

11.2.1 The `psc_models.py` Class

This module is the module that will need to be updated regularly. In this module, you will define the parameters for every 'flavor' of PSC that will be calibrated or tested.

11.2.1.1 Overview

This module instantiates a number of dataclasses that define calibration and test parameters for each unit.
Each PSC model definition contains the following:

- Hardware calibration constants
- Fault threshold limits
- Scaling factors
- Functional test configuration parameters
- Test setpoints and some pass/fail test criteria

Both the calibration and functional test modules use the data from this single file to be configured, allowing one location to provide all data required for a given flavor of PSC.

11.2.1.2 PSC Model Structure

Every PSC is defined as a `PSCModel` object, registered in the global `MODELS` dictionary:

```
MODELS = {
        "MY_NEW_MODEL": PSCModel(
        ...
        ),
}
```

Ensure the dictionary key that is used is unique! Duplicate keys will cause the script to fail.

11.2.1.3 Adding a New PSC Model

The easiest way to create new PSC definitions is to copy one of the existing models and update the values for the new model.
Example:

```
"MY_NEW_MODEL": PSCModel(
        model_id="MY_NEW_MODEL",
        display_name="My New PSC",
        description="PSC-4CH-MYMODEL",
        designation="PSC-4CH-MYMODEL",
```

```
6
7           channels=(1,2,3,4),
8           drive_channels=(1,2,3,4),
9           readback_channels=(1,2,3,4),
10
11          calibration_parameters=...,
12
13          reg=...,
14          smooth=...,
15          jump=...
16  )
```

Common Parameters

Parameter	Description
model_id	Internal identifier used by software
display_name	Name shown in selection menus
description	Full PSC description
designation	String used during calibration reporting
channels	Physical channels present on the PSC
drive_channels	Channels used for control output
readback_channels	Channels used for measurement feedback

Most channels use a 1:1 mapping for channels, drive_channels, and readback_channels:

```
1  channels=(1,2,3,4)
2
3  drive_channels=(1,2,3,4)
4  readback_channels=(1,2,3,4)
```

But some specialty units with hardware modifications, may require a different parameter set. One example was a 'Dual-Mode' PSC, which had a wire jumper added so the PID loop for Channel 3 drove the controls for both Channel 3 and Channel 4. Channels 1 and 2 were unused. This particular unit is why this functionality was added, as that unit became:

```
1  channels=(3,4)
2
3  drive_channels=(3,3)
4  readback_channels=(3,4)
```

Calibration Parameters

The calibration parameters are configured using the CalibrationParameters class. Many values are initialized to default values via the subclasses, which initialize the values to standard values, which may be modified in the calibration_parameters instantiation if so desired:

```
calibration_parameters=CalibrationParameters(
        ndcct=1000.0,
        burden_resistors=ChannelValues(ch1=83.333333, ch2=83.333333,
                                       ch3=83.333333, ch4=83.333333),

        fault_limits=PSCFaultThresholdsLimits(
                ovc1_threshold=ChannelValues(ch1=10, ch2=10, ch3=10, ch4=10),
                ovc2_threshold=ChannelValues(ch1=10, ch2=10, ch3=10, ch4=10),
                ovv_threshold=ChannelValues(ch1=15, ch2=15, ch3=15, ch4=15),
        ),

        scale_factors=PSCScaleFactors(
                sf_vout=ChannelValues(ch1=1.9, ch2=1.9, ch3=1.9, ch4=1.9),
                sf_spare=ChannelValues(ch1=-5.0, ch2=-5.0, ch3=-5.0, ch4=-5.0),
        ),
),
```

These parameters are as follows:

Parameter	Description
ndcct	Nominal DC current transformer turns ratio used during current scaling and calibration calculations.
burden_resistors	Burden resistor values associated with DCCT channel. These values are used to convert sensed current into a measurable voltage.
fault_limits	
scale_factors	

Table 11.1: PSC calibration parameters

The PSCFaultThresholdsLimits are as follows:

Parameter	Description
ovc1_threshold	
ovc2_threshold	
ovv_threshold	
err1_threshold	
err2_threshold	
ignd_threshold	
ovc1_flt_cnt	
ovc2_flt_cnt	
ovv_flt_cnt	
err1_flt_cnt	
err2_flt_cnt	
ignd_flt_cnt	
dcct_flt_cnt	
flt1_flt_cnt	
flt2_flt_cnt	
flt3_flt_cnt	
flt_on_cnt	
flt_heartbeat_cnt	

Table 11.2: PSCFaultThresholdsLimits parameters

The PSCScaleFactors are as follows:

Parameter	Description
sf_ramp_rate	
sf_dcct_scale	
sf_vout	
sf_ignd	
sf_spare	
sf_regulator	
sf_error	

Table 11.3: PSCScaleFactors parameters

Functional Test Parameters

The test parameters are configured in a few sections as follows:
Test Enabling/Disabling: By default, all functional tests will be run. Individual tests can be enabled or disabled by adding a call for `FuncSuite()` as follows:

```
func_tests=FuncSuite(
regulation=True,
smooth=True,
jump=True
)
```

You may also disable specific channels:

```
func_tests=FuncSuite(
jump=ChannelValues(
        ch1=False,
        ch2=False,
        ch3=True,
        ch4=True
        )
)
```

Regulator Test Parameters: The regulator test parameters are as follows:

```
1         reg=RegulatorTestParams(
2                 setpoints=(reg_pts := ChannelValues(ch1=5,
3                                                     ch2=5,
4                                                     ch3=5,
5                                                     ch4=5
6                                                     )
7                            ),
8                 settling_time=10
9         ),
```

Parameter	Description
setpoints	Current setpoint, per channel, to run the stability test. No default - Must be defined!
settling_time	Settling time in seconds, after setting the current before beginning data collection. Defaults to 10 seconds.
tolerance	The pass/fail threshold for automated go/no-go test, defaults to 50mA.
ramp_rate	Ramp rate used during setup to the regulator test. Defaults to 10A/s
num_samples	Number of datapoints to collect for sample, defaults to 180.
sample_interval	Sampling interval, defaults to 300ms

Table 11.4: Regulator Test Parameters

Smooth Ramp Test Parameters: The smooth ramp test parameters are as follows:

```
smooth=SmoothRampTestParams(
        start_setpoints=ChannelValues(ch1=-5,
                                      ch2=-5,
                                      ch3=-5,
                                      ch4=-5),
        end_setpoints=ChannelValues(ch1=5,
                                    ch2=5,
                                    ch3=5,
                                    ch4=5),
        ramp_rate=ChannelValues(ch1=10,
                                ch2=10,
                                ch3=10,
                                ch4=10),
        settling_time=10,
        tolerance=0.05
),
```

Parameter	Description
start_setpoints	Current at which the ramp will start. Must be positive for unipolar mode! Positive or negative acceptable for bipolar. No default - Must be defined!
end_setpoints	Current at which the ramp will end. Must be positive for unipolar mode! Positive or negative acceptable for bipolar. No default - Must be defined!
ramp_rate	The ramp rate in Amps/second that the channel will ramp at. No default - Must be defined!
settling_time	Settling time delay, seconds, that the test will wait after setting the start setpoint but before initiating the ramp. Defaults to 10 seconds.
tolerance	The pass/fail threshold for automated go/no-go test, defaults to 50mA.
waveforms	This is the WaveformFlags class, which can be used to enable or disable plotting of specific waveforms in the report. This was used for specialty units, and initializes the defaults to enable plotting all values

Table 11.5: Smooth Ramp Test Parameters

To use the Waveform Flags, add the desired flag that you want to disable (or you can add all and explicitly enable/disable). This was added to support units in which one PID loop drives two channels, the driven channel's outputs are of interest, but the inputs are unused, e.g., in ALSu's `BTA-DA_B4_B7-8`:

```
smooth=SmoothRampTestParams(
        start_setpoints=ChannelValues(ch1=None,
                                      ch2=-5,
                                      ch3=-5,
                                      ch4=-5,
        end_setpoints=ChannelValues(ch1=None,
                                    ch2=5,
                                    ch3=5,
                                    ch4=5,
        ramp_rate=ChannelValues(ch1=None,
                                ch2=10,
                                ch3=10,
                                ch4=10,
        settling_time=10,
        tolerance=0.05,
        waveforms=ChannelValues(
                ch1=None,
                ch2=WaveformFlags(), # using defaults
                ch3=WaveformFlags(), # using defaults
                ch4=WaveformFlags(DAC=False,
                                  DCCT1=False,
                                  DCCT2=False,
                                  ERROR=False,
                                  REG=True,
                                  VOLT=True,
                                  IGND=False,
                                  SPARE=True
                )
        )
),
```

Parameter	Description
DAC	DAC Waveform
DCCT1	DCCT1 Waveform
DCCT2	DCCT2 Waveform
ERROR	ERROR Waveform
REG	REG Waveform
VOLT	PS Voltage Waveform
IGND	Ground Waveform
SPARE	SPARE Waveform

Table 11.6: WaveformFlags class instance attributes

Jump Test Parameters: The jump/transient test parameters are described below. Detail is not shown, but the WaveformFlags can be defined for this test as well.

```
jump=JumpTestParams(
        start_setpoints=reg_pts,
        step_size=ChannelValues(ch1=0.05,
                                ch2=0.05,
                                ch3=0.05,
                                ch4=0.05),
        sample_window=500,
        tolerance=0.05
)
```

Parameter	Description
start_setpoints	Current setpoint, per channel, to run the stability test. No default - Must be defined!
step_size	Magnitude of the step/jump, in Amps
sample_window	Number of points to show before and after the jump in the 'zoom' view
tolerance	Ground current pass/fail threshold in Amps
waveforms	This is the WaveformFlags class, which can be used to enable or disable plotting of specific waveforms in the report. This was used for specialty units, and initializes the defaults to enable plotting all values

Table 11.7: Jump Test Parameters

11.2.2 EPICS Adapters/Drivers

There are two modules, `ate_epics.py` and `psc_epics.py` which provide a library of function calls to read and write various EPICS PVs. As firmware and IOC changes are made, some changes may be required in these drivers to add new features, or fix features that are removed or break.

These two modules are little more than a pyepics wrapper, providing easier formatting, less repetition in the code manually creating PV strings, and providing typehinting/typechecking and auto-complete suggestions.

11.2.3 The `initialize_dut.py` Utility

This is a small utility that collects information from the PSC EEPROM, generate the filenames, timestamps, and directory structures to be used during calibration and test. It shouldn't require any changes in the forseeable future. The directory structures are created dynamically based on the directory it lives in, and should resolve without issue both on Linux and Windows. The utility will create the `cal_reports`, `test_reports`, and `test_data` directories.

11.3 Calibration

11.3.1 The `initialize_qspi.py` Module

This module contains a few simple functions that will do the following:

1. Write scale factors
2. Write fault thresholds
3. Write fault count limits
4. Initialize gains to 1 and offsets to 0
5. Write these values to QSPI

Any existing values will be overwritten.

11.3.2 The `psc_calibration.py` Module

This module performs the calibration of the PSC, interfacing the ATE via raw UDP socket packets, and DMM via psyserial to send ASCII commands to the GPIB interface.

There are a few items that must be changed in some test setups:

1. `ATE_IP_ADDRESS`:
   This must be set for the ATE IP Address as configured in your testbench.

2. DMM USB Port `ser1 = serial.Serial('/dev/ttyUSB0', 115200, timeout=30')`:
   If using the HP 3458A, then the command structure need not change, however, the USB port used, e.g., `'/dev/ttyUSB0'` may need to be changed, depending on how it populates on your system, or if using an ethernet-based GPIB adapter.

No other parameters should need to be modified.

11.4 Functional Test

The functional test is largely dictated by the configuration in the `psc_models.py` class. If all tests are enabled however, it follows this order:

1. EVR Test:
   Check EVR timestamp, and ensure the 1Hz event can be seen and decoded; ensure each trigger is 1 second apart.

2. ATE Fault Tests:
   Test that each fault bit is able to be toggled (set and clear)

3. PS Regulation Stability Test:
   Set current setpoint to some value and collect so many data points at a set interval, e.g, 180 datapoints at a 300ms interval.

4. PS Jump/Transient Test:
   Take the power supply to some setpoint, e.g., 10A. Then apply a jump/transient in Jump mode, e.g., 50mA jump for 10A→10.05A setting, and observe the transient response of the output.

5. Smooth Ramp Test:
   Ramp the output from one setpoint to another at a specified ramp rate and plot the waveforms.

6. Fast Orbit Feedback (FOFB) Test:
   If the unit is a 'Fast' bandwidth, the FOFB test will be performed. A packet will be written to set the currents via the FOFB loop, and these setpoint changes will be verified. The packets will be dumped into the report via TCPDump as well.

Chapter 12

Initial Booting of the PSC and Configuration of the EEPROM

12.1 Overview

This section will discuss the steps required for the first time booting a new PSC, and initializing the EEPROM.

The EEPROM must be configured before many functions will work.

It is important to note that once the EEPROM is configured, many parameters of the PSC will still provide inconsistent or invalid data until the QSPI values are initialized properly via the scripts outlined in chapter 13.

12.1.1 Creating the SD Card

First, a new SD card must be generated and installed in the PSC. We have been using Microcenter 32GB microSD cards, though other brands and sizes may (or may not) work.

1. Download the BOOT.bin binaries from the git repository
   For convenience, the latest stable version of firmware is located in the test script repostiory outlined in chapter 13.

   Copy to your SD card this BOOT.bin

2. Create and Save the NET.CNF file to the SD card
   A NET.CNF file is included in the repository as well, and can be used with no modifications so long as only one PSC is connected to the network at a time. Connecting more requires changing the static IP address set in this file for each PSC.

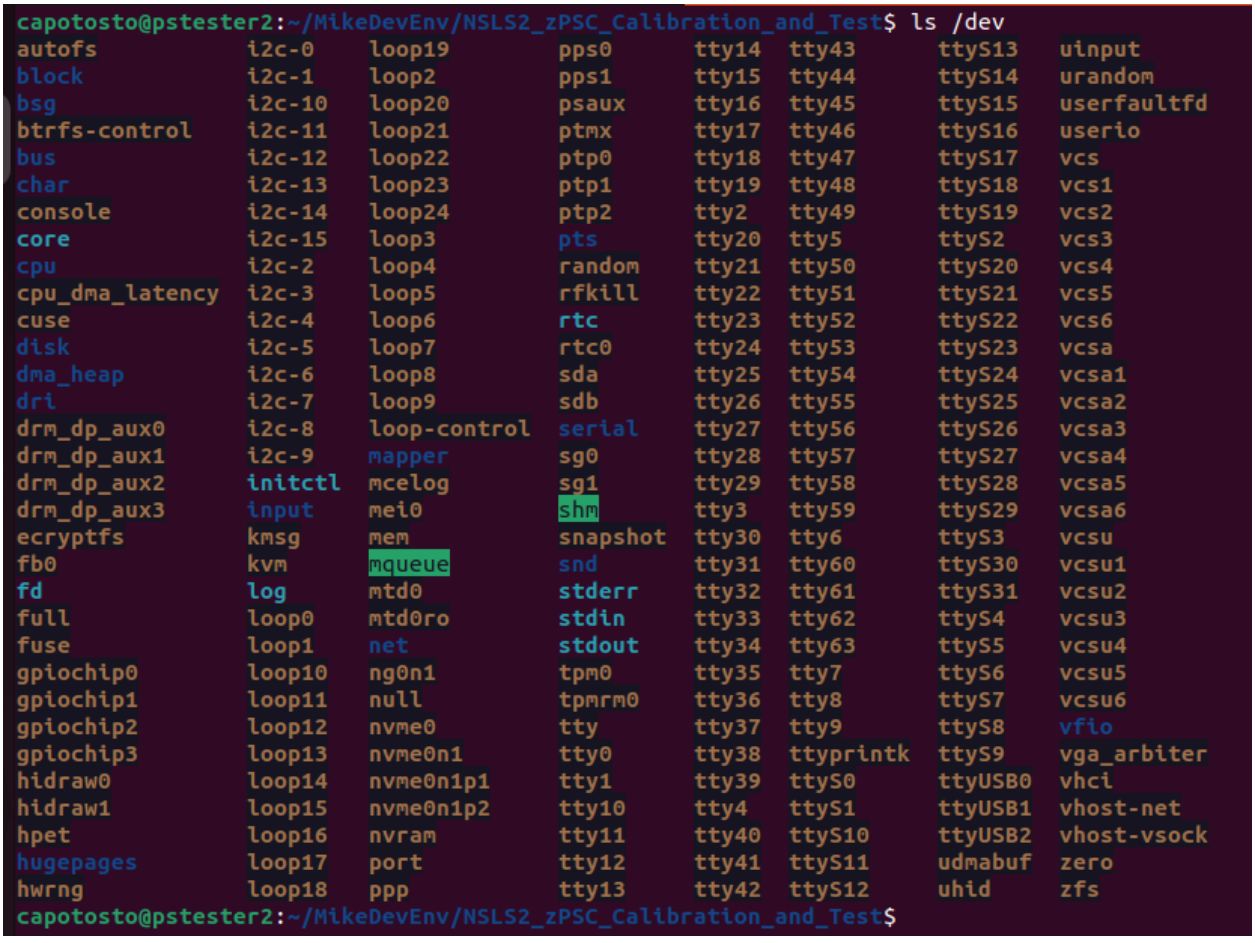
3. Eject the SD card, and install in the zPSC, it should now be ready to boot.

4. Turn on the PSC, it should boot, indicated by front panel LED D8 rapidly flashing. It may take 60 to 90 seconds for the PSC to connect to the network and ping, be patient. The PSC will take an extended amount of time to boot the RTOS if no ethernet cable is connected at all.

12.1.2 Configuring the EEPROM

Once the PSC has booted, we can proceed to writing the EEPROM configuration.

1. Connect the micro-USB cable to the front panel port of the PSC, and identify which serial port it appears as on your terminal.
   You may need to do a directory list before and after plugging in, to see which serial device appears:
   `ls /dev`
   In this example, the PSC will appear as /dev/ttyUSB2.



2. Open Minicom using the port you determined: `minicom -D /dev/ttyUSB2`

```
Welcome to minicom 2.8

OPTIONS: I18n
Port /dev/ttyUSB2, 20:56:01

Press CTRL-A Z for help on special keys



Select an option:
  A:  Display PSC Settings
  B:  Set Number of Channels (2 or 4)
  C:  Set Resolution (High or Medium)
  D:  Set Bandwidth (Fast or Slow)
  E:  Set Polarity (Bipolar or Unipolar)
  F:  Set Ch3-Ch4 to Dual Mode
  G:  Set Main Dipole Mode
  H:  Display Snapshot Stats
  I:  Print FreeRTOS Stats
  J:  Dump EEPROM
  K:  Clear EEPROM
  L:  Test EEPROM
  M:  Dave Bergman Calibration Mode
  Q:  quit
```

3. Set Items B through E for a standard PSC, and F, G as needed.

```
Select an option:
  A:  Display PSC Settings
  B:  Set Number of Channels (2 or 4)
  C:  Set Resolution (High or Medium)
  D:  Set Bandwidth (Fast or Slow)
  E:  Set Polarity (Bipolar or Unipolar)
  F:  Set Ch3-Ch4 to Dual Mode
  G:  Set Main Dipole Mode
  H:  Display Snapshot Stats
  I:  Print FreeRTOS Stats
  J:  Dump EEPROM
  K:  Clear EEPROM
  L:  Test EEPROM
  M:  Dave Bergman Calibration Mode
  Q:  quit
B


Set Number of Channels of PSC: 0 = 2 Channel, 1 = 4 Channel  1
```

Figure 12.1: Setting B

```
Select an option:
  A:  Display PSC Settings
  B:  Set Number of Channels (2 or 4)
  C:  Set Resolution (High or Medium)
  D:  Set Bandwidth (Fast or Slow)
  E:  Set Polarity (Bipolar or Unipolar)
  F:  Set Ch3-Ch4 to Dual Mode
  G:  Set Main Dipole Mode
  H:  Display Snapshot Stats
  I:  Print FreeRTOS Stats
  J:  Dump EEPROM
  K:  Clear EEPROM
  L:  Test EEPROM
  M:  Dave Bergman Calibration Mode
  Q:  quit
C


Set Resolution of PSC: 0 = Medium (18bit), 1 = High (20bit)  1
```

Figure 12.2: Setting C

```
Select an option:
  A:  Display PSC Settings
  B:  Set Number of Channels (2 or 4)
  C:  Set Resolution (High or Medium)
  D:  Set Bandwidth (Fast or Slow)
  E:  Set Polarity (Bipolar or Unipolar)
  F:  Set Ch3-Ch4 to Dual Mode
  G:  Set Main Dipole Mode
  H:  Display Snapshot Stats
  I:  Print FreeRTOS Stats
  J:  Dump EEPROM
  K:  Clear EEPROM
  L:  Test EEPROM
  M:  Dave Bergman Calibration Mode
  Q:  quit
D


Set Bandwidth of PSC: 0 = Fast, 1 = Slow  1
```

Figure 12.3: Setting D

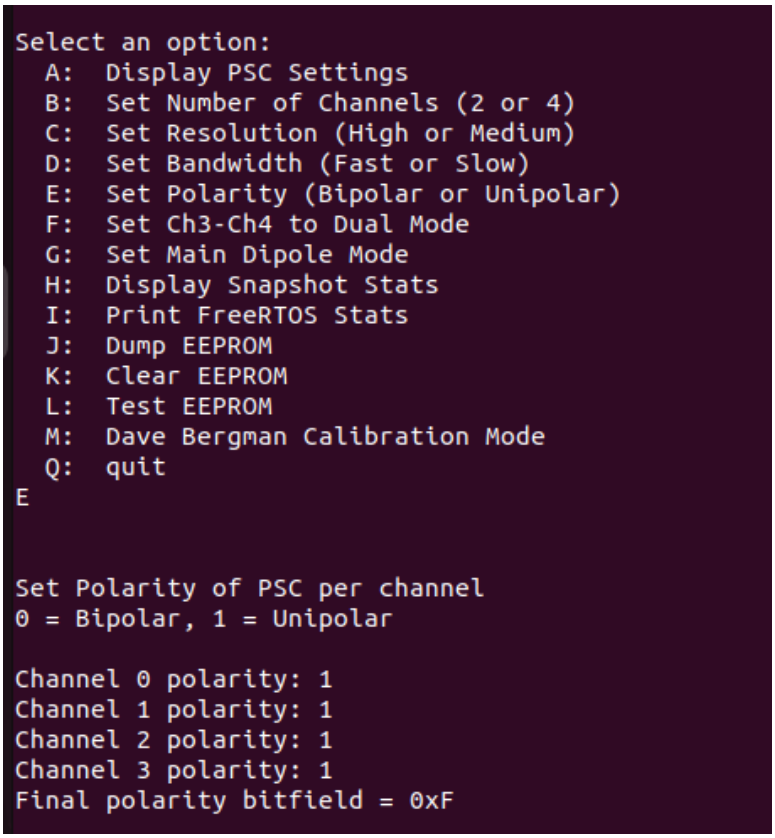


Figure 12.4: Setting E

4. Verify the settings with Option A:

```
Select an option:
  A:  Display PSC Settings
  B:  Set Number of Channels (2 or 4)
  C:  Set Resolution (High or Medium)
  D:  Set Bandwidth (Fast or Slow)
  E:  Set Polarity (Bipolar or Unipolar)
  F:  Set Ch3-Ch4 to Dual Mode
  G:  Set Main Dipole Mode
  H:  Display Snapshot Stats
  I:  Print FreeRTOS Stats
  J:  Dump EEPROM
  K:  Clear EEPROM
  L:  Test EEPROM
  M:  Dave Bergman Calibration Mode
  Q:  quit
A

Number of Channels: 4
Resolution: High
Bandwidth: Slow
Polarity Ch0: Bipolar
Polarity Ch1: Bipolar
Polarity Ch2: Bipolar
Polarity Ch3: Bipolar
Dual Mode is disabled for Ch3-Ch4
Main Dipole Mode is disabled
```

Figure 12.5: Setting A

5. If satisfied, it is now safe to unplug the USB cable, no further action is necessary. To exit Minicom, press `ctrl+a`, release, press `x`, release, and finally press `<return>` for 'Yes' to exit.

Chapter 13

Running the Calibration and Test Script

13.1 Overview

This section will describe how to run and use the Calibration and Test Script, assuming it is configured properly. The following chapters will go into detail on adding new PSC models, and specific details regarding the calibration side and test side of the scripts.

> **It is assumed that secondary ops have been completed for the PSC to be tested. This will include installing fuses, setting various jumpers, verifying various voltages, etc. Refer to the example secondary ops checklist in Appendix A**
>
> **The EEPROM must be configured in the PSC prior to using any of these scripts. Failure to configure the EEPROM, or incorrectly configuring it, will cause issues with the script at even the first menu selection stages. Refer to chapter 12 for instructions**

13.2 Running the Script: Launcher Menu

To run the script, simply execute the launcher.py script from the repository root directory:
`python3 launcher.py`
A menu will appear in your terminal.

```
(.venv) PS C:\users\capotosto\Desktop\NSLS2_zPSC_Calibration_and_Test> python .\launcher.py

  ------------------------------
  Select Execution Mode:
  1. Initialize QSPI Only
  2. Calibrate Only
  3. Test Only
  4. Calibrate and Test
  ------------------------------

  Enter selection (1-4):
```

Choose an option from the menu.

1. Initialize QSPI Only
   This option will set the QSPI parameters normally written at the end of calibration, so values such as OVP/OCP, scale factors, and such, will be written to the flash. All gains are written as 1.00, and all offsets written as 0.0. This will overwrite any values stored in QSPI!

   This is especially useful for initial testing, where you may want to verify all channels work visually, before committing the time to calibrate and test.

2. Calibrate Only
   This option will perform a calibration only, without any tests. This will write all QSPI values at the end of the calibration for each channel.

3. Test Only
   This option will perform only the functional tests, without a calibration. This makes no changes to the QSPI.

4. Calibrate and Test
   This option will run a full calibration, and immediately upon completion of the calibration, it will begin the functional test. This allows a calibration and test cycle to be started, and then left alone; so long as the calibration doesn't fail in a way that exits the script, the test can be left unattended for the duration and it will complete without any further user input once it has began (with the exception of the Fast Orbit Feedback Test, which requires a user password)

13.2.1 Option 1: Initialize QSPI Only

An example screenshot is provided below.
After selecting Menu Option 1, Initialize QSPI Only, the script will sweep the network to detect PSCs. If only one PSC pings, it'll automatically select this unit. If it times out, it'll prompt for the PV prefix of the PSC.

The script reads the EEPROM configuration, and trims the option list to those PSCs that match the EEPROM.

In this example, a 4Ch High-Stability Fast EIC PSC was being configured, so we chose Option 4. Each channel's QSPI was then written, and the script exited gracefully.

```
capotosto@pstester2:~/MikeDevEnv/NSLS2_zPSC_Calibration_and_Test$ python3 launcher.py

------------------------------
Select Execution Mode:
1. Initialize QSPI Only
2. Calibrate Only
3. Test Only
4. Calibrate and Test
------------------------------

Enter selection (1-4): 1

Enter PSC serial number: (Enter "H" for help):0001
Attempting to auto-detect PSCs
Network Sweep: Attempt 1 of 3...
[*] Success: Discovered PSC 2 at 10.69.26.31
pv_prefix = lab{2}



 Reading EEPROM...
EEPROM # Of Channels: 4
EEPROM Resolution: HS (20bit)
EEPROM Bandwidth: F
EEPROM Polarity: Unipolar, Unipolar, Unipolar, Unipolar




Detected 4 channels. Showing 4-channel models:
--------------------------------------------------
  1) C29-ARI-SXN                          [C29-ARI-SXN]
  2) 4Ch-MSS-ID_XYCorr (ARI/SXN Cell 29)  [4Ch-MSS-ID_XYCorr]
  3) 4CH-MSS-Cur_Strip (ARI/SXN Cell 29)  [4CH-MSS-Cur_Strip]
  4) 4CH-HSF-EIC                          [PSC-4CH-HSF-EIC]
  5) 4CH-MSS-SR-SK                        [PSC-4CH-MSS-SR-SK]
--------------------------------------------------

Enter Type (or 'q' to quit): 4
--> Selected: 4CH-HSF-EIC

QSPI Written
QSPI Written
QSPI Written
QSPI Written
capotosto@pstester2:~/MikeDevEnv/NSLS2_zPSC_Calibration_and_Test$
```

13.2.2 Option 2: Calibrate Only

An example screenshot is provided below.
After selecting Menu Option 2, Calibrate Only, the script will sweep the network to detect PSCs. If only one PSC pings, it'll automatically select this unit. If it times out, it'll prompt for the PV prefix of the PSC.

The script reads the EEPROM configuration, and trims the option list to those PSCs that match the EEPROM.

In this example, a 4Ch High-Stability Slow PSC was being calibrated, so we chose Option 5.

A few new prompts will appear:

1. How long do you want to sleep before beginning? (Minutes):
   This is asking how long to delay the script's start. For example, if you want the script to wait 15 minutes for the unit to warm up and thermally stabilize, you can simply type 15 and hit `return`. The script will wait 15 minutes, and then start automatically.

2. Are the correct tuning boards installed in the ATE? <Y/N>:
   This is a reminder to check and install the correct tuning boards in the ATE. This does not have any effect on the script's function - it does not perform any self tests or verification! It is simply a reminder. Incorrect tuning boards will cause the unit to behave unpredictably.
   Install the correct tuning boards before proceeding.

3. Have you run the DMM Self-Calibration within the last 8 hours? <Y/N>:
   This is another reminder prompt. If you haven't, simply perform the auto-cal on the DC Volts range before proceeding.

4. The calibration script will now take the reigns from here. No further user input is required, the calibration will run, printing results to the terminal as it progresses. Once it completes, it will automatically generate a report text file, timestamped with the model and serial number of the PSC in the filename, located in the `/cal_reports` directory.

```
capotosto@pstester2:~/MikeDevEnv/NSLS2_zPSC_Calibration_and_Test$ python3 launcher.py

------------------------------
Select Execution Mode:
1. Initialize QSPI Only
2. Calibrate Only
3. Test Only
4. Calibrate and Test
------------------------------

Enter selection (1-4): 2

Enter PSC serial number: (Enter "H" for help):2
Attempting to auto-detect PSCs
Network Sweep: Attempt 1 of 3...
[*] Success: Discovered PSC 2 at 10.69.26.31
pv_prefix = lab{2}



 Reading EEPROM...
EEPROM # Of Channels: 4
EEPROM Resolution: HS (20bit)
EEPROM Bandwidth: S
EEPROM Polarity: Bipolar, Bipolar, Bipolar, Bipolar




Detected 4 channels. Showing 4-channel models:
--------------------------------------------------
  1) C29-ARI-SXN                          [C29-ARI-SXN]
  2) 4Ch-MSS-ID_XYCorr (ARI/SXN Cell 29)  [4Ch-MSS-ID_XYCorr]
  3) 4CH-MSS-Cur_Strip (ARI/SXN Cell 29)  [4CH-MSS-Cur_Strip]
  4) 4CH-HSF-EIC                          [PSC-4CH-HSF-EIC]
  5) NSLS-II_TEST_4-CH-SLOW__MSS/HSS      [PSC-4-CH-SLOW__MSS/HSS]
  6) NSLS-II_TEST_4-CH-FAST__MSF/HSF      [PSC-4-CH-SLOW__MSF/HSF]
--------------------------------------------------

Enter Type (or 'q' to quit): 5
--> Selected: NSLS-II_TEST_4-CH-SLOW__MSS/HSS

How long do you want to sleep before beginning? (Minutes): 0
Are the correct tuning boards installed in the ATE? <Y/N>: y
Have you run the DMM Self-Calibration within the last 8 hours? <Y/N>y
Sleeping 0 Minutes
Minutes remaining: 0.0
Time remaining: 00:00
Starting Calibration script...



------------------------------------------------------


Calibrating PSC model NSLS-II_TEST_4-CH-SLOW__MSS/HSSSN0002
```

Figure 13.1: Menu to launch a Calibrate Only calibration of a PSC

13.2.3 Option 3: Test Only

An example screenshot is provided below.
After selecting Menu Option 3, Test Only, the script will sweep the network to detect PSCs. If only one PSC pings, it'll automatically select this unit. If it times out, it'll prompt for the PV prefix of the PSC.

The script reads the EEPROM configuration, and trims the option list to those PSCs that match the EEPROM.

In this example, a 4Ch High-Stability Slow PSC was being tested, so we chose Option 5.

A few new prompts will appear:

1. How long do you want to sleep before beginning? (Minutes):
   This is asking how long to delay the script's start. For example, if you want the script to wait 15 minutes for the unit to warm up and thermally stabilize, you can simply type 15 and hit `return`. The script will wait 15 minutes, and then start automatically.

2. Are the correct tuning boards installed in the ATE? <Y/N>:
   This is a reminder to check and install the correct tuning boards in the ATE. This does not have any effect on the script's function - it does not perform any self tests or verification! It is simply a reminder. Incorrect tuning boards will cause the unit to behave unpredictably.
   Install the correct tuning boards before proceeding.

3. The functional test script will now take the reigns from here. No further user input is required, the functional tests will run, printing results to the terminal as it progresses. Once it completes, it will automatically generate a report PDF file, timestamped with the model and serial number of the PSC in the filename, located in the `/test_reports` directory.

   There is one caveat - in that the Fast-bandwidth units will require one input from the user at the very end of the test. The user will be prompted for the SUDO password, as TCPDUMP requires root permission to sniff traffic. The script will pause while waiting for the password, before continuing. This last step requires less than 90 seconds to complete the test and generate the report once the password is entered.

```
capotosto@pstester2:~/MikeDevEnv/NSLS2_zPSC_Calibration_and_Test$ python3 launcher.py

------------------------------
Select Execution Mode:
1. Initialize QSPI Only
2. Calibrate Only
3. Test Only
4. Calibrate and Test
------------------------------

Enter selection (1-4): 3

Enter PSC serial number: (Enter "H" for help):2
Attempting to auto-detect PSCs
Network Sweep: Attempt 1 of 3...
[*] Success: Discovered PSC 2 at 10.69.26.31
pv_prefix = lab{2}



 Reading EEPROM...
EEPROM # Of Channels: 4
EEPROM Resolution: HS (20bit)
EEPROM Bandwidth: S
EEPROM Polarity: Bipolar, Bipolar, Bipolar, Bipolar




Detected 4 channels. Showing 4-channel models:
--------------------------------------------------
  1) C29-ARI-SXN                          [C29-ARI-SXN]
  2) 4Ch-MSS-ID_XYCorr (ARI/SXN Cell 29)  [4Ch-MSS-ID_XYCorr]
  3) 4CH-MSS-Cur_Strip (ARI/SXN Cell 29)  [4CH-MSS-Cur_Strip]
  4) 4CH-HSF-EIC                          [PSC-4CH-HSF-EIC]
  5) NSLS-II_TEST_4-CH-SLOW__MSS/HSS      [PSC-4-CH-SLOW__MSS/HSS]
  6) NSLS-II_TEST_4-CH-FAST__MSF/HSF      [PSC-4-CH-SLOW__MSF/HSF]
--------------------------------------------------

Enter Type (or 'q' to quit): 5
--> Selected: NSLS-II_TEST_4-CH-SLOW__MSS/HSS

How long do you want to sleep before beginning? (Minutes): 0
Are the correct tuning boards installed in the ATE? <Y/N>: y
Sleeping 0 Minutes
Minutes remaining: 0.0
Time remaining: 00:00
Beginning functional test...
```

Figure 13.2: Menu to launch a Test Only test of a PSC

13.2.4 Option 4: Calibrate and Test

This option will have a menu identical to that of Option 2: Calibrate Only. This simply runs the calibration script, then on completion, runs the test script. No additional figures have been included for this.

Appendix A

Example Secondary Ops Checklist

4-Channel Power Supply Controller Checklist

S/N:_____ **Date:**_____ **Initials:**_____

☐ **1. Adjust Power Supplies**

A. Disconnect the following power connections prior to adjusting:
- Controller Filter Board: J1 & J2
- Digital Board: J2 & J4
- Regulator Board: J1 & J2

B. Power on the unit, ensure each power supply has the corresponding green LED lit.

C. Adjust the voltages as follows:
- ☐ +6.3V on rightmost Power Supply (Red lead on +V, Black lead on -V)
- ☐ -16.5V on middle Power Supply (Black lead on +V, Red lead on -V)
- ☐ +16.5V on leftmost Power Supply (Red lead on +V, Black lead on -V)

D. Turn off the Unit.

☐ **2. Install Jumpers and Fuses**

Controller Filter Board
- ☐ (2) White Jumpers
- ☐ (3) 3.15A Fuses

Digital Board
- ☐ (1) White Jumper
- ☐ (3) 3.15A Fuses

Regulator Board
- ☐ (55) White Jumpers[1]
- ☐ (1) 1A Fuse – [F2]
- ☐ (2) 2.15A Fuses – [F1, F3]
- ☐ (3) Blue Jumpers – [W1, W2, W3]

☐ **3. Reinstall Power Connections**

☐ **4. Perform Voltage Measurements (see page 2)**

☐ **5. Install SOM Board, Heat Sink, and Fan**
- Ensure Dip Switches are both pushed towards the left edge (SD Card Mode)
- Ensure SOM board green LED is lit [D3]
- Install the heat sink parallel to the Digital Board (air flow directed out of back)
- #4 flat washers may be required to ensure fan does not catch on heat sink

☐ **6. Install Burden Resistor Boards**

R1:_____ R2:_____ R3:_____ R4:_____

☐ **7. Install Compensation Boards**

X1:_____µF X2:_____Ω

☐ **8. Install SD Card into slot, ensure blue LED is lit. [D3]**

☐ **9. Unit Passed** **Initials**_____

Comments:_____

[1] Do not install: W16, W24, W28, W32. Install all vertical 3-prongs on the top two prongs.
For horizontal 3-prong: Install W15, W23, W27, W31 on two left prongs; for the other 8 install on two right prongs.

Voltage Measurements

1. **Power on the Unit and Check the Following LEDs:**

   Note: If any are not lit, use a thermal imager to assess the board

   A. Regulator Board
      - (8) Orange [D52, D6, D8, D9, D12, D11, D16, D15]

   B. Digital Board
      - (3) Orange [DS1002, DS1001, DS1000]
      - (3) Green [D1, D5, D4]

2. **Measure the following on the Digital / Regulator Boards. Record values.**

   Note: U23 Tab is Universal Ground (G)

1. TP2 → G. Value should be +6.2V ± 0.1V _____V
2. TP3 → G. Value should be +5.0V ± 0.1V _____V
3. TP4 → G. Value should be +1.8V ± 0.1V _____V
4. TP5 → G. Value should be +3.3V ± 0.1V _____V
5. TP6 → G. Value should be +1.0V ± 0.1V _____V
6. TP7 → G. Value should be +1.2V ± 0.1V _____V
7. TP9 → G. Value should be +1.8V ± 0.1V _____V
8. U30 pin 1 → G. Value should be -16.5V ± 0.1V _____V
9. U30 pin 4 → G. Value should be -15.0V ± 0.1V _____V
10. U33 pin 4 → G. Value should be +10V ± 0.1V _____V
11. U34 pin 4 → G. Value should be +5.3V ± 0.1V _____V
12. U29 pin 3 → G. Value should be +15.0V ± 0.1V _____V
13. U29 pin 1 → G. Value should be +16.5V ± 0.1V _____V
14. U35 pin 3 → G. Value should be +2.5V ± 0.1V _____V
15. U32 pin 5 → G. Value should be -5.25V ± 0.1V _____V
16. U31 pin 5 → G. Value should be -9.85V ± 0.15V _____V
17. U1003 pin 4 → G. Value should be +3.3V ± 0.1V _____V

---

1. U1 pin 3 → G. Value should be +5.0V ± 0.1V _____V
2. U7 pin 3 → G. Value should be +5.0V ± 0.1V _____V
3. U53 pin 1 → G. Value should be +16.5V ± 0.1V _____V
4. U53 pin 3 → G. Value should be +15.0V ± 0.2V _____V
5. U54 pin 1 → G. Value should be -16.5V ± 0.1 V _____V
6. U54 pin 5 → G. Value should be -15.0V ± 0.2V _____V
7. U51 pin 3 → G. Value should be +15.0V ± 0.2V _____V
8. U52 pin 5 → G. Value should be -15.0V ± 0.2V _____V
9. U43 pin 3 → G. Value should be +15.0V ± 0.2V _____V
10. U50 pin 5 → G. Value should be -15.0V ± 0.2V _____V

Appendix B

Example Calibration Report

```
Report of Calibration
PSC PSC-4CH-HSF-EIC_S/N 0001
2026-05-19 14:08:16

Calibration Current standard: BNL PSC ATE S/N 001
Calibration Volt standard: HP 3458A-002 S/N 2823A 23647
Calibration Resistance standard: Fluke 742A-1 S/N 1063008
End Header


lab{2}Chan1
Burden resistor = 33.3333

Measuring initial gains and offsets
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.002134    1.004211   -1.011351   -1.011165    1.001779    0.023016
  -27.001049   27.014790  -26.827587  -26.827612   27.041689    0.052203

                                  dacSP         dcct1         dcct2         dacRB
Initial measured offsets:      -0.001627     -0.016258     -0.016064      0.000000
Initial measured gains:         1.000449      0.992974      0.992982      1.001128
Gain corrections:               1.000449      1.007076      1.007068      1.000000

Writing gain and offset corrections for dacSP, dcct1, and dcct2 to PSC

Measuring DAC readback gain and offset
DAC SP   DAC RB
1.000000   0.999602
27.000000   27.040516

Measured offset: -0.001971
Measured gain: 1.001574
Gain correction: 0.998429

Writing gain and offset constants for dacRB to PSC

Verification
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.002083    1.001881   -1.002053   -1.002082    1.002442    0.007083
  -27.000989   27.000880  -27.000942  -27.000990   27.001160    0.039925


                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets:        0.000206      0.000029      0.000002      0.000572
Final measured gains:          1.000004      0.999999      1.000000      0.999989



                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets mean: -0.000061     -0.000069     -0.000082      0.000454
Final measured offsets stdev: 0.000333      0.000085      0.000065      0.000214
Final measured gains mean:    0.999997      0.999993      0.999994      0.999999
Final measured gains stdev:   0.000009      0.000007      0.000006      0.000006

Saving channel 1 calibration constants to qspi



Page 1 of 4
```

```
lab{2}Chan2
Burden resistor = 33.3333

Measuring initial gains and offsets
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.001896    1.005391   -1.013423   -1.010625    1.001104   -0.006992
  -27.001133   27.010829  -26.821861  -26.799139   27.052780    0.013247

                                  dacSP         dcct1         dcct2         dacRB
Initial measured offsets:      -0.003256     -0.018880     -0.016849      0.000000
Initial measured gains:         1.000239      0.992661      0.991895      1.001778
Gain corrections:               1.000239      1.007393      1.008171      1.000000

Writing gain and offset corrections for dacSP, dcct1, and dcct2 to PSC

Measuring DAC readback gain and offset
DAC SP   DAC RB
1.000000   0.999262
27.000000   27.051821

Measured offset: -0.002759
Measured gain: 1.002021
Gain correction: 0.997983

Writing gain and offset constants for dacRB to PSC

Verification
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.002069    1.001908   -1.002223   -1.002130    1.002042   -0.037007
  -27.001283   27.001929  -27.001709  -27.001572   27.002224    0.044991


                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets:        0.000192     -0.000143     -0.000052      0.000128
Final measured gains:          1.000031      1.000010      1.000009      1.000006



                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets mean:   0.000065     -0.000055     -0.000032     -0.000442
Final measured offsets stdev:  0.000093      0.000066      0.000051      0.000311
Final measured gains mean:     1.000006      0.999993      0.999994      1.000005
Final measured gains stdev:    0.000015      0.000010      0.000010      0.000005

Saving channel 2 calibration constants to qspi
```

```
lab{2}Chan3
Burden resistor = 33.3333

Measuring initial gains and offsets
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.001365    1.004346   -1.010886   -1.009898    1.003824    0.017980
  -27.000641   27.000858  -26.807045  -26.813667   27.014460    0.049934

                                  dacSP         dcct1         dcct2         dacRB
Initial measured offsets:      -0.003087     -0.017343     -0.016062      0.000000
Initial measured gains:         0.999894      0.992188      0.992480      1.000543
Gain corrections:               0.999894      1.007874      1.007577      1.000000

Writing gain and offset corrections for dacSP, dcct1, and dcct2 to PSC

Measuring DAC readback gain and offset
DAC SP   DAC RB
1.000000   1.002393
27.000000   27.013741

Measured offset: 0.001956
Measured gain: 1.000436
Gain correction: 0.999564

Writing gain and offset constants for dacRB to PSC

Verification
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.001401    1.001458   -1.001531   -1.001573    1.002080    0.006694
  -27.000487   27.000075  -27.000189  -27.000257   27.000923    0.026097


                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets:        -0.000075     -0.000147     -0.000188      0.000614
Final measured gains:           0.999982      0.999984      0.999985      1.000009



                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets mean:    0.000022     -0.000128     -0.000140      0.000769
Final measured offsets stdev:   0.000177      0.000094      0.000082      0.000121
Final measured gains mean:      0.999991      0.999989      0.999989      1.000001
Final measured gains stdev:     0.000020      0.000008      0.000008      0.000013

Saving channel 3 calibration constants to qspi
```

```
lab{2}Chan4
Burden resistor = 33.3333

Measuring initial gains and offsets
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.001620    1.004751   -1.011366   -1.010974    1.001680    0.017581
  -27.000917   27.003424  -26.819748  -26.814087   27.035374    0.054696

                                  dacSP         dcct1         dcct2         dacRB
Initial measured offsets:      -0.003156     -0.017101     -0.016912      0.000000
Initial measured gains:         0.999976      0.992657      0.992454      1.001347
Gain corrections:               0.999976      1.007397      1.007603      1.000000

Writing gain and offset corrections for dacSP, dcct1, and dcct2 to PSC

Measuring DAC readback gain and offset
DAC SP   DAC RB
1.000000   1.000222
27.000000   27.034683

Measured offset: -0.001104
Measured gain: 1.001325
Gain correction: 0.998676

Writing gain and offset constants for dacRB to PSC

Verification
       Itest       dacSP       dcct1       dcct2       dacRB         err
   -1.001558    1.001739   -1.001855   -1.001890    1.001905    0.007935
  -27.001001   27.000845  -27.000896  -27.000874   27.000404    0.040057


                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets:        -0.000194     -0.000313     -0.000350      0.000189
Final measured gains:           0.999987      0.999985      0.999982      0.999977



                                  dacSP         dcct1         dcct2         dacRB
Final measured offsets mean:    0.000009     -0.000107     -0.000094      0.000065
Final measured offsets stdev:   0.000235      0.000187      0.000201      0.000217
Final measured gains mean:      0.999993      0.999989      0.999988      0.999994
Final measured gains stdev:     0.000006      0.000006      0.000005      0.000013

Saving channel 4 calibration constants to qspi




Test data reviewed by _____   Date_____

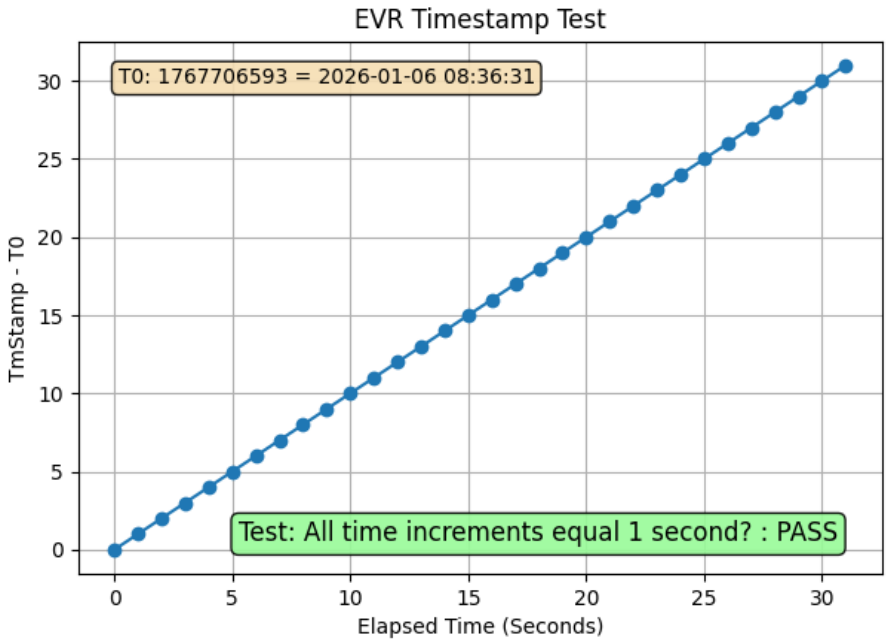

Page 4 of 4
```

Appendix C

Example Test Report

PSC Functional Test Report

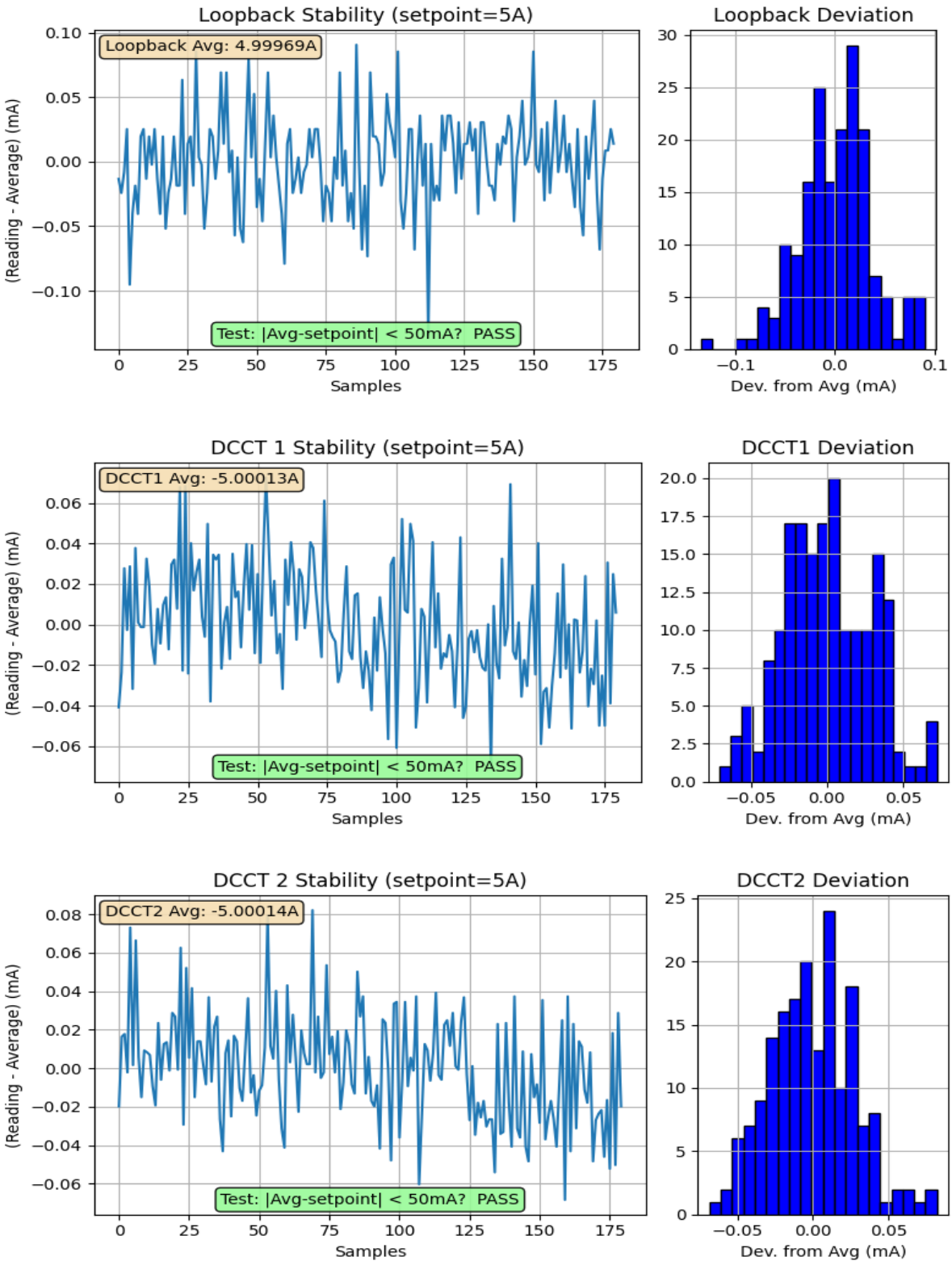
PSC Functional Test Results	
Power Supply Controller Configuration	
Description	PSC-4CH-HSF-EIC
Serial Number	0001
Number of Channels	4
Resolution	HS (20bit)
Bandwidth	F
Polarity	Bipolar



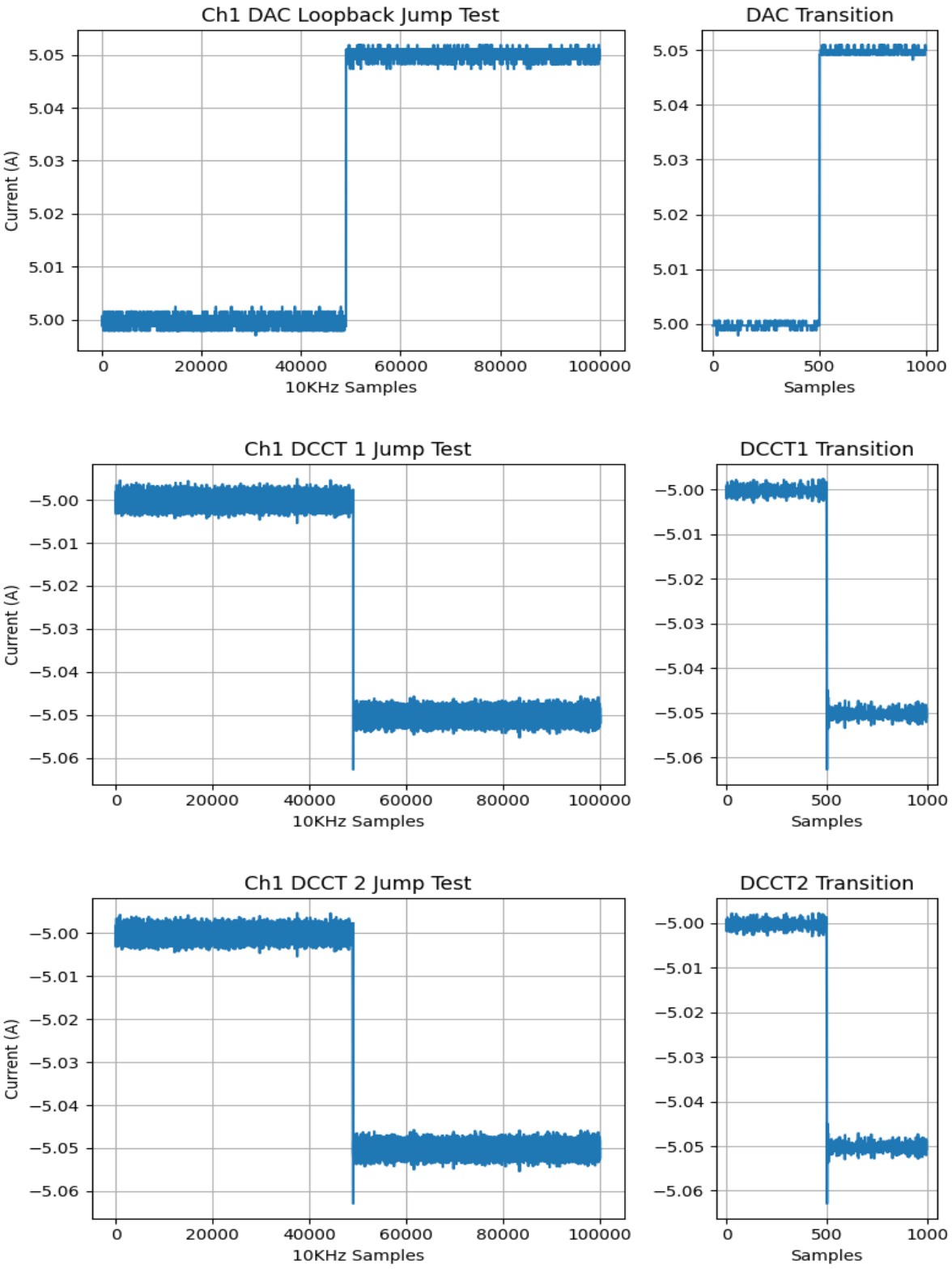
Channel 1

ATE Fault Tests for Channel 1	
Fault #1 Generated and Cleared	PASS
Fault #2 Generated and Cleared	PASS
Fault SPARE Generated and Cleared	PASS
Fault DCCT Generated and Cleared	PASS

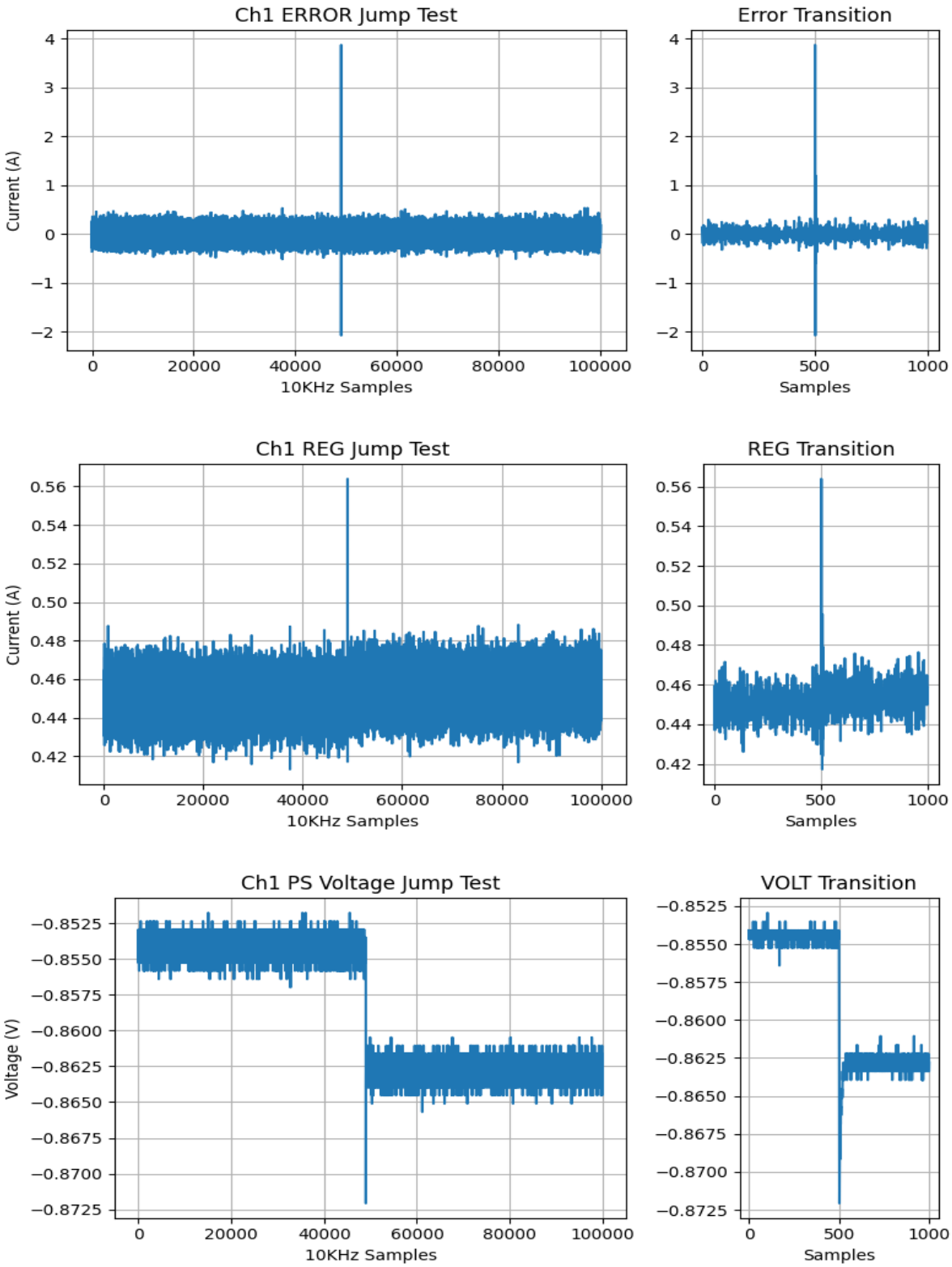
Power Supply Regulation for Channel 1:



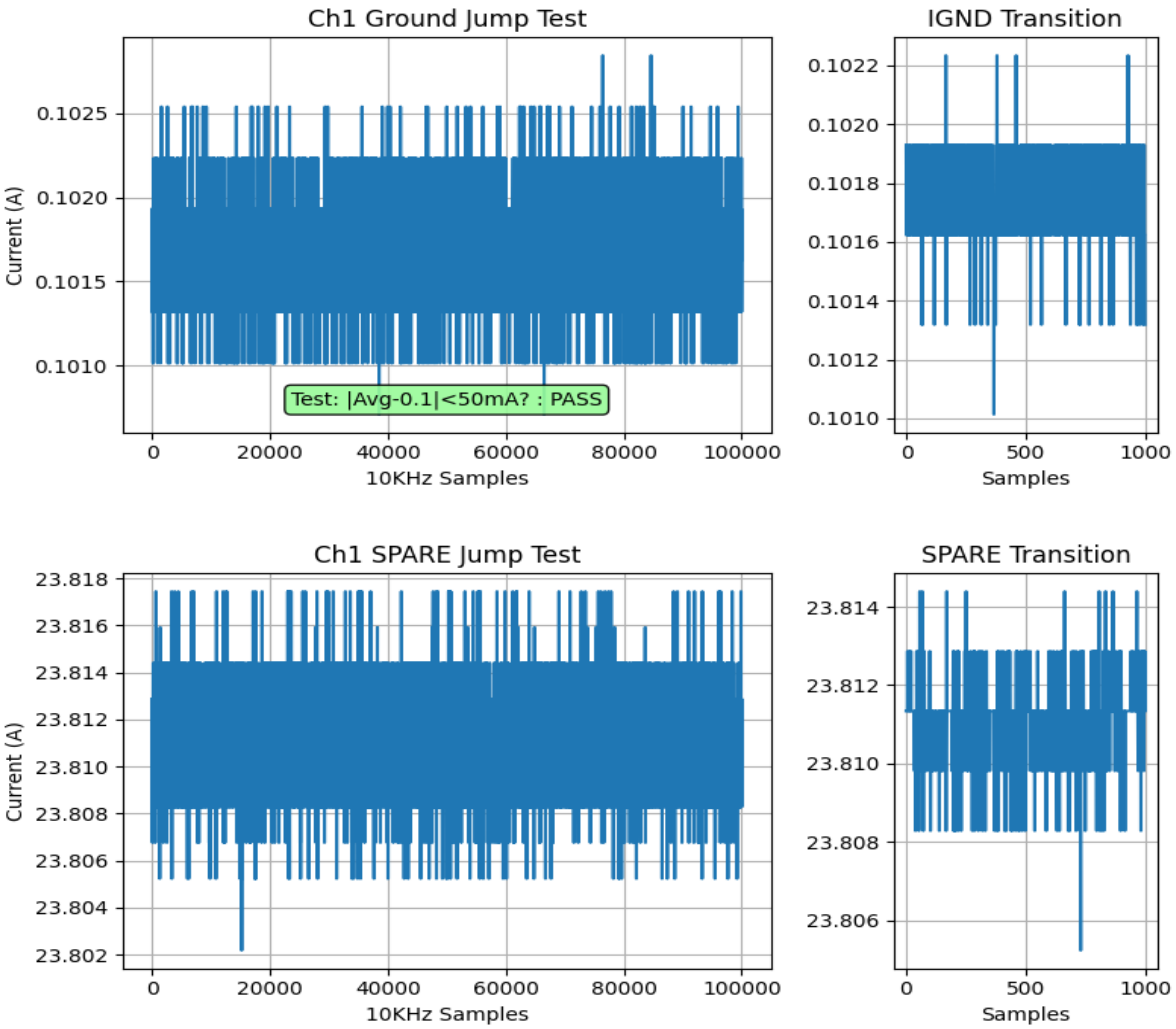
Jump Test Results: CH1



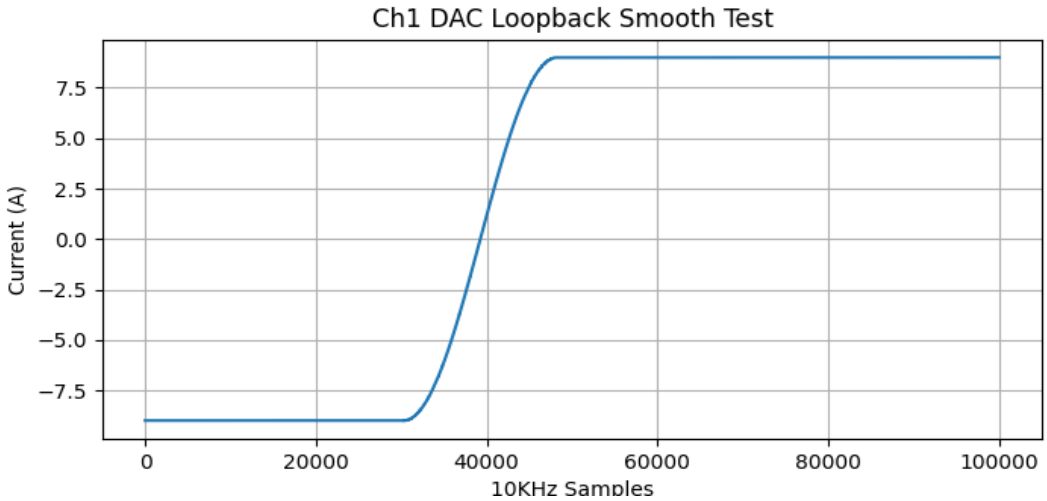
Jump Test Results: CH1

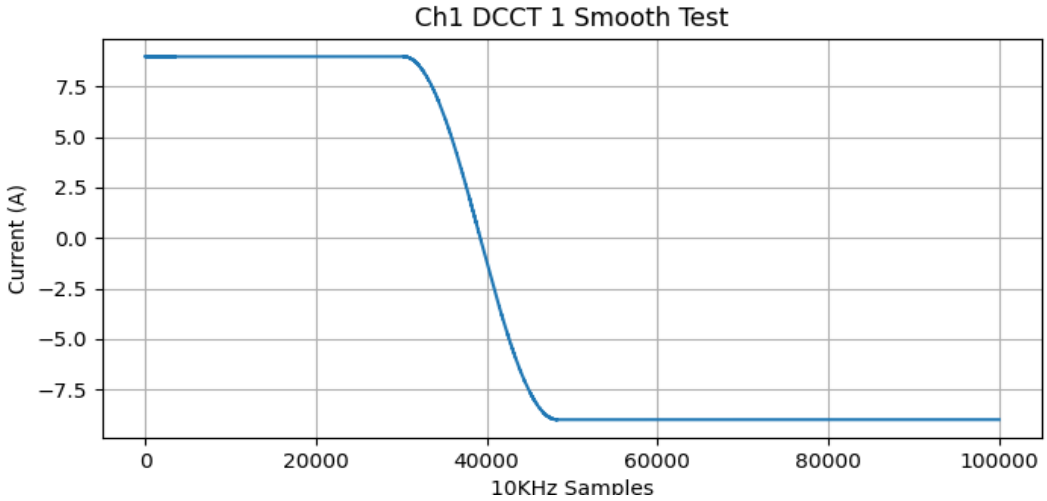


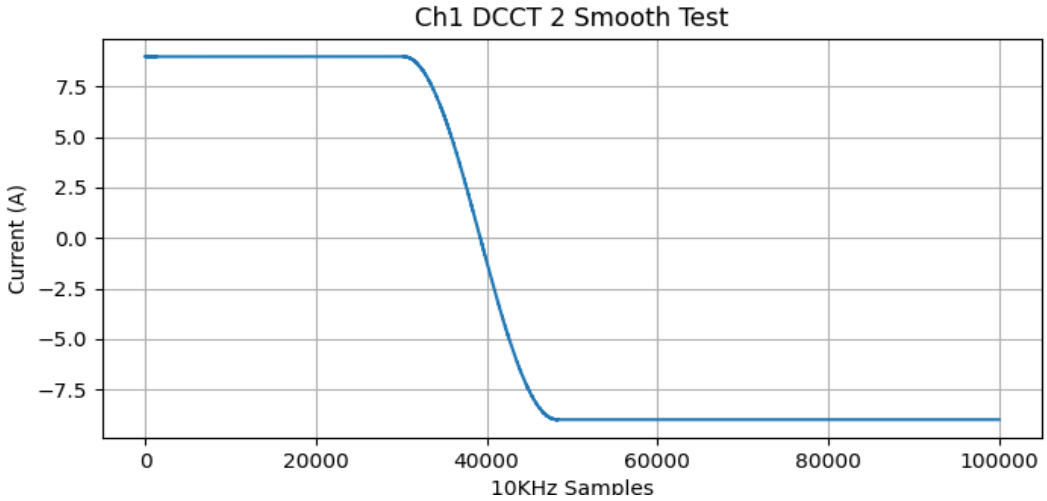
Jump Test Results: CH1



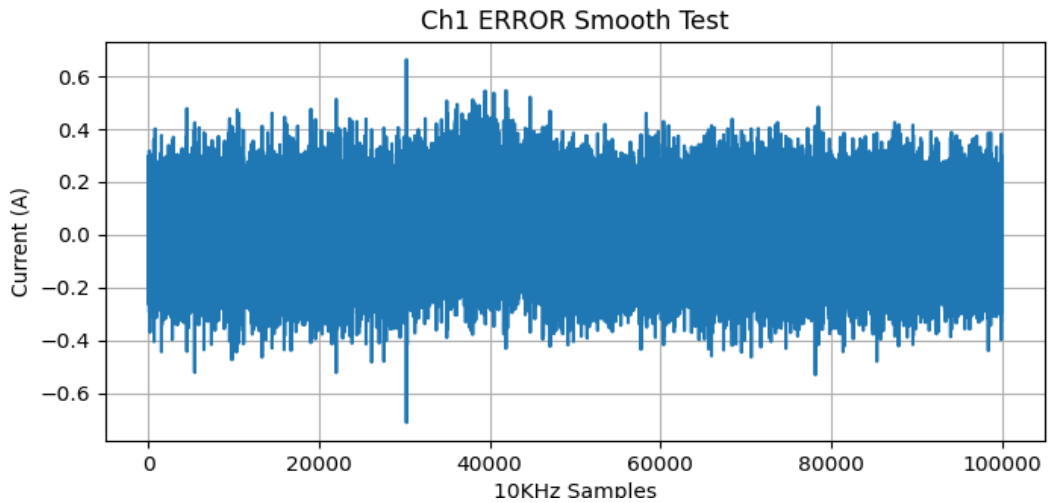
Smooth Test Results: CH1

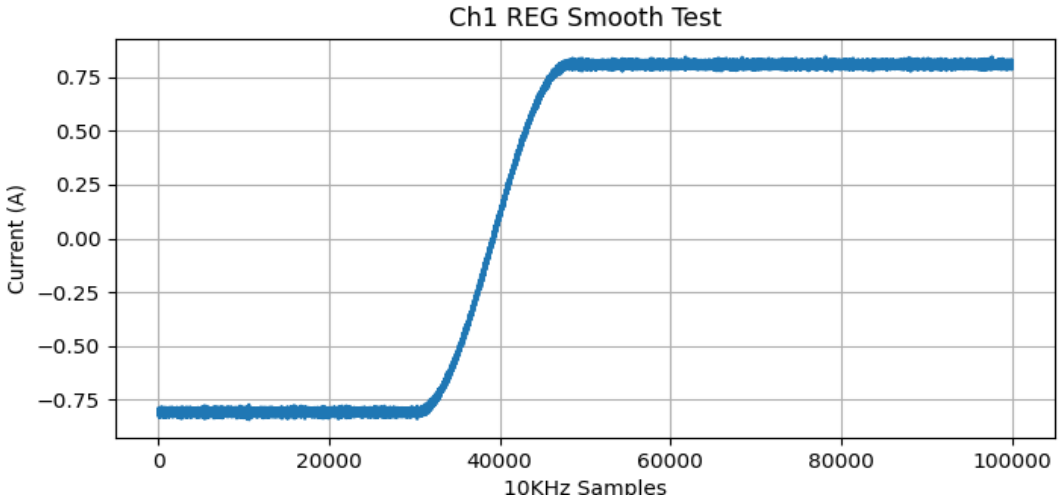


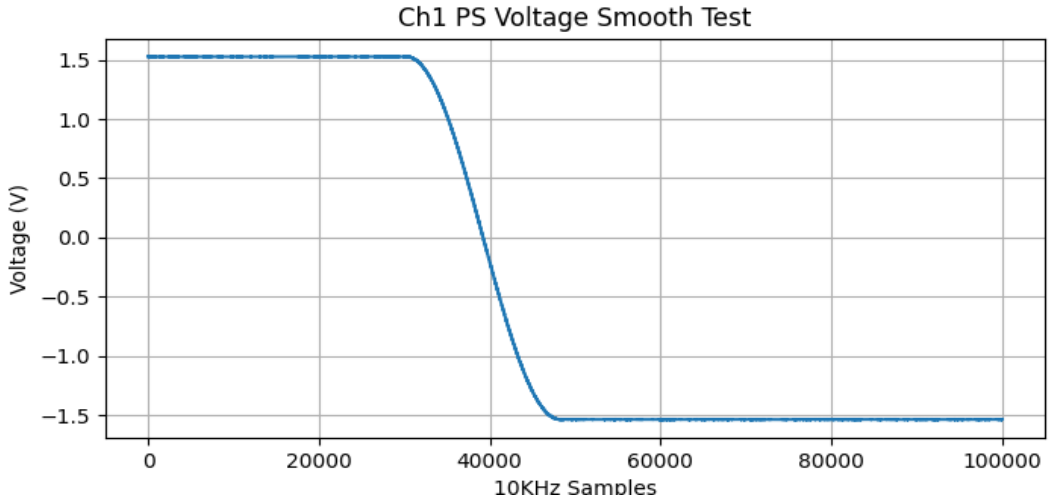




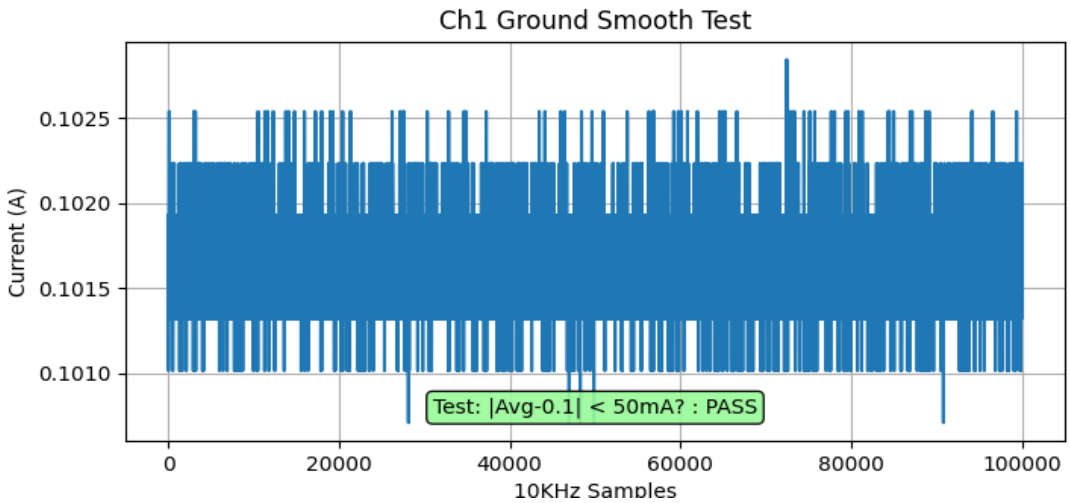
Smooth Test Results: CH1

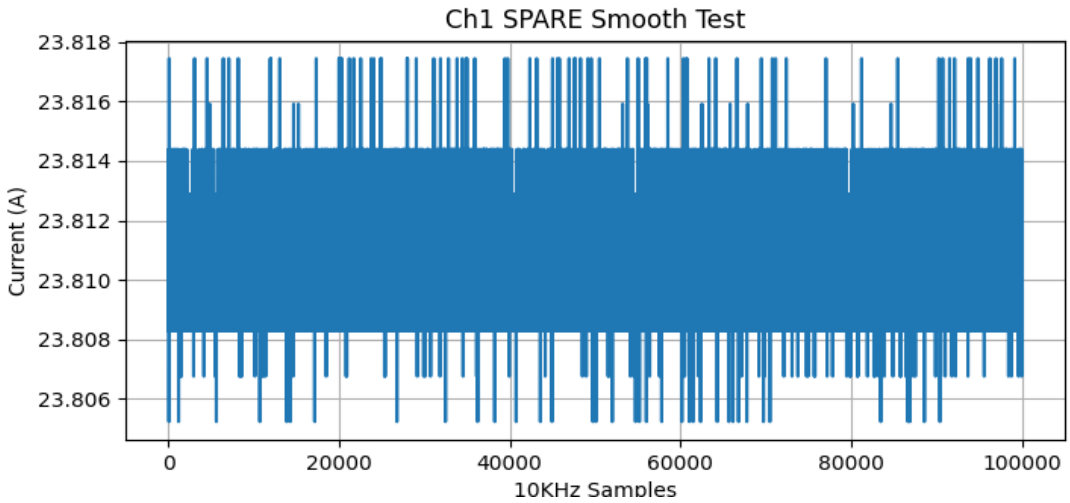






Smooth Test Results: CH1

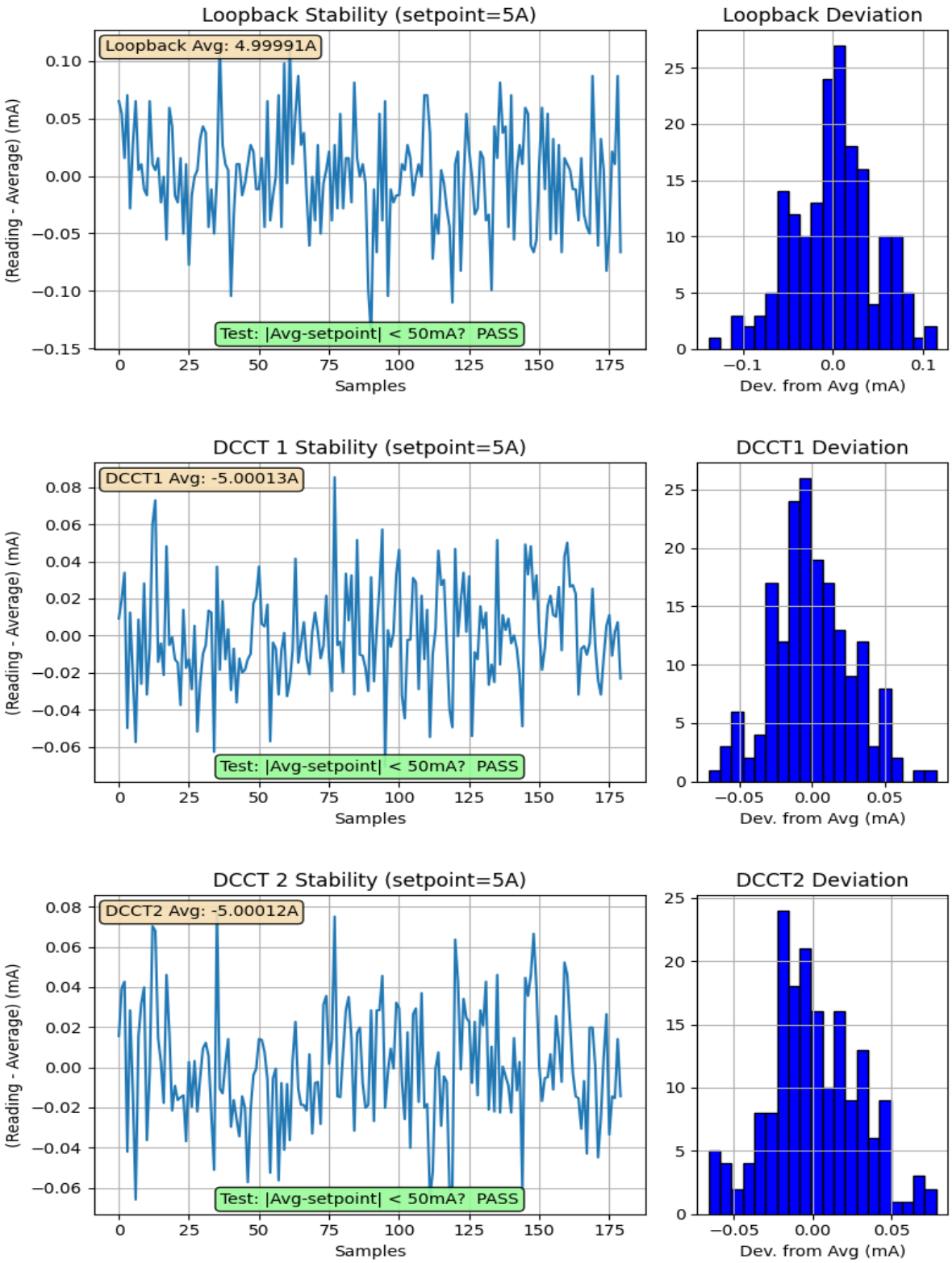




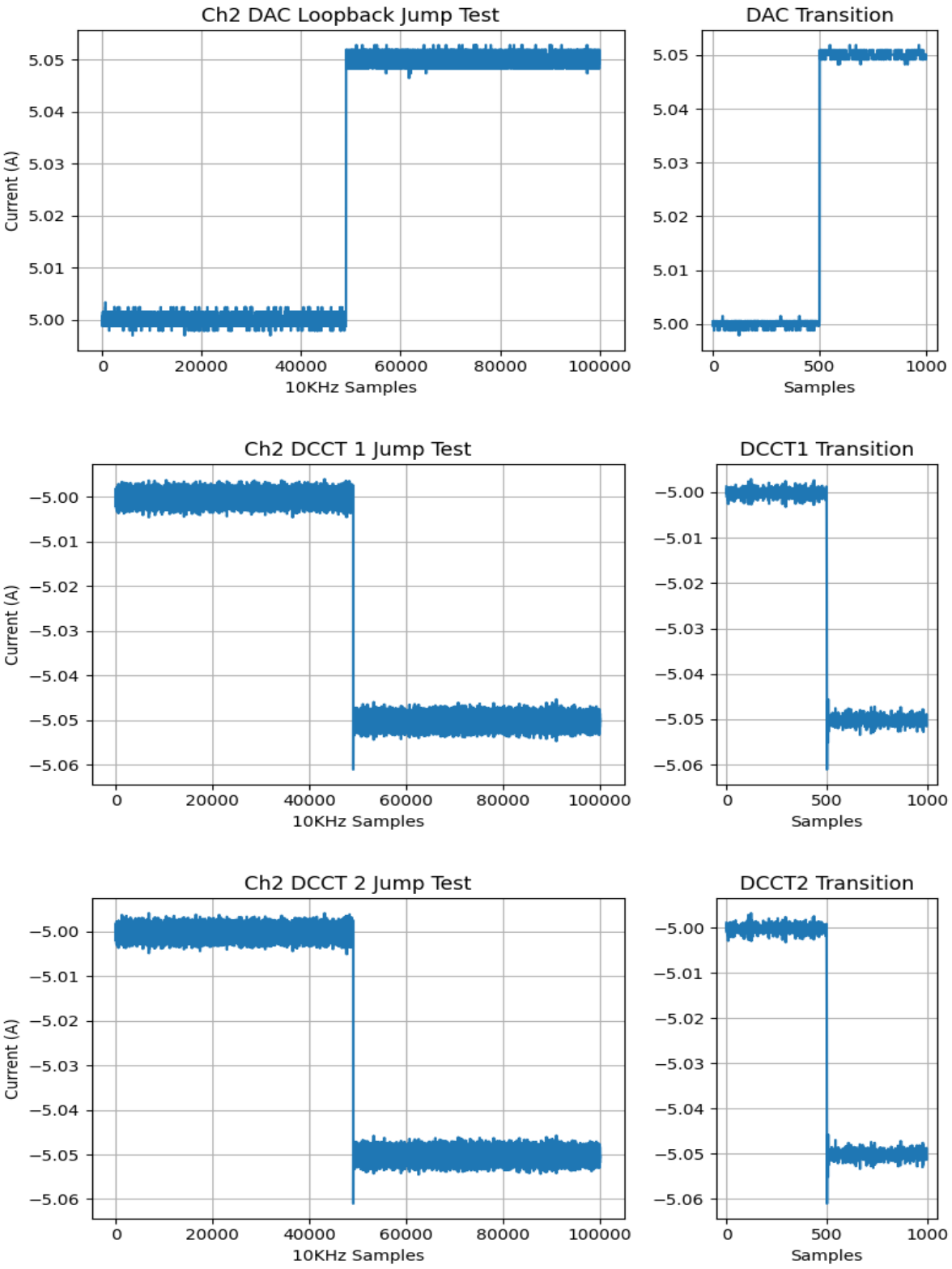
Channel 2

ATE Fault Tests for Channel 2	
Fault #1 Generated and Cleared	PASS
Fault #2 Generated and Cleared	PASS
Fault SPARE Generated and Cleared	PASS
Fault DCCT Generated and Cleared	PASS

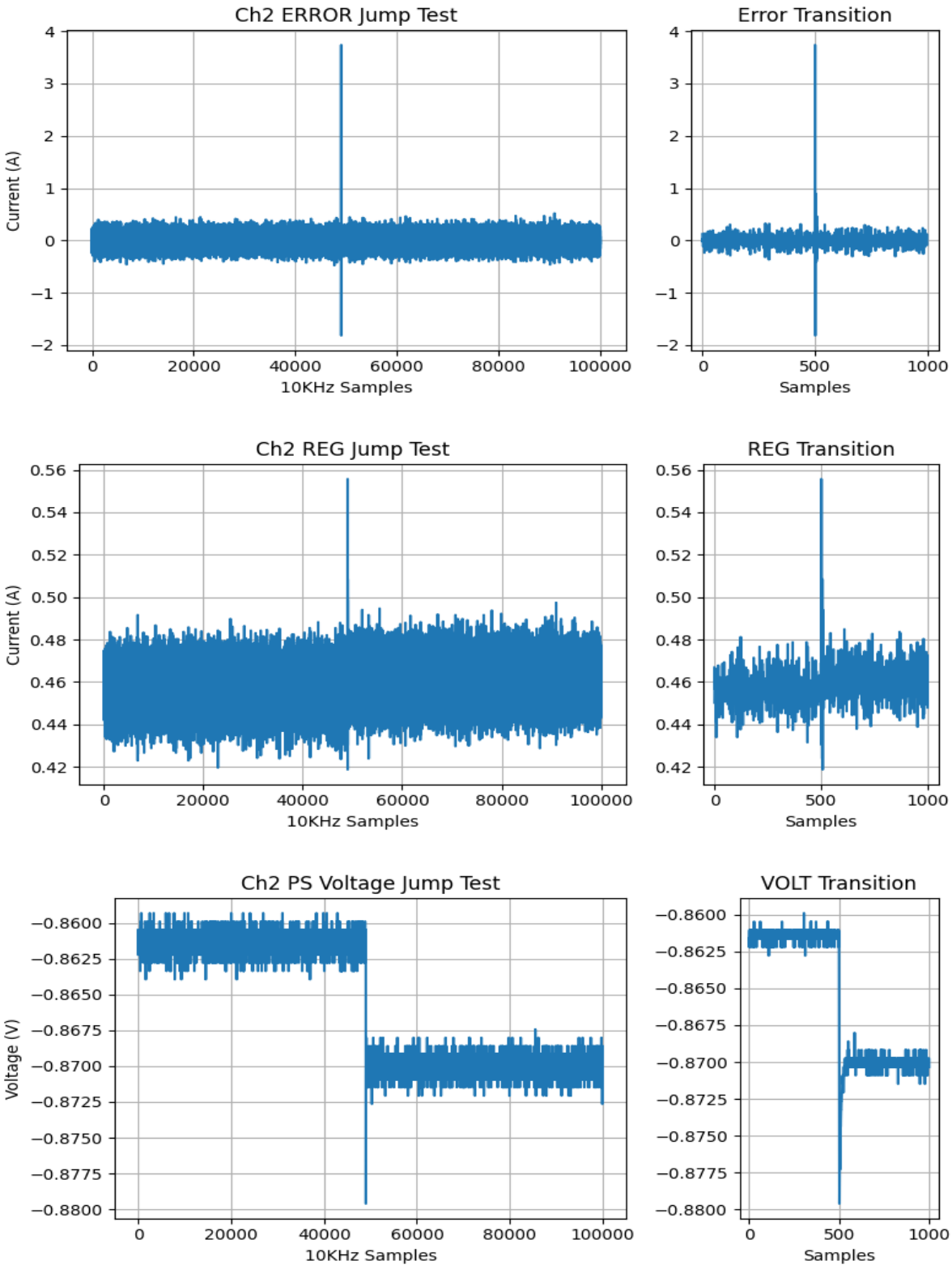
Power Supply Regulation for Channel 2:



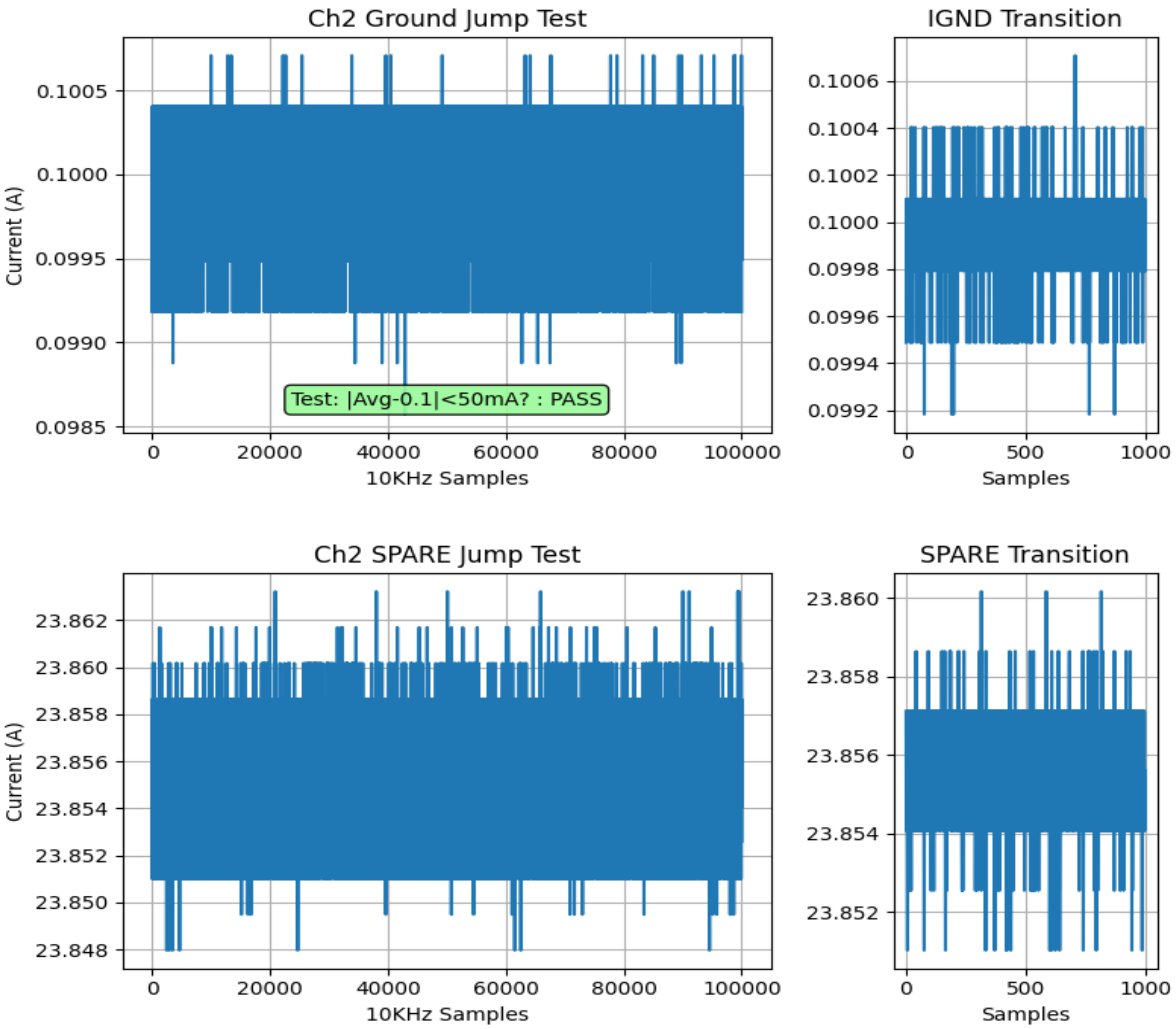
Jump Test Results: CH2



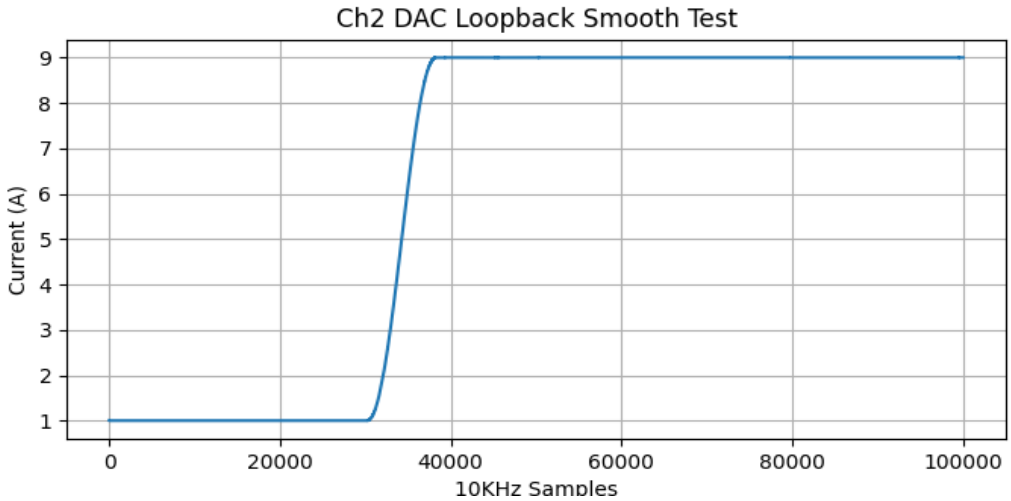
Jump Test Results: CH2

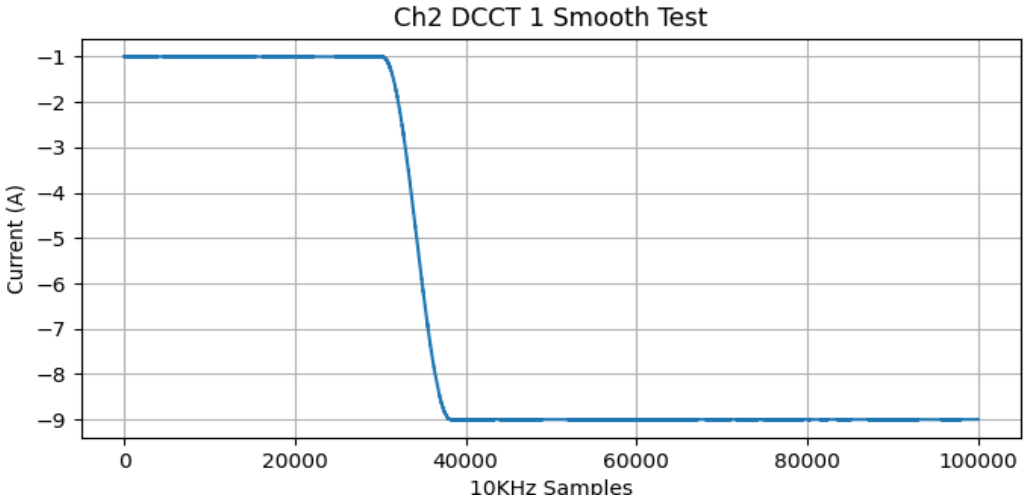


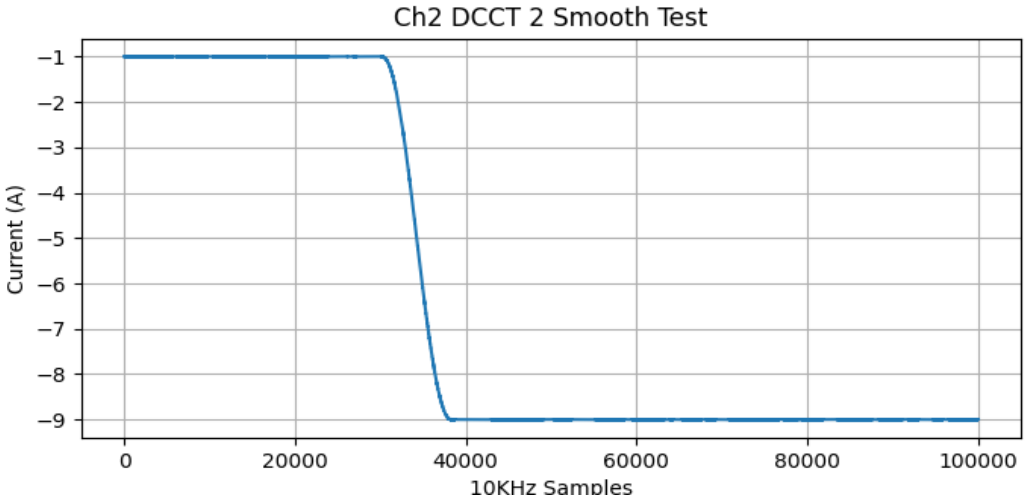
Jump Test Results: CH2



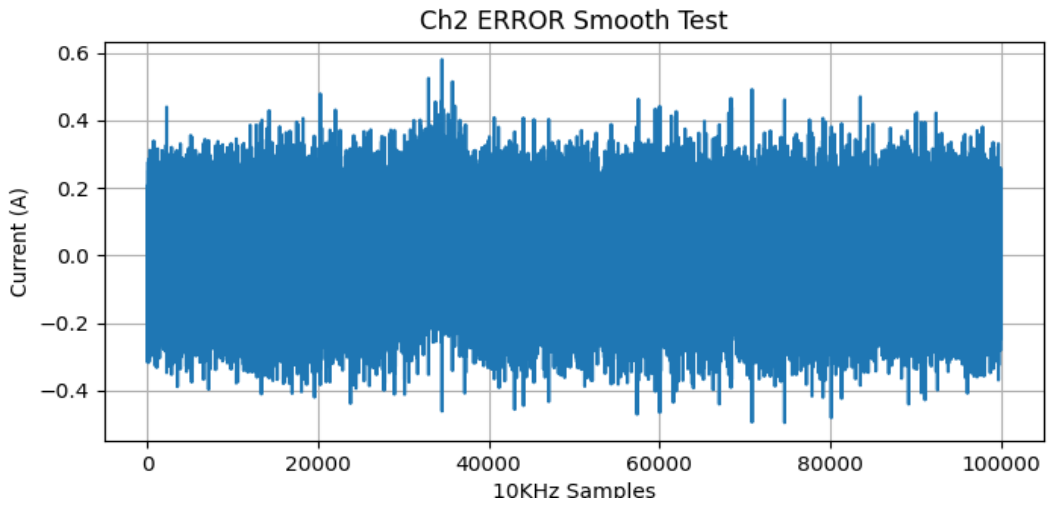
Smooth Test Results: CH2

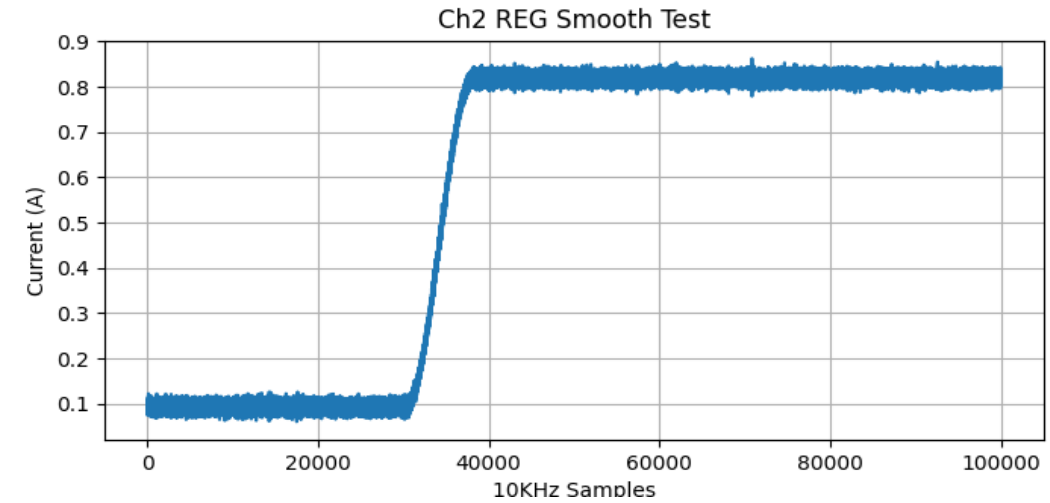


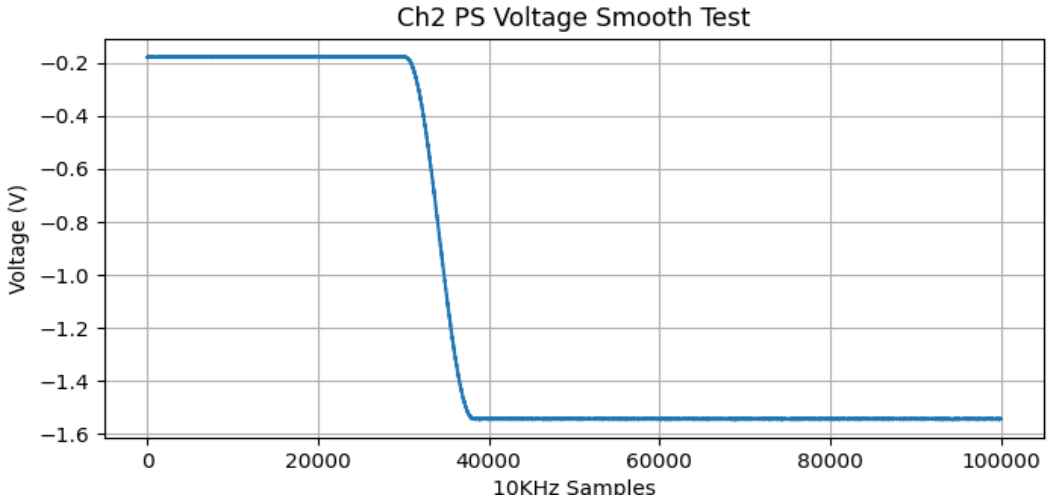




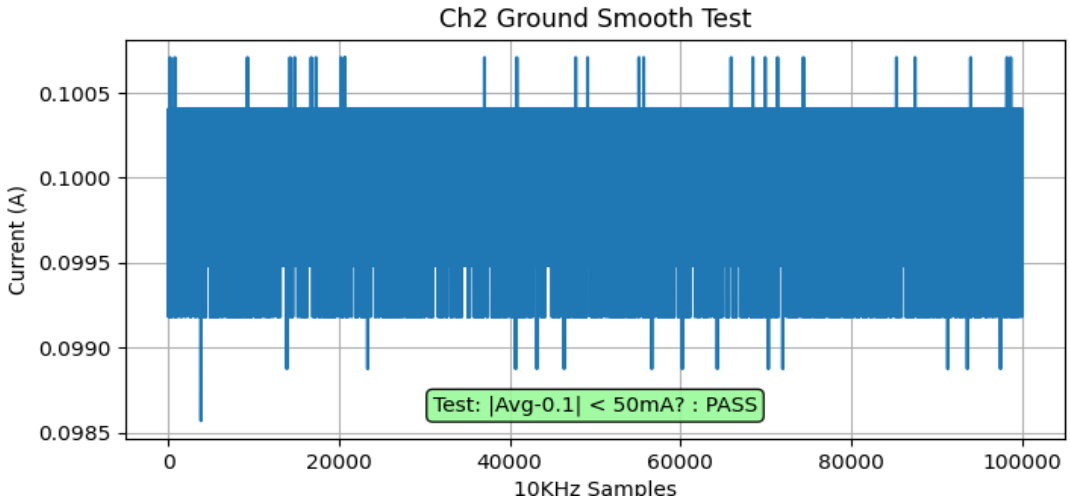
Smooth Test Results: CH2

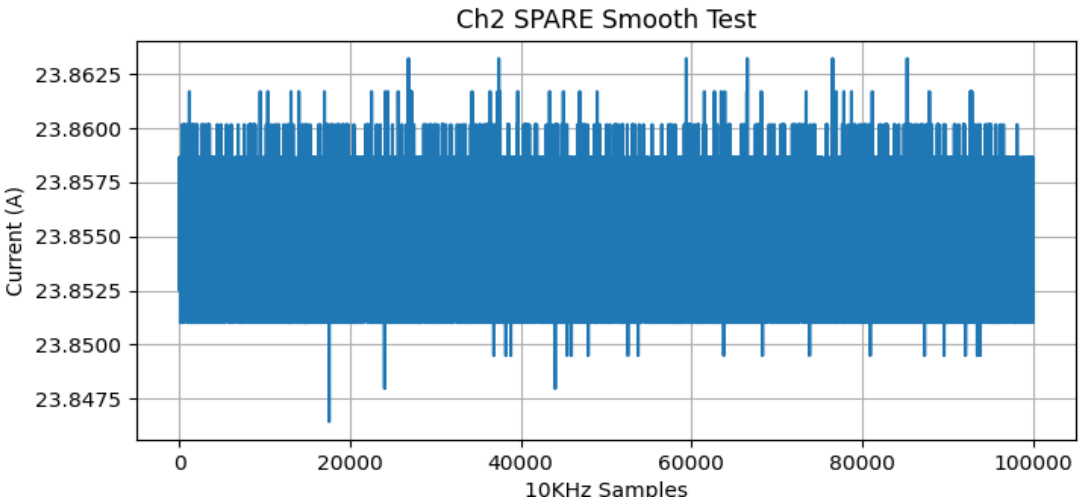






Smooth Test Results: CH2

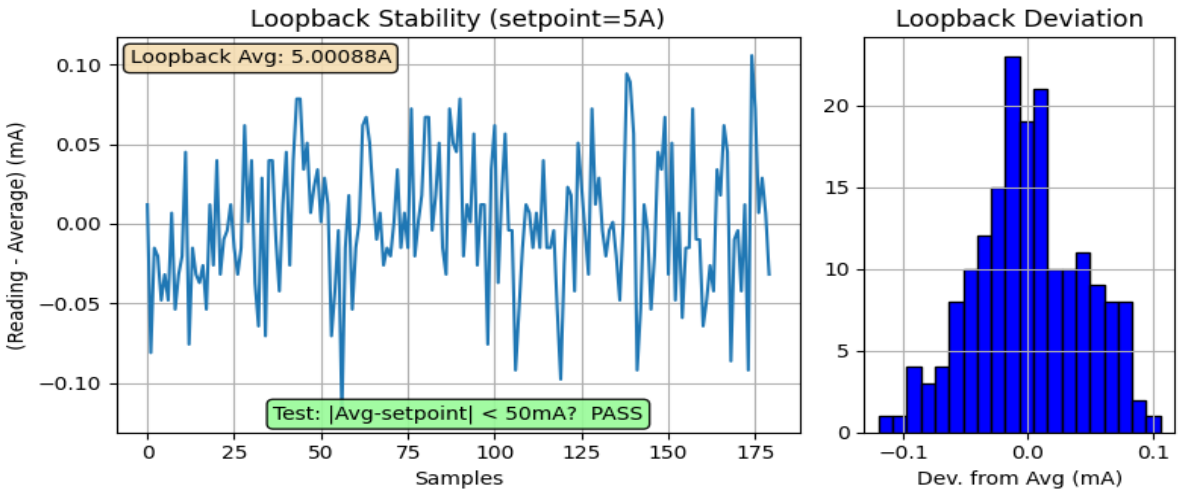


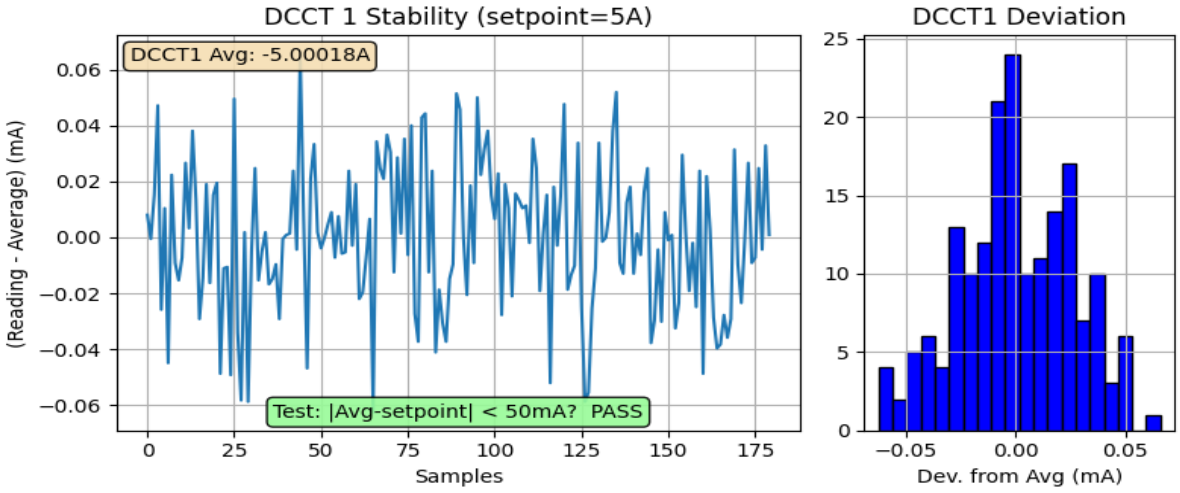


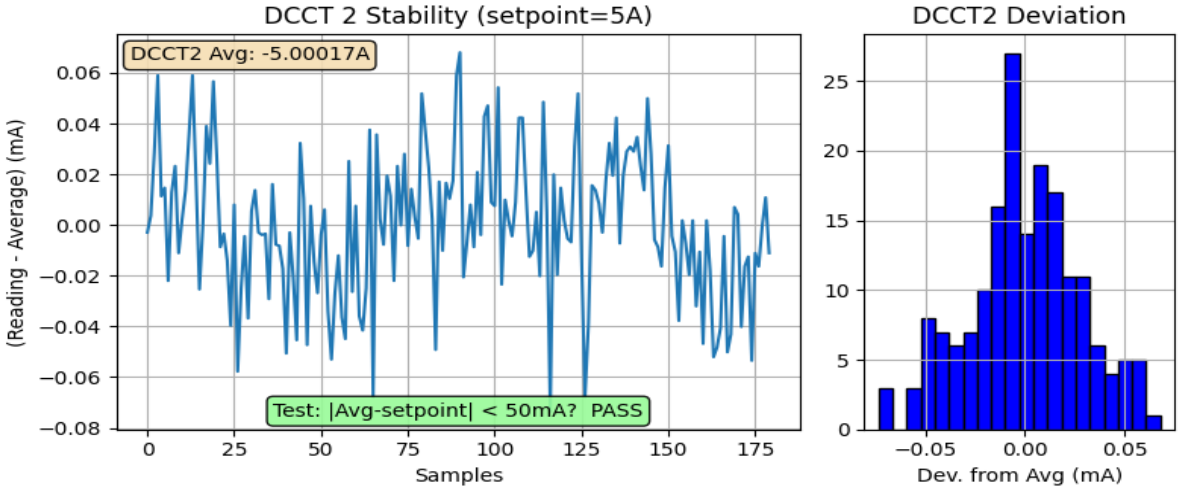
Channel 3

ATE Fault Tests for Channel 3	
Fault #1 Generated and Cleared	PASS
Fault #2 Generated and Cleared	PASS
Fault SPARE Generated and Cleared	PASS
Fault DCCT Generated and Cleared	PASS

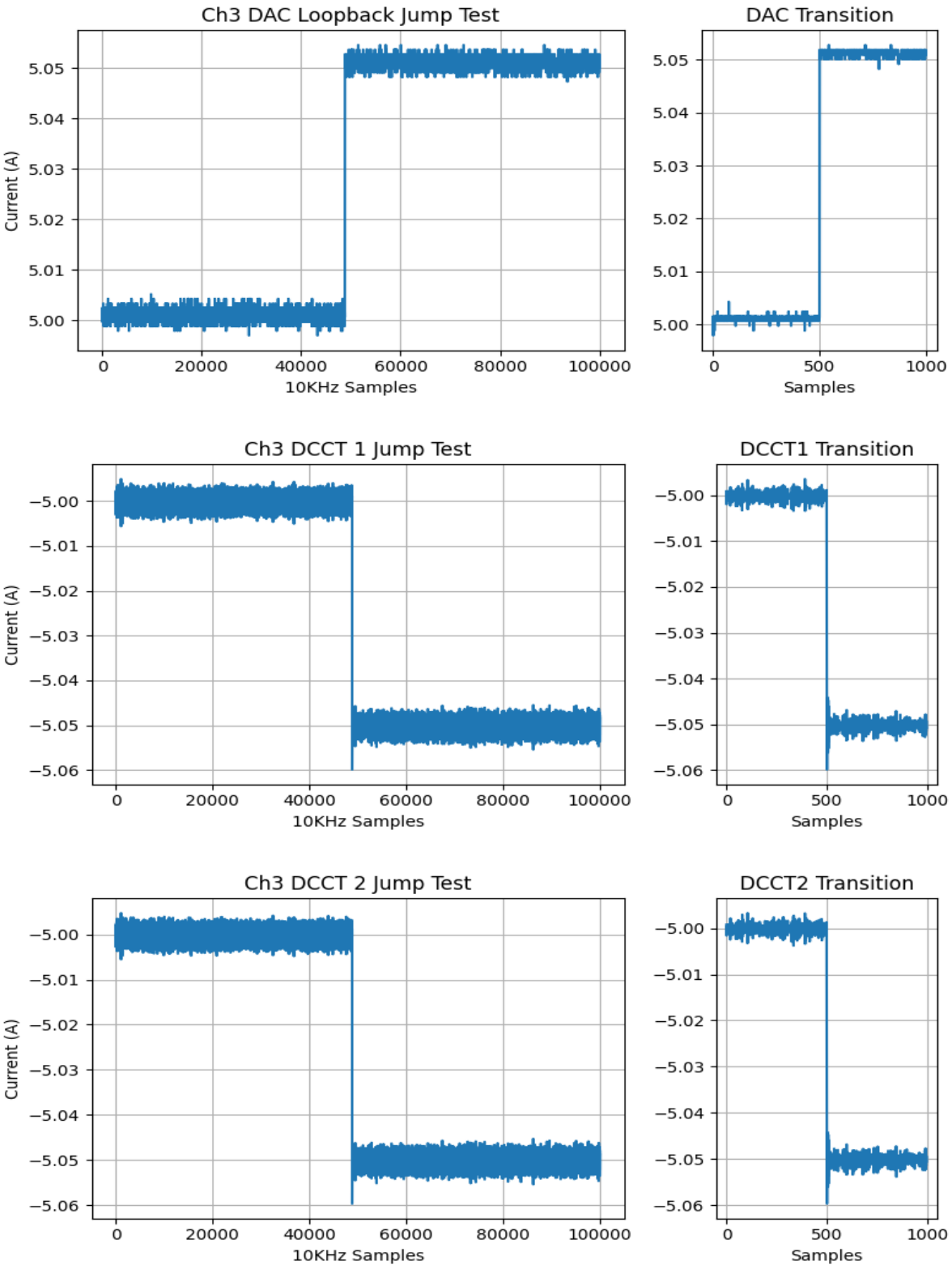
Power Supply Regulation for Channel 3:



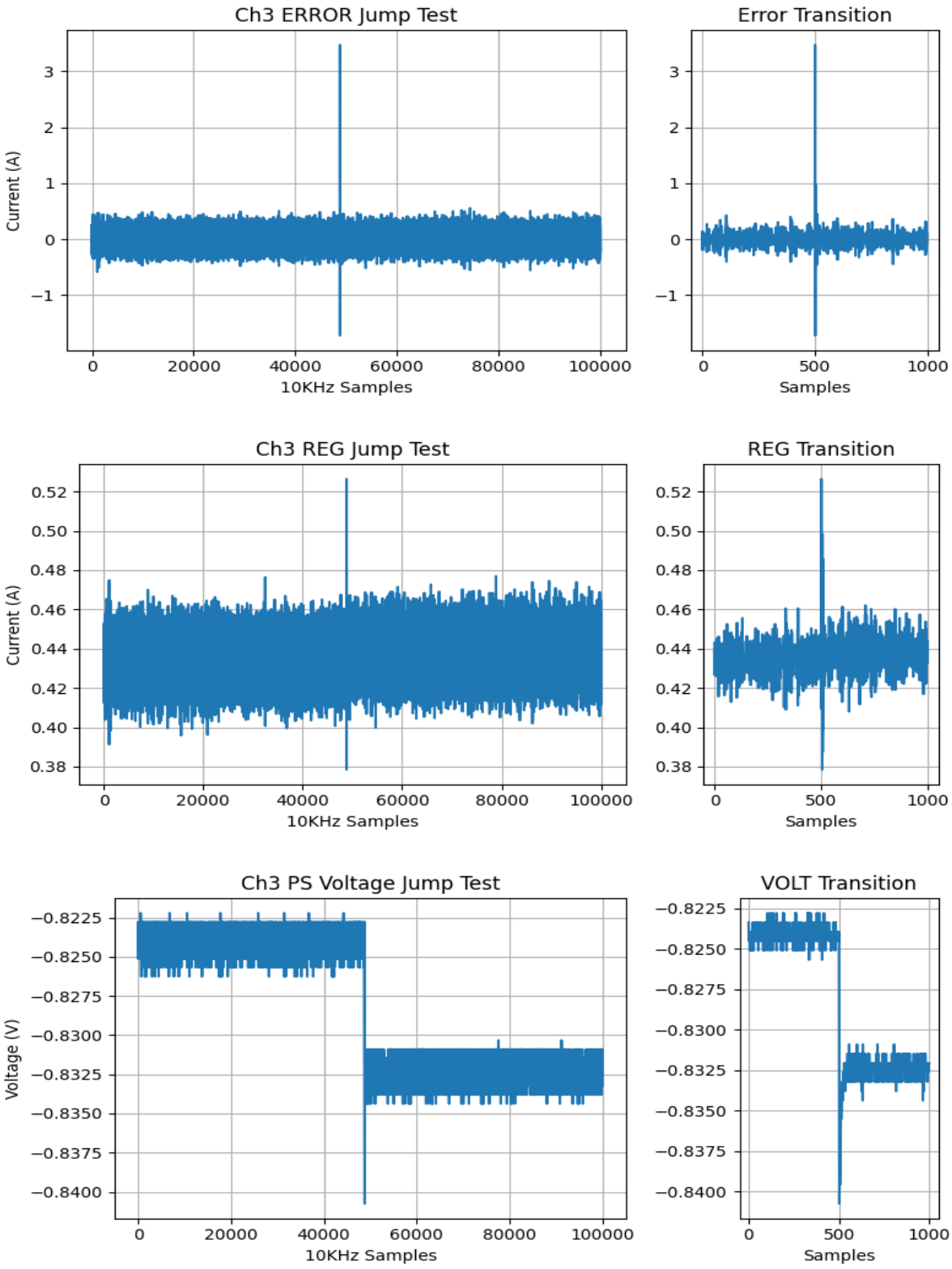




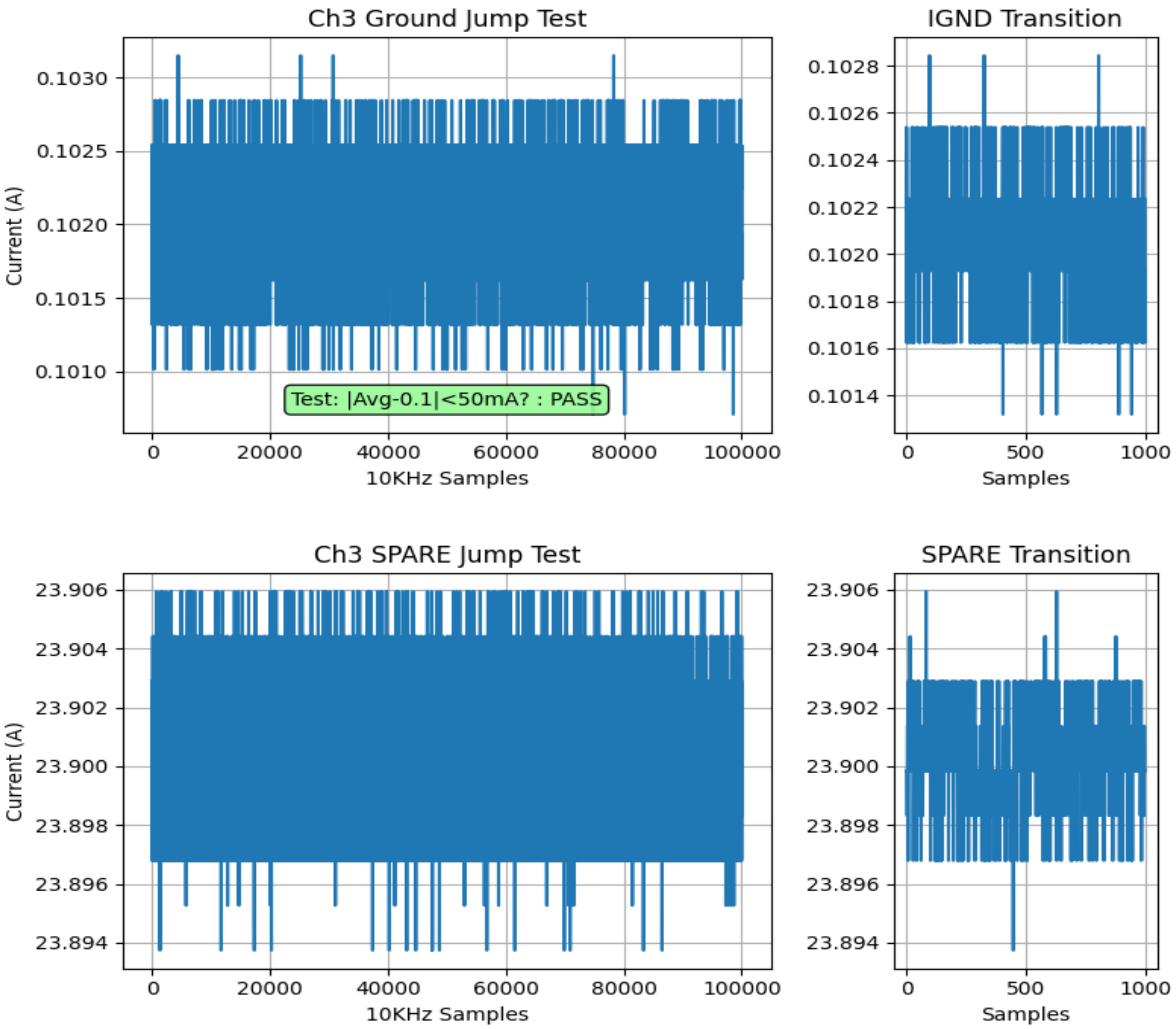
Jump Test Results: CH3



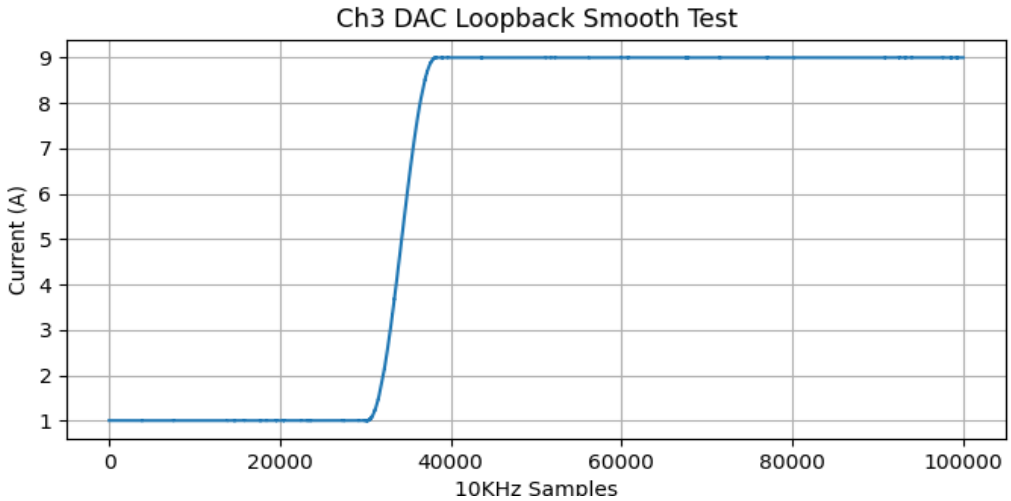
Jump Test Results: CH3

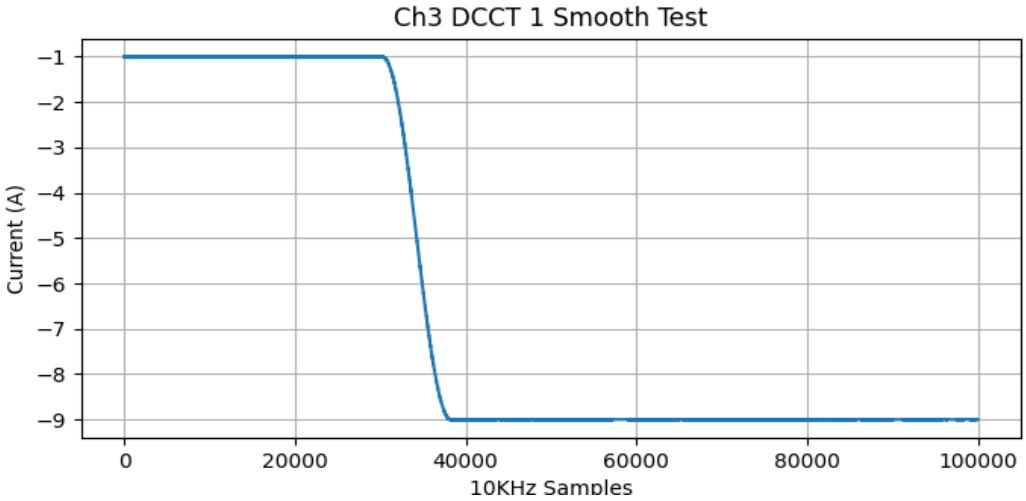


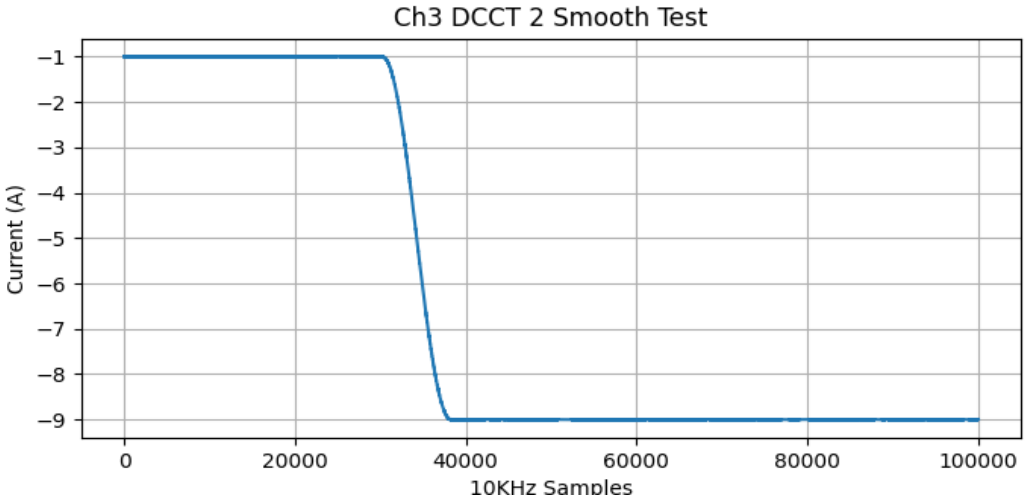
Jump Test Results: CH3



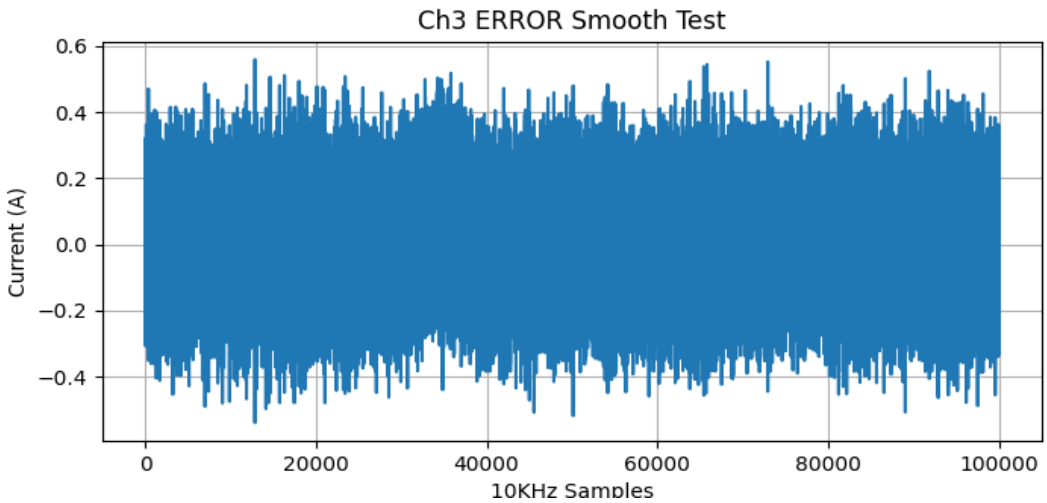
Smooth Test Results: CH3

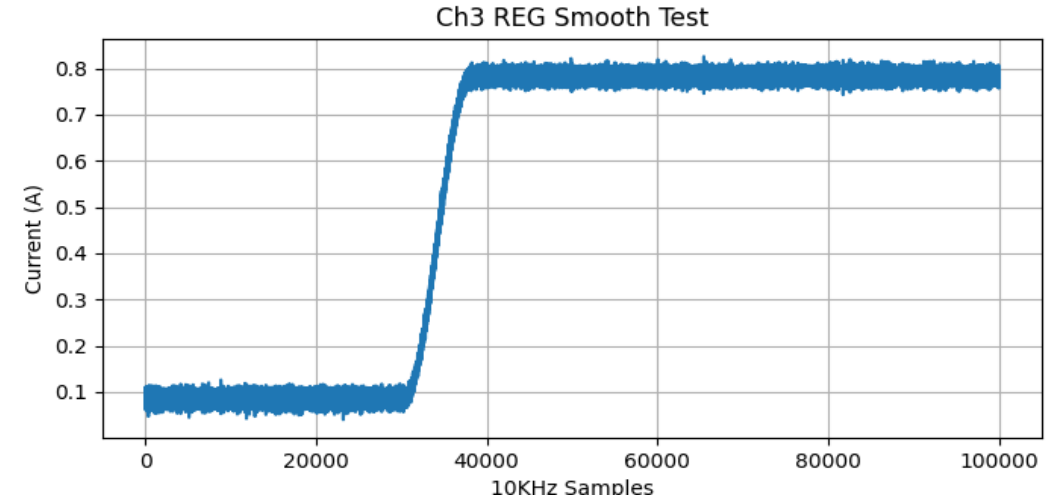


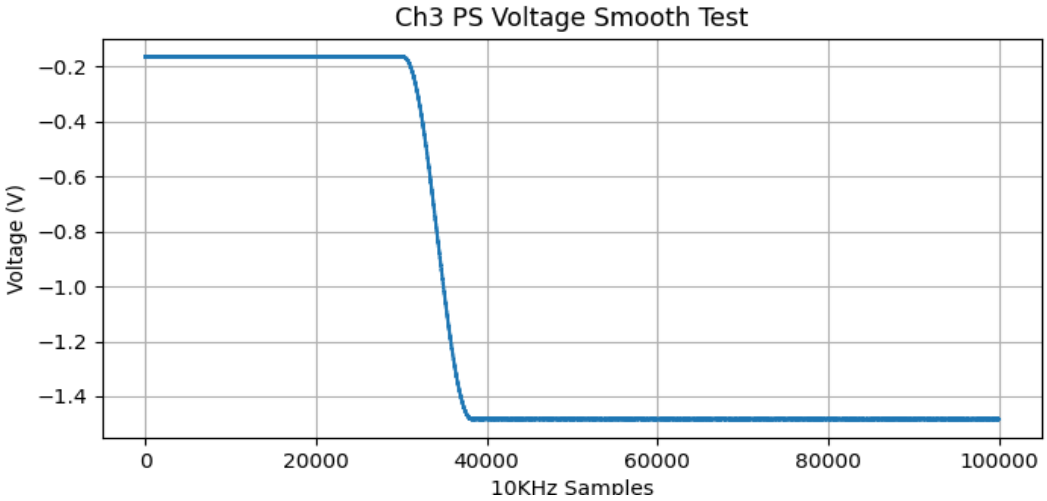




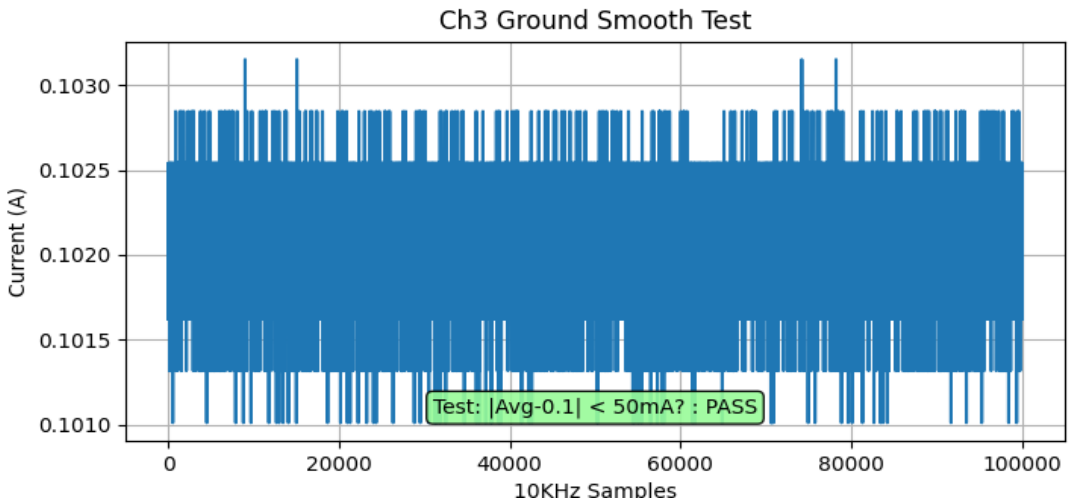
Smooth Test Results: CH3

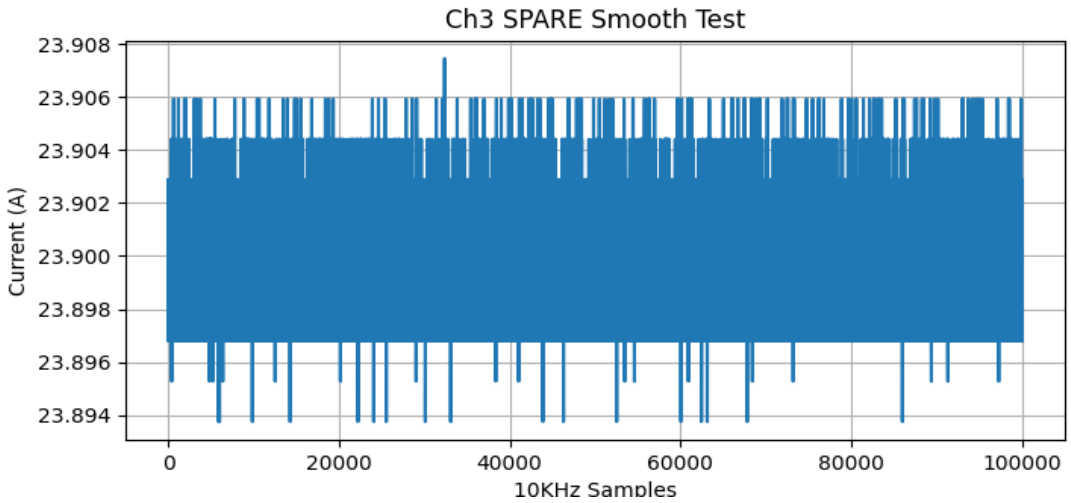






Smooth Test Results: CH3

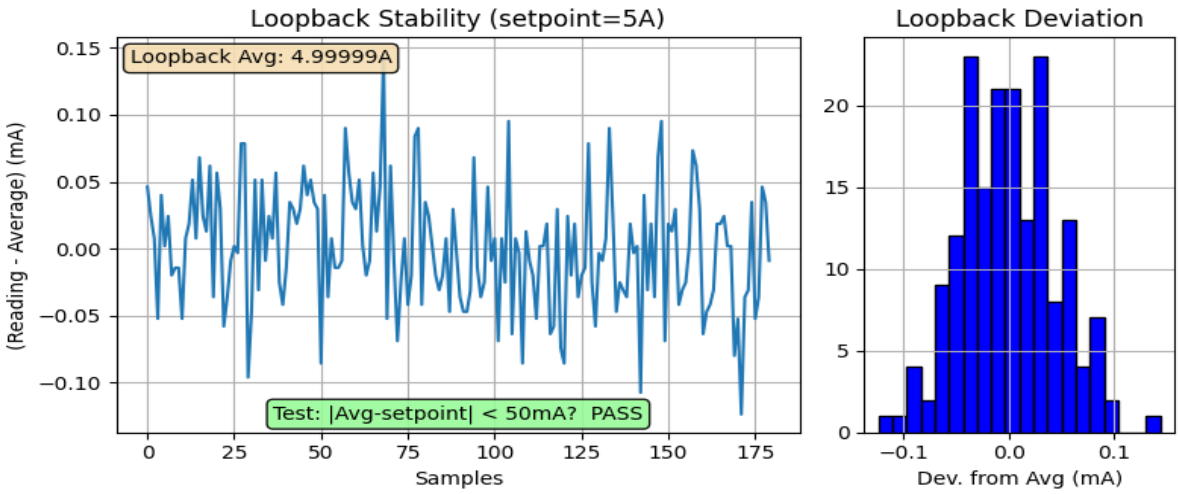


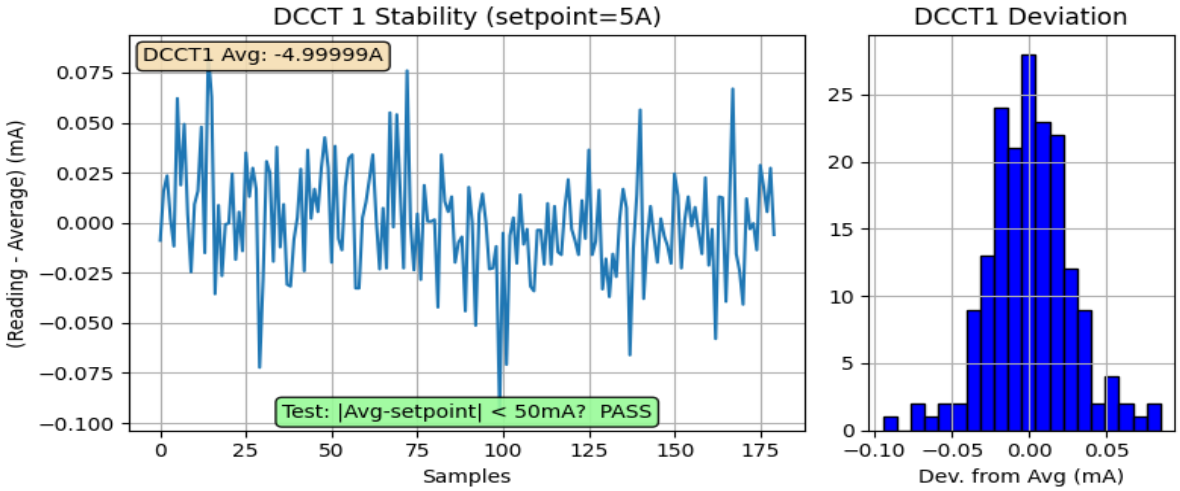


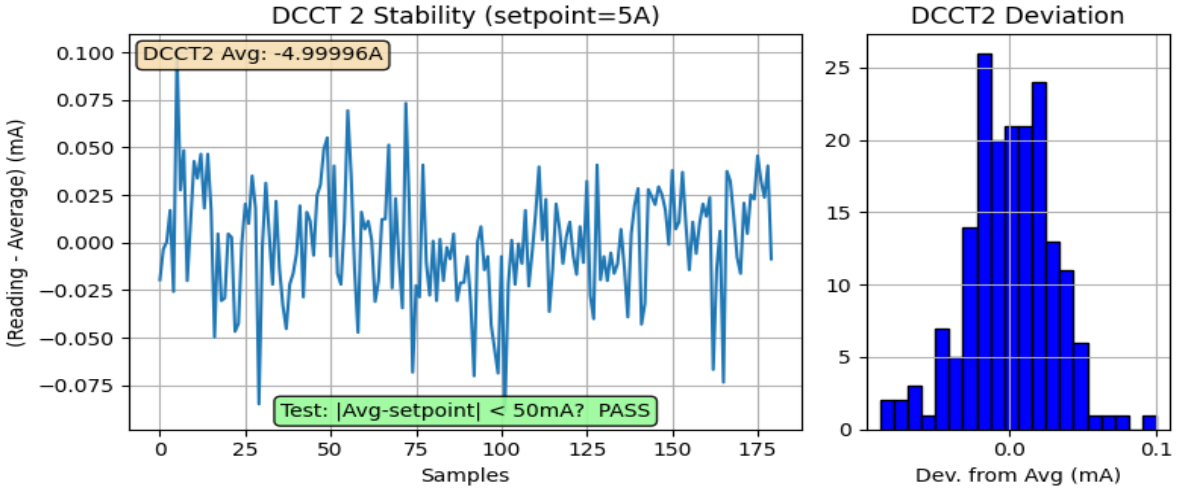
Channel 4

ATE Fault Tests for Channel 4	
Fault #1 Generated and Cleared	PASS
Fault #2 Generated and Cleared	PASS
Fault SPARE Generated and Cleared	PASS
Fault DCCT Generated and Cleared	PASS

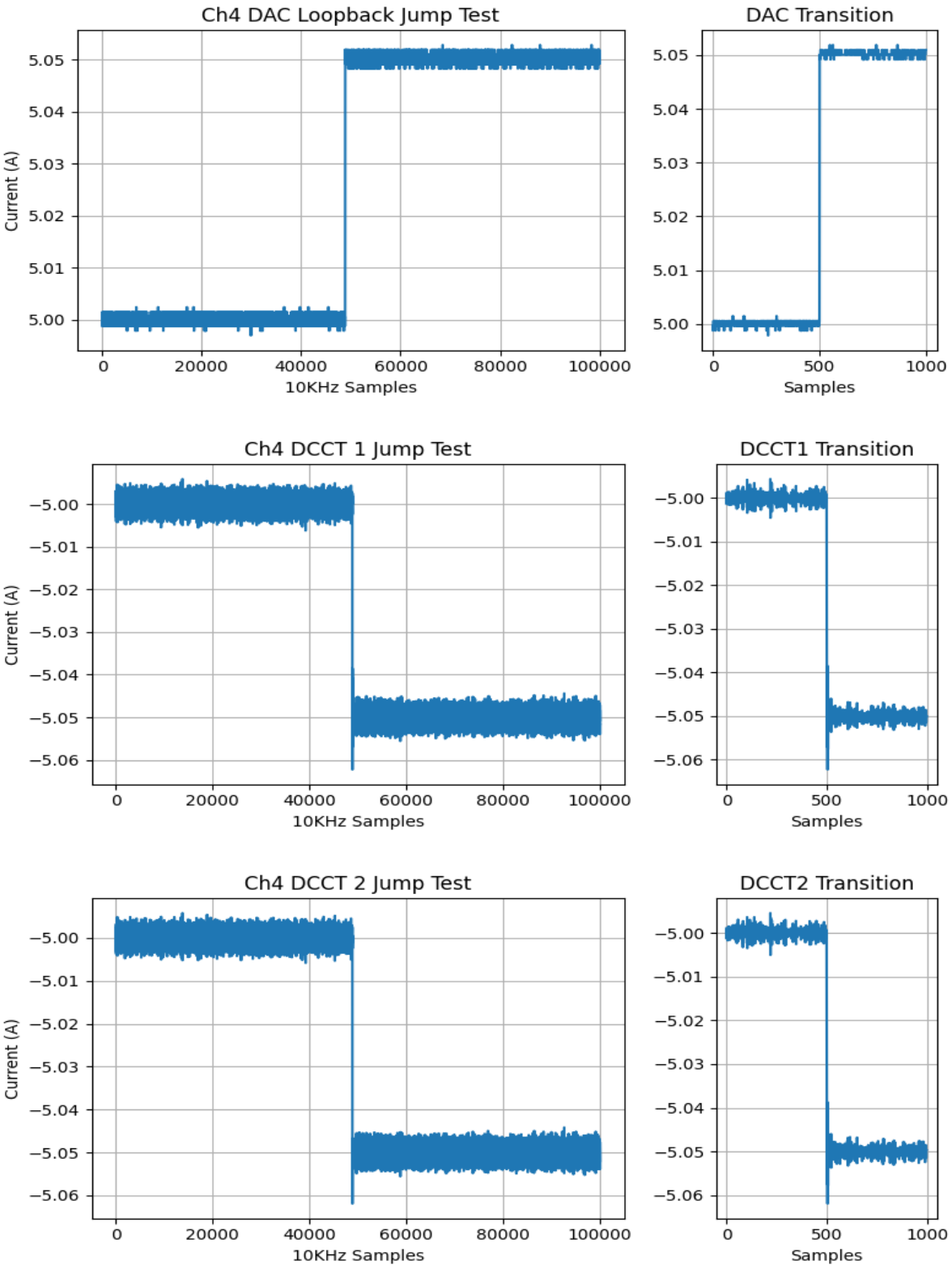
Power Supply Regulation for Channel 4:



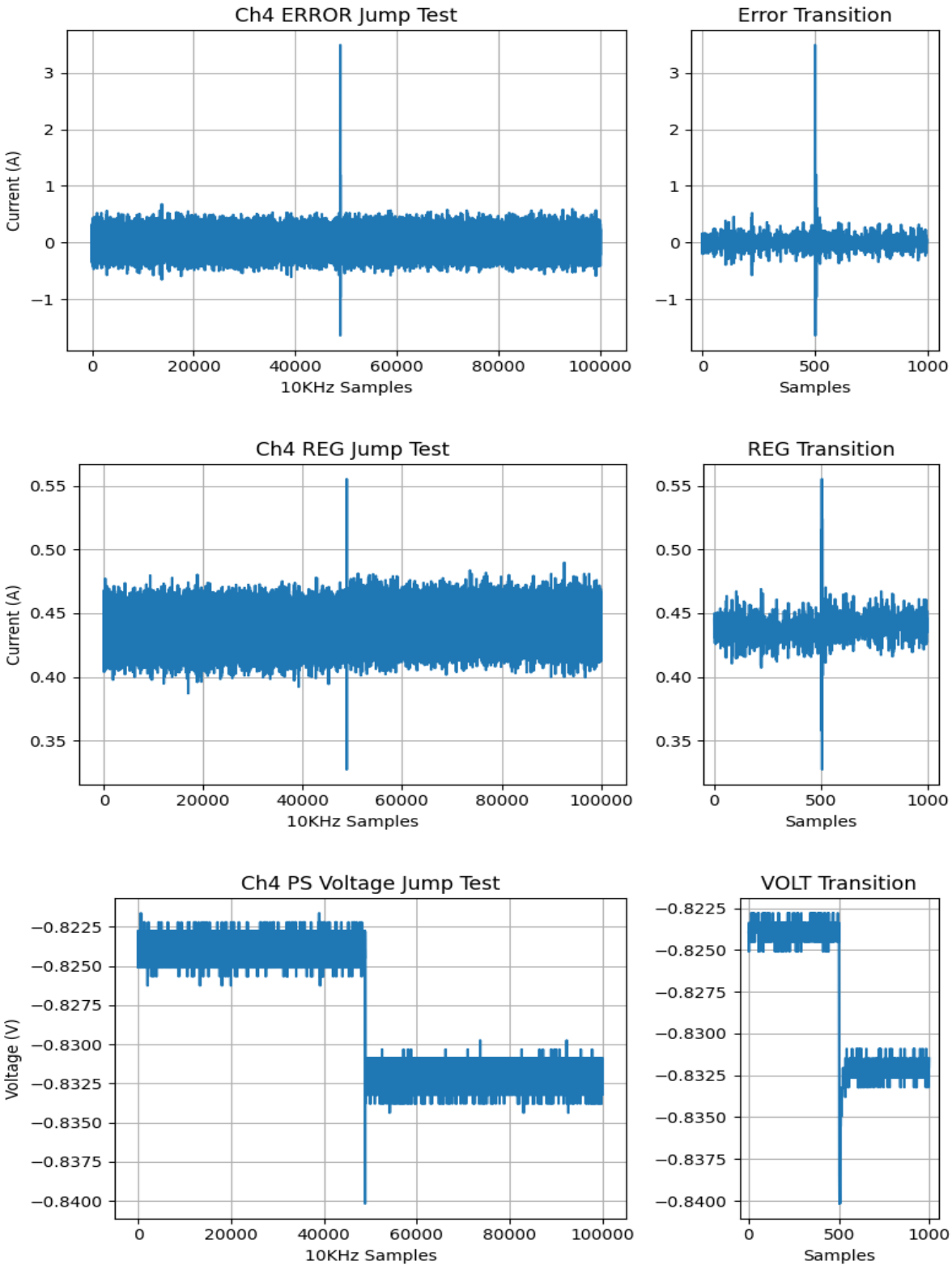




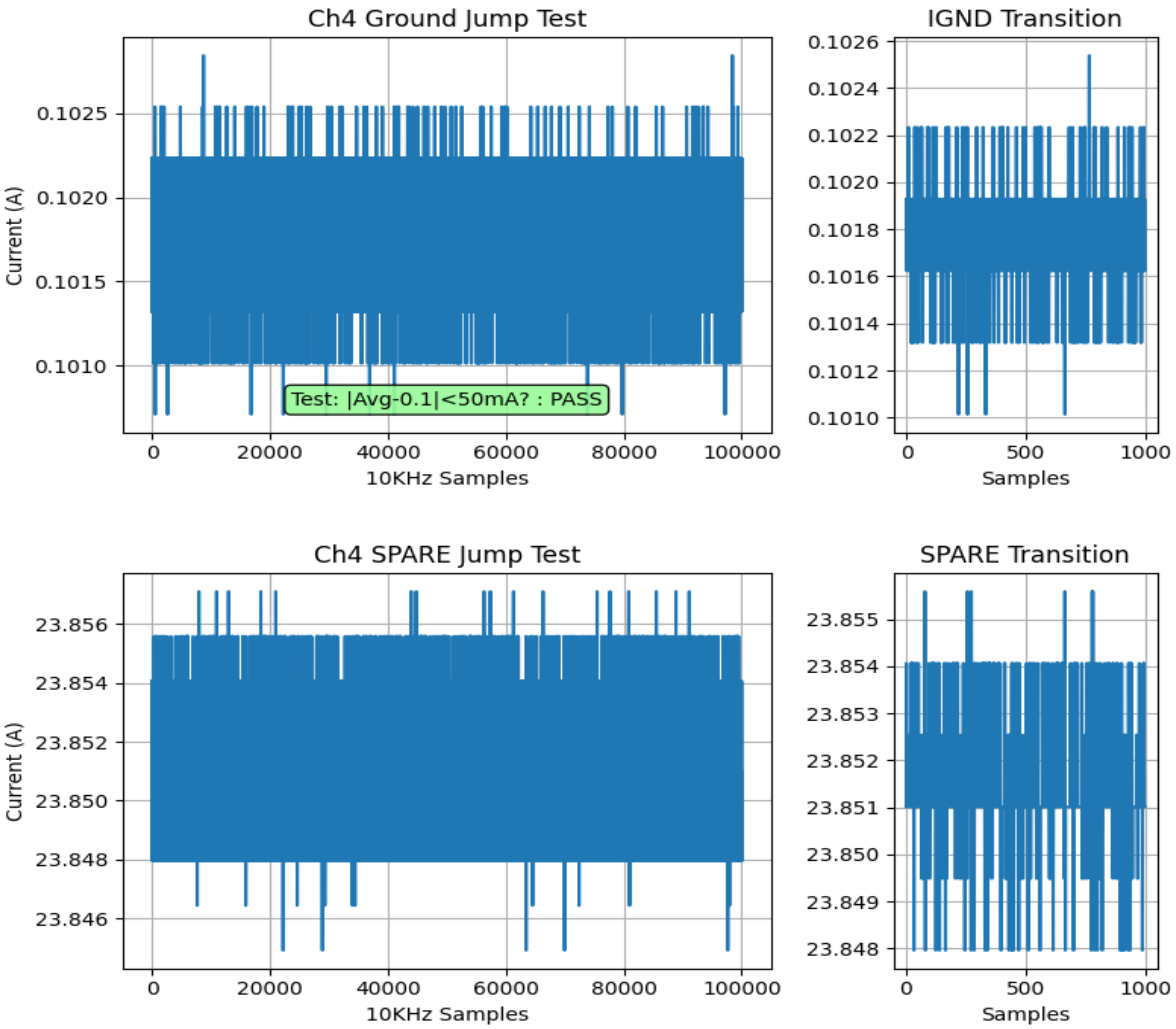
Jump Test Results: CH4



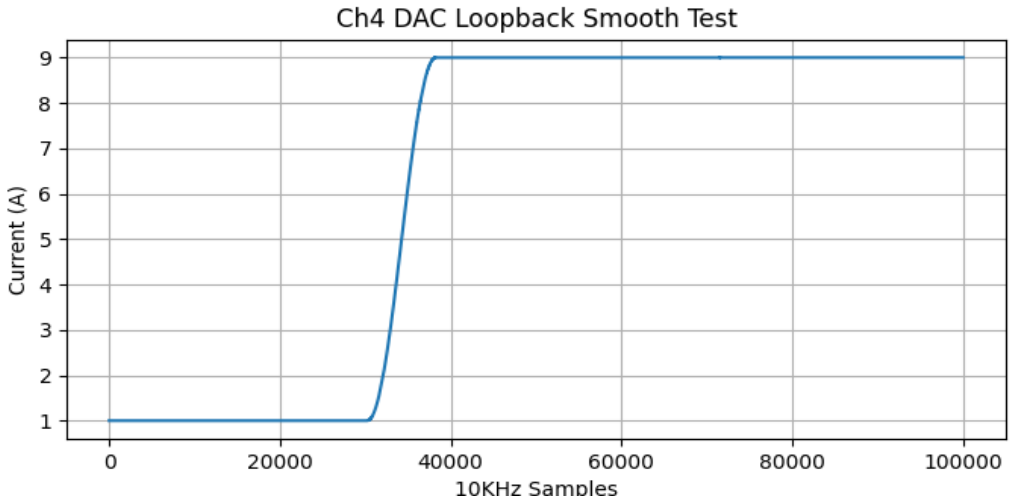
Jump Test Results: CH4

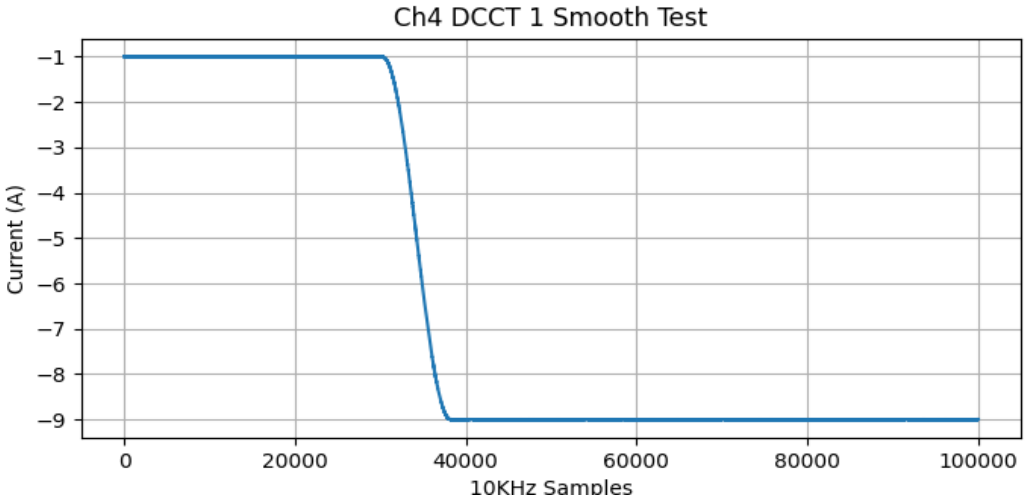


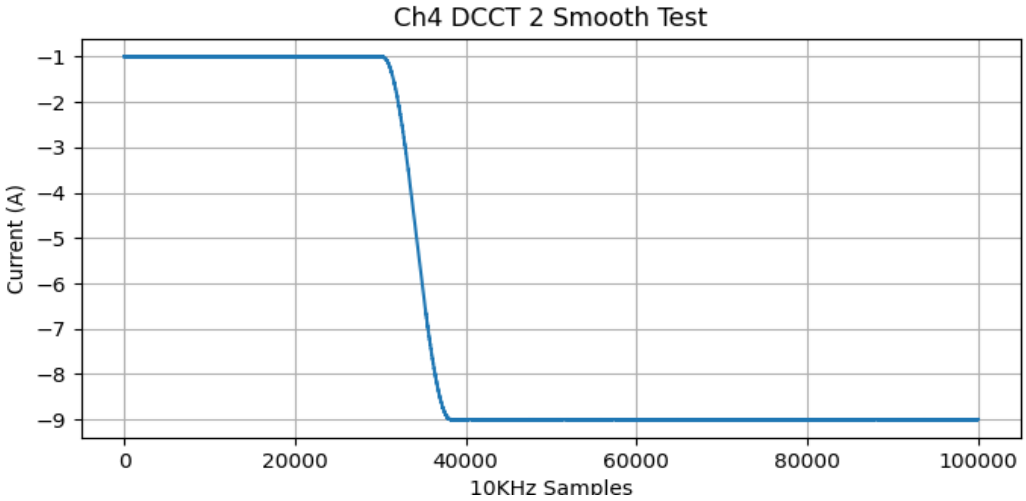
Jump Test Results: CH4



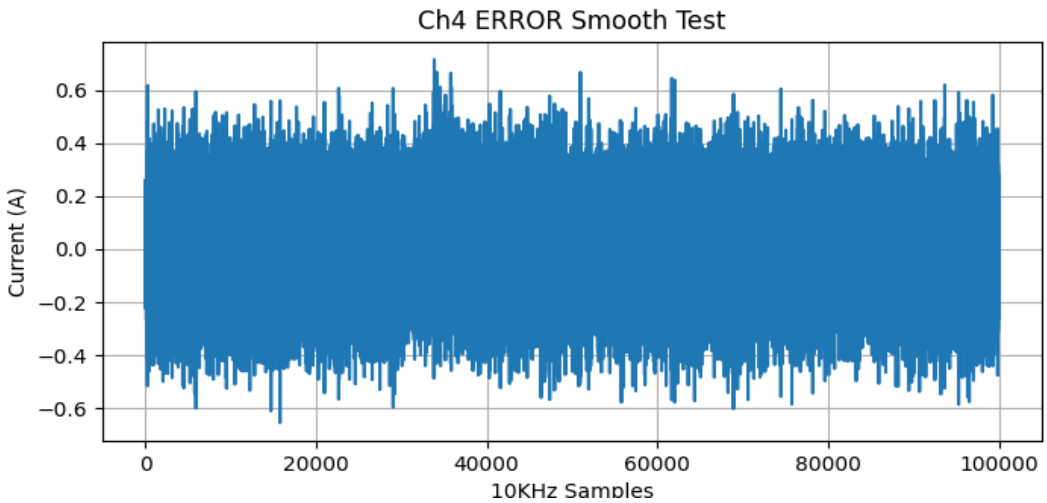
Smooth Test Results: CH4

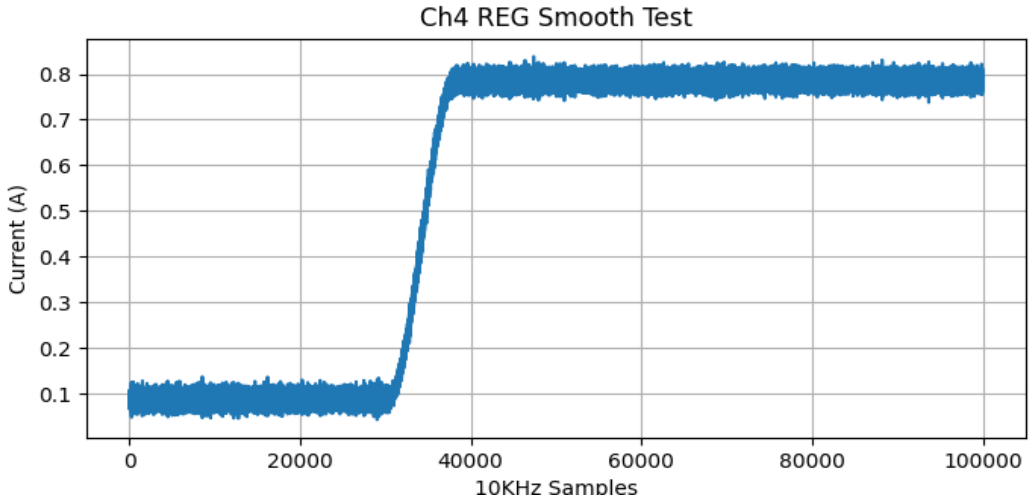


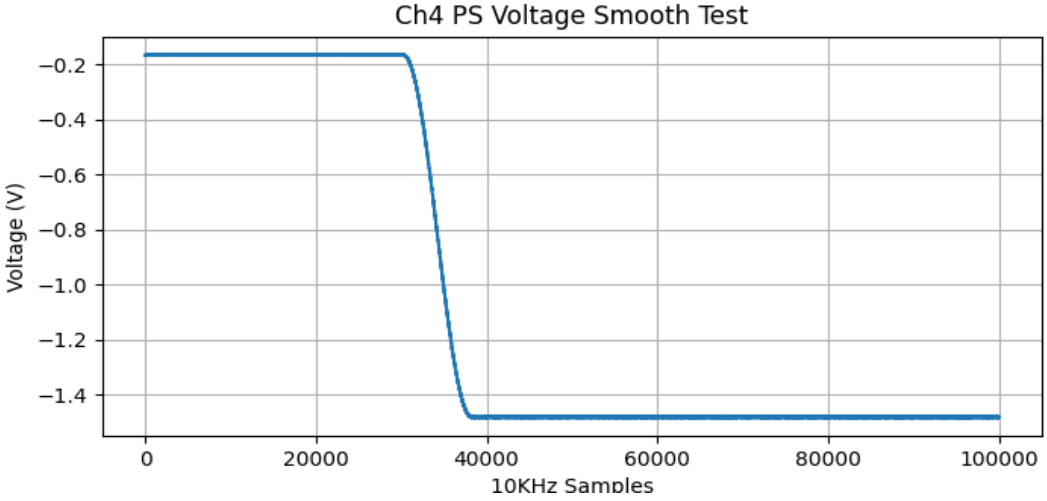




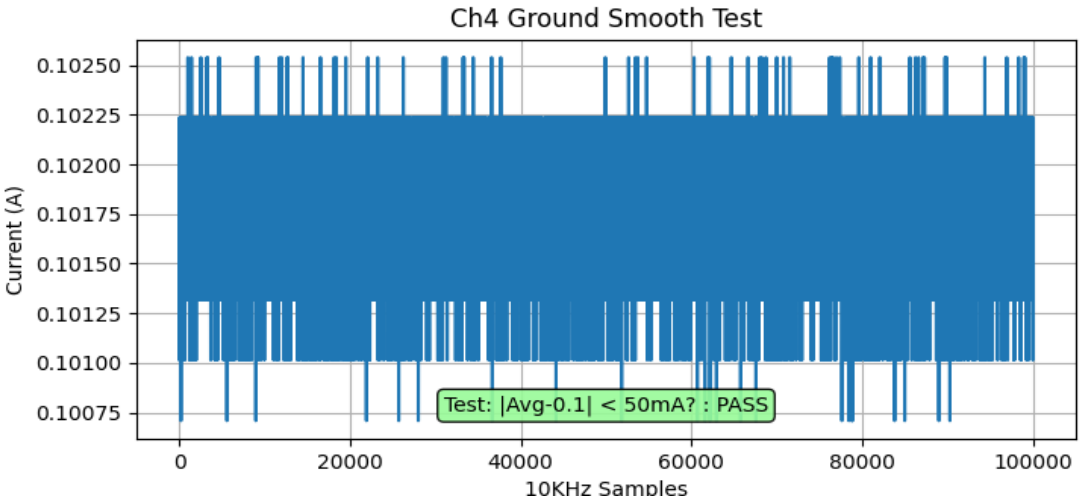
Smooth Test Results: CH4

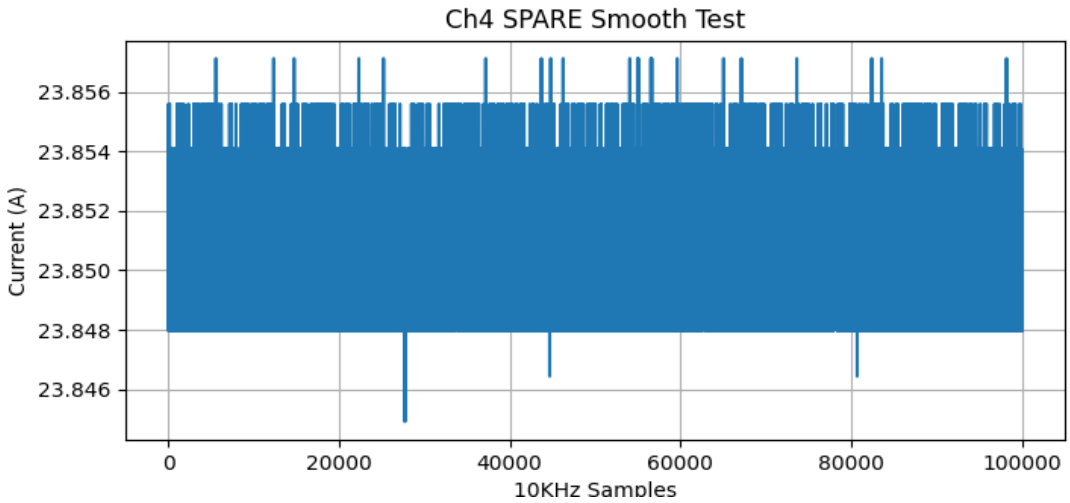






Smooth Test Results: CH4





FOFB TX test (Bandwidth-Mode = Fast)

PV	Value	Pass?
lab{2}Chan1:DAC-I	11.4988431930542	PASS
lab{2}Chan2:DAC-I	11.50019645690918	PASS
lab{2}Chan3:DAC-I	11.500794410705566	PASS
lab{2}Chan4:DAC-I	11.499785423278809	PASS

UDP RX Packet Test

```
23:18:00.587370 IP (tos 0x0, ttl 64, id 57082, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.50654 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.587371 IP (tos 0x0, ttl 64, id 57083, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.49639 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.598781 IP (tos 0x0, ttl 64, id 57084, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.55376 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.598781 IP (tos 0x0, ttl 64, id 57085, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.41305 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.610654 IP (tos 0x0, ttl 64, id 57087, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.55951 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.610655 IP (tos 0x0, ttl 64, id 57088, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.60133 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.622344 IP (tos 0x0, ttl 64, id 57093, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.55128 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.622344 IP (tos 0x0, ttl 64, id 57094, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.42697 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.633813 IP (tos 0x0, ttl 64, id 57095, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.36116 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.633814 IP (tos 0x0, ttl 64, id 57096, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.36741 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.645485 IP (tos 0x0, ttl 64, id 57103, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.54688 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.645486 IP (tos 0x0, ttl 64, id 57104, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.37310 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.656951 IP (tos 0x0, ttl 64, id 57110, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.50865 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.656951 IP (tos 0x0, ttl 64, id 57111, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.39005 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.668238 IP (tos 0x0, ttl 64, id 57120, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.34493 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.668239 IP (tos 0x0, ttl 64, id 57121, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.53892 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.679692 IP (tos 0x0, ttl 64, id 57127, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.37693 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.679693 IP (tos 0x0, ttl 64, id 57128, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.47450 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.690998 IP (tos 0x0, ttl 64, id 57140, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.35218 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
23:18:00.690998 IP (tos 0x0, ttl 64, id 57141, offset 0, flags [DF], proto UDP (17), length 64)
    10.69.26.1.58880 > 10.69.26.55.12345: [udp sum ok] UDP, length 36
```

(Full tcpdump log saved to /home/pstester/NSLS-II_PSCs/NEXT-II/NSLS2_zPSC_Calibration_and_Test/Test_Data/Raw_Logs/4ch_HSF_SN0001_RawData_04-30-26_22-45/tcpdump_full.log)

Test	Result
UDP RX FOFB	PASS